

TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA CÔNG NGHỆ THÔNG TIN



ĐỒ ÁN LẬP TRÌNH TÍNH TOÁN

**Xây dựng công cụ hỗ trợ tra cứu Kanji tích hợp
số hóa giáo trình 512 Kanji Look And Learn**

Người hướng dẫn: **TS. TRƯỜNG NGỌC CHÂU**

Sinh viên thực hiện:

Nguyễn Hoàng Sơn

LỚP: 25T_NHAT1

Nguyễn Văn Tây

LỚP: 25T_NHAT1

Đà Nẵng, 06/2026

MỤC LỤC

MỤC LỤC.....	1
DANH MỤC HÌNH VẼ.....	3
DANH MỤC BẢNG.....	4
MỞ ĐẦU.....	1
1. TỔNG QUAN ĐỀ TÀI.....	2
1.1. Lý do chọn đề tài.....	2
1.2. Mục tiêu đề tài.....	3
2. CƠ SỞ LÝ THUYẾT	4
2.1. Ý tưởng.....	4
2.2. Cơ sở lý thuyết	7
2.2.1. Kiến thức nền tảng	7
2.2.2. Công nghệ, ngôn ngữ và công cụ phát triển.....	11
3. TỔ CHỨC CẤU TRÚC DỮ LIỆU VÀ THUẬT TOÁN.....	12
3.1. Phát biểu bài toán	12
3.1.1. Nạp và phân giải dữ liệu từ tệp cấu trúc JSON	12
3.1.2. Quản lý và hiển thị dữ liệu phân hoạch theo bài học	12
3.1.3. Tìm kiếm chính xác dựa trên cơ chế phân băm	12
3.1.4. Tìm kiếm tiền tố ứng dụng cấu trúc cây Trie	13
3.1.5. Tìm kiếm theo chuỗi con bằng thuật toán KMP	13
3.1.6. Tìm kiếm mờ tính toán khoảng cách Levenshtein	13
3.1.7. Lọc ra danh sách từ vựng chỉ chứa các Kanji đã học.....	14
3.1.8. Phân tích các Kanji trong câu tiếng Nhật.....	14
3.2. Cấu trúc dữ liệu	15
3.2.1. Cấu trúc tệp tin dữ liệu JSON (kanjiData.json)	15
3.2.2. Cấu trúc dữ liệu lỗi.....	17

3.2.3.	Cấu trúc dữ liệu cho chức năng tra cứu.....	19
3.2.4.	Cấu trúc dữ liệu cho chức năng lọc từ vựng đã học.....	21
3.2.5.	Cấu trúc dữ liệu cho chức năng phân tích câu	22
3.3.	Thuật toán.....	22
3.3.1.	Thuật toán băm (Hash) cho tìm kiếm chính xác	22
3.3.2.	Thuật toán tìm kiếm mờ Levenshtein Distance.....	26
3.3.3.	Thuật toán tìm kiếm chuỗi con KMP (Knuth-Morris-Pratt)	28
3.3.4.	Thuật toán ứng dụng cây tiền tố cho tìm kiếm Romaji	31
3.3.5.	Thuật toán lọc từ vựng dựa trên Kanji đã học.....	34
3.3.6.	Thuật toán phân tích các Kanji trong câu.....	37
4.	CHƯƠNG TRÌNH VÀ KẾT QUẢ.....	40
4.1.	Tổ chức chương trình	40
4.1.1.	Cấu trúc thư mục dự án	40
4.1.2.	Tổ chức các thành phần chức năng	41
4.2.	Ngôn ngữ cài đặt	43
4.2.1.	Ngôn ngữ, thư viện và kỹ thuật cài đặt	43
4.2.2.	Hướng dẫn biên dịch và chạy chương trình	44
4.3.	Kết quả	45
4.3.1.	Giao diện chính của chương trình	45
4.3.2.	Kết quả thực thi của chương trình	47
4.3.3.	Nhận xét đánh giá.....	53
5.	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	54
5.1.	Kết luận	54
5.2.	Hướng phát triển.....	54
	TÀI LIỆU THAM KHẢO.....	56
	PHỤ LỤC.....	59

DANH MỤC HÌNH VẼ

Hình 3.1: Tổ chức dữ liệu file kanjiData.json.....	16
Hình 3.2: Sơ đồ quan hệ giữa các cấu trúc dữ liệu lỗi.....	19
Hình 4.1: Sơ đồ cấu trúc thư mục dự án	40
Hình 4.2: Sơ đồ tổ chức các thành phần chức năng (Giao diện chính).....	41
Hình 4.3: Sơ đồ cấu trúc các thuật toán tìm kiếm.....	41
Hình 4.4: Sơ đồ logic chức năng hiển thị và quản lý thông tin từ điển	42
Hình 4.5: Sơ đồ cách hoạt của chức năng lọc từ vựng.....	42
Hình 4.6: Sơ đồ tổng quan các logic xử lý chức năng phân tích câu.....	43
Hình 4.7: Giao diện chính kèm danh sách các chức năng	45
Hình 4.8: 04 phương thức tìm kiếm.....	45
Hình 4.9: Giao diện chức năng tìm kiếm chính xác (Hashtable).....	45
Hình 4.10: Giao diện các lựa chọn chức năng tra cứu theo chuỗi con (KMP Algorithm).....	46
Hình 4.11: Giao diện chức năng tìm kiếm mờ (Levenshtein Distance)	46
Hình 4.12: Giao diện chức năng tìm kiếm theo tiền tố (Trie).....	46
Hình 4.13: Giao diện quản lý toàn bộ kanji và các bài học	47
Hình 4.14: Giao diện kết quả tìm kiếm chính xác với Kanji và Hán Việt.....	47
Hình 4.15: Giao diện kết quả tìm kiếm chính xác với từ vựng Kanji, Furigana, Romaji và Tiếng Việt.....	48
Hình 4.16: Giao diện kết quả tìm kiếm theo chuỗi con trên các trường Kanji, Romaji, Furigana, Tiếng Việt	48
Hình 4.17: Giao diện kết quả tìm kiếm mờ với từ khóa Kanji, Furigana và Romaji	49
Hình 4.18: Giao diện kết quả tìm kiếm theo tiền tố.....	49
Hình 4.19: Giao diện hiển thị danh sách kanji các bài học.....	50
Hình 4.20: Giao diện hiển thị chi tiết nội dung từng Kanji	50
Hình 4.21: Giao diện kết quả lọc từ vựng các bài thỏa mãn điều kiện	51
Hình 4.22: Giao diện kết quả tìm thông tin của các Kanji trong câu.....	52

DANH MỤC BẢNG

Bảng 2.1: Tổng hợp các vấn đề thực tế và giải pháp đề xuất	5
Bảng 2.2: Các hàm cJSON sử dụng trong dự án.....	8
Bảng 3.1: Bảng so sánh nhanh các tiêu chí của 04 thuật toán tìm kiếm.....	33

MỞ ĐẦU

Trong xu thế toàn cầu hóa và sự hợp tác ngày càng phát triển giữa Việt Nam và Nhật Bản trong lĩnh vực Công nghệ thông tin, việc sở hữu năng lực tiếng Nhật chuyên ngành đang trở thành một lợi thế cạnh tranh chiến lược đối với nguồn nhân lực kỹ thuật tương lai. Ngôn ngữ này không chỉ là phương tiện giao tiếp thông thường mà còn là chìa khóa then chốt giúp người học tiếp cận trực tiếp với các công nghệ tiên tiến, quy trình làm việc chuẩn mực và văn hóa doanh nghiệp Nhật Bản – một trong những đối tác chiến lược hàng đầu của Việt Nam.

Hệ thống chữ viết tiếng Nhật bao gồm ba bộ chữ chính: Hiragana, Katakana và Kanji. Trong đó, Kanji – còn gọi là chữ Hán Nhật, có nguồn gốc từ Trung Quốc – là hệ thống chữ biểu ý (logographic), mỗi ký tự mang trong mình một ý nghĩa độc lập, được thể hiện qua cấu trúc hình khối phức tạp, nhiều nét vẽ và tầng nghĩa đa dạng [1]. Chính vì đặc điểm này, Kanji được xem là phần kiến thức khó nhất nhưng cũng là nền tảng cốt lõi để người học có thể tiếp cận tiếng Nhật ở mức độ chuyên sâu, đặc biệt trong việc đọc hiểu các tài liệu kỹ thuật và chuyên ngành.

Mặc dù giữ vai trò quyết định, việc học và tra cứu Kanji vẫn là một thách thức lớn. Phương pháp tiếp cận truyền thống dựa trên sách giấy khiến người học mất nhiều thời gian tra cứu thủ công, thiếu tính linh hoạt và hạn chế khả năng tương tác trực quan. Trong khi đó, các ứng dụng hỗ trợ hiện hành thường mang tính phổ thông, chưa gắn kết chặt chẽ với cấu trúc bài học của từng giáo trình cụ thể, dẫn đến việc khó đáp ứng nhu cầu thực tế phát sinh của người học.

Nhằm khắc phục những hạn chế kể trên, đề tài ***“Xây dựng công cụ hỗ trợ tra cứu Kanji (Hán tự) tích hợp số hóa giáo trình 512 Kanji Look And Learn”*** được thực hiện với mục tiêu xây dựng một chương trình dòng lệnh bằng ngôn ngữ lập trình C. Công cụ này tập trung hỗ trợ tra cứu nhanh chóng nhờ tích hợp các thuật toán tìm kiếm đa dạng, quản lý dữ liệu một cách khoa học và cung cấp tính năng luyện tập trắc nghiệm hiệu quả. Cấu trúc và chức năng của sản phẩm được chuẩn hóa nhằm đáp ứng tối ưu lộ trình học tập đặc thù trong khuôn khổ chương trình đào tạo chiến lược Xseeds do công ty Sun Asterisk triển khai.

1. TỔNG QUAN ĐỀ TÀI

1.1. Lý do chọn đề tài

Bối cảnh thực tế

Năm 2006, một chương trình đào tạo được khởi xướng thông qua sự hợp tác giữa Đại học Bách khoa Hà Nội (Hanoi University of Science and Technology - HUST) và JICA (Japan International Cooperation Agency) – cơ quan thuộc Bộ Ngoại giao Nhật Bản, dưới hình thức dự án Hỗ trợ Phát triển Chính thức (ODA). Dự án mang tên "HEDSPI" được thiết lập như một ngành học chính thức trong trường đại học, đặt ra mục tiêu chiến lược là đào tạo nguồn nhân lực Công nghệ Thông tin chất lượng cao có khả năng sử dụng thành thạo tiếng Nhật.

Năm 2014, Công ty Sun Asterisk chính thức tiếp nhận và phát triển chương trình. Dựa trên nền tảng này, Sun* đã xây dựng lại chương trình giảng dạy, cập nhật giáo trình phù hợp với nhu cầu thực tiễn của doanh nghiệp, đồng thời mở rộng quy mô đến nhiều trường đại học khác trên toàn quốc. Đến năm 2021, dự án được tái định vị thương hiệu với tên gọi mới là "Xseeds" – một chương trình giáo dục toàn cầu nhằm phát hiện, đào tạo và kết nối những "hạt giống tài năng" với cơ hội nghề nghiệp tại Nhật Bản [2]. Hiện nay, chương trình Xseeds đang hợp tác với 6 trường đại học hàng đầu tại châu Á, cung cấp các khóa học tiếng Nhật kết hợp kiến thức CNTT thực tiễn cho khoảng 1.800 sinh viên mỗi năm, hướng tới mục tiêu chuẩn bị nguồn nhân lực đáp ứng tiêu chuẩn khắt khe của các doanh nghiệp Nhật Bản.

Trong bối cảnh đó, Công ty Sun Asterisk đã chính thức triển khai chương trình đào tạo tiếng Nhật chuyên sâu dành riêng cho sinh viên ngành Công nghệ Thông tin tại bốn trường đại học kỹ thuật hàng đầu Việt Nam gồm: Đại học Bách khoa Hà Nội (HUST), Đại học Bách khoa – Đại học Đà Nẵng (DUT), Đại học Bách khoa – ĐHQG TP. Hồ Chí Minh (HCMUT) và Đại học Công nghệ – ĐHQG Hà Nội (VNU-UET) [3].

Trong khuôn khổ dự án, giáo trình 512 Kanji Look and Learn của Nhà xuất bản The Japan Times (gồm 512 chữ Kanji mức độ từ sơ cấp đến trung cấp) được lựa chọn làm tài liệu cốt lõi [4]. Giáo trình này đồng hành cùng sinh viên từ giai đoạn nhập môn cho đến các học phần chuyên sâu, đóng vai trò quan trọng trong việc xây dựng nền tảng Kanji vững chắc – một kỹ năng then chốt đối với sinh viên CNTT có định hướng làm việc tại thị trường Nhật Bản.

Hạn chế

Mặc dù giáo trình 512 Kanji Look and Learn có chất lượng nội dung tốt, song hình thức học liệu truyền thống vẫn tồn tại nhiều hạn chế lớn. Sách in đòi hỏi

người học phải tra cứu thủ công bằng cách lật tìm nhiều trang, thiếu tính linh hoạt và không hỗ trợ tìm kiếm theo đa tiêu chí. Bên cạnh đó, sách giấy khó tích hợp các bài tập trắc nghiệm tương tác cũng như tính năng lọc từ vựng theo tiến độ học tập thực tế của sinh viên.

Các ứng dụng tra cứu Kanji hiện có trên thị trường (web và mobile) cũng chưa đáp ứng được yêu cầu chuyên biệt. Hầu hết các ứng dụng này không bám sát cấu trúc bài học của giáo trình cụ thể, thiếu chức năng tra cứu nhanh, không cho phép tùy biến dữ liệu hoặc thường đưa ra nội dung vượt quá trình độ của người học. Đặc biệt, chưa có một giải pháp nào được thiết kế xuất phát từ nhu cầu thực tế của học viên tham gia chương trình Xseeds.

Giải pháp

Xuất phát từ những hạn chế trên, đề tài này được thực hiện với mục tiêu xây dựng một phiên bản kỹ thuật số chuẩn mực trên nền tảng ngôn ngữ C, hướng đến việc khắc phục toàn diện những bất cập của học liệu truyền thống, đồng thời cung cấp một công cụ học tập hiện đại, hiệu năng cao và được thiết kế đặc thù để đồng bộ chặt chẽ với hệ sinh thái học liệu của chương trình Xseeds.

Cụ thể, đề tài tập trung số hóa toàn bộ 32 bài học với 512 chữ Kanji và hơn 3.500 từ vựng dưới dạng tệp JSON có cấu trúc. Trên nền tảng dữ liệu này, các cấu trúc dữ liệu và thuật toán tìm kiếm chuyên biệt được tích hợp để xây dựng một ứng dụng dòng lệnh (Command Line Application) chạy ổn định trên hệ điều hành Windows. Ứng dụng đóng vai trò là phiên bản hỗ trợ độc quyền, giúp người học tra cứu, quản lý, luyện tập và ôn tập Kanji một cách hệ thống, khoa học và tối ưu nhất.

1.2. Mục tiêu đề tài

Xây dựng thành công một công tra cứu và quản lý Hán tự trên nền tảng ngôn ngữ C, vận hành ổn định trên hệ điều hành Windows, hỗ trợ người học tiếng Nhật (đặc biệt sinh viên trong chương trình Xseeds) tra cứu, luyện tập và ôn tập 512 chữ Kanji theo giáo trình Kanji Look and Learn một cách hệ thống, khoa học và có hiệu năng cao.

1) Nghiên cứu và làm chủ các kiến thức cốt lõi:

- Cấu trúc dữ liệu: cấu trúc lồng ghép (nested structs). bảng băm (hash table) với hàm băm djb2 và xử lý xung đột bằng phương pháp chaining, cây tiền tố (trie), danh sách liên kết (linked list), mảng một chiều, mảng hai chiều và mảng động

- Thuật toán: tìm kiếm chính xác bằng hàm băm (hashing), tìm kiếm tiền tố bằng cây Trie, tìm kiếm chuỗi con bằng thuật toán Knuth–Morris–Pratt (KMP), tìm kiếm mờ dựa trên khoảng cách Levenshtein.
 - Định dạng dữ liệu JSON, thư viện cJSON, JSON Parsing (chuyển đổi từ JSON Array thành cấu trúc struct C)
 - Phương pháp xử lý Unicode (UTF-8/UTF-16) trên Windows sử dụng Win32 API (ReadConsoleW, WideCharToMultiByte...). [5] [6] [7]
 - Giáo trình Kanji Look and Learn (512 chữ Kanji, 32 bài học) làm nguồn dữ liệu chính.
 - Xây dựng cơ sở dữ liệu Kanji dưới định dạng JSON, chứa đầy đủ thông tin của 512 ký tự, bao gồm: chữ Kanji, âm Onyomi, âm Kunyomi, phiên âm Hán Việt, bộ thủ, số nét, giải nghĩa, từ vựng liên quan và câu ví dụ song ngữ Nhật – Việt.
- 2) *Thiết kế và triển khai hoàn chỉnh các chức năng cốt lõi của sản phẩm:*
- Tra cứu chính xác Kanji và từ vựng bằng bảng băm (độ phức tạp $O(1)$).
 - Tìm kiếm tiền tố Romaji bằng cây Trie.
 - Tìm kiếm theo chuỗi con bằng thuật toán KMP.
 - Tìm kiếm mờ dựa trên khoảng cách Levenshtein (ngưỡng sai lệch ≤ 2).
 - Quản lý và duyệt chi tiết 32 bài học với 512 chữ Kanji.
 - Phân tích câu tiếng Nhật, chiết xuất và tra cứu tức thì các ký tự Kanji xuất hiện trong câu.
 - Tích hợp tính năng lọc từ vựng thông minh dựa trên tiến độ hoàn thành bài học, tự động loại bỏ từ vựng chứa Kanji chưa học để tối ưu hóa lộ trình học tập.

2. CƠ SỞ LÝ THUYẾT

2.1. Ý tưởng

Ý tưởng cốt lõi của đề tài xuất phát từ việc mô hình hóa hành vi nhận thức và quy trình học tập của sinh viên khi tiếp cận giáo trình Kanji Look and Learn. Trong quá trình tiếp thu kiến thức và thực hành, dòng tư duy của người học liên tục dịch chuyển giữa nhiều trạng thái tra cứu tùy theo ngữ cảnh cụ thể: từ nhu cầu truy xuất khi nhớ chính xác từ khóa, tìm kiếm gợi ý khi chỉ nhớ phân đoạn đầu, xử lý sai số khi gõ sai chính tả, cho đến việc bóc tách các ký tự xuất hiện trong một văn bản tổng hợp hay lọc các từ vựng theo nhu cầu thực tế. Các công cụ hiện hành (như các nền tảng

web hoặc công cụ di động đại trà) thường chưa tối ưu hóa cho các tình huống đầy biến động này, dẫn đến hiện tượng ngất mạch tư duy và làm giảm hiệu suất học tập.

Từ thực tiễn đó, đề tài định hướng xây dựng một chương trình dòng lệnh (Command Line Interface - CLI) bằng ngôn ngữ lập trình C, được thiết kế để tổ chức dữ liệu Kanji một cách hệ thống và phản hồi tức thì theo nhiều tiêu chí truy vấn. Công cụ này không chỉ hỗ trợ các phương thức tìm kiếm cốt lõi (tra cứu chính xác, tìm kiếm tiền tố, tìm theo chuỗi con, tra cứu mờ) mà còn tích hợp các phân hệ quản lý tiến độ bài học, phân tích cú pháp câu và sàng lọc từ vựng thông minh nhằm đáp ứng tối ưu lộ trình học tập cá nhân hóa.

Sự tương quan giữa các tình huống thực tế trong quá trình học và giải pháp kỹ thuật tương ứng được thể hiện chi tiết qua bảng đối chiếu dưới đây:

Bảng 2.1: Tổng hợp các vấn đề thực tế và giải pháp đề xuất

STT	Vấn đề thực tế	Mô tả kịch bản tương tác	Chức năng cần phát triển	Cấu trúc dữ liệu & Giải thuật dự định triển khai
1	Người học nhớ chính xác thông tin và cần tra cứu nhanh	Người học nhớ rõ chữ Kanji “先生”, romaji “taberu” hoặc nghĩa “học sinh” và muốn xem ngay thông tin chi tiết kèm ví dụ minh họa mà không mất thời gian.	Tra cứu chính xác (Exact Search)	Bảng băm (HashTable) kích thước 10007 bucket, sử dụng hàm băm DJB2 để ánh xạ khóa thành chỉ số, xử lý xung đột bằng phương pháp chaining.
2	Chỉ nhớ phần đầu của từ (Romaji)	Người học chỉ nhớ phần mở đầu của romaji, ví dụ: bắt đầu bằng “ka-”, và mong muốn hệ thống gợi ý nhanh các từ như “kanji”, “kaisha”, “kazoku”, “kawaii”...	Tra cứu tiền tố (Prefix Search)	Cấu trúc dữ liệu cây tiền tố đa nhánh (Trie) cho phép duyệt nhanh các nút con có chung gốc mà không cần quét lại toàn bộ cơ sở dữ liệu.
3	Chỉ nhớ một phần đoạn bất kỳ trong từ	Người học chỉ nhớ một phần giữa hoặc cuối của từ, ví dụ: chứa “-shitsu” như	Tra cứu chuỗi con (Substring Search)	Giải thuật Knuth-Morris-Pratt (KMP) kết hợp xây dựng mảng tiền tố LPS để tối ưu hóa

		trong 教室 (kyoushitsu) hoặc 研究室 (kenkyuushitsu).		việc dịch chuyển con trỏ, tránh duyệt cạn.
4	Gõ sai chính tả hoặc nhớ sai quy tắc phát âm	Người học nhập sai do rào cản ngữ âm, ví dụ: gõ “gako” hoặc “gakou” thay vì “gakkou” (学校).	Tra cứu mờ (Fuzzy Search)	Thuật toán khoảng cách Levenshtein dựa trên ma trận quy hoạch động để tính toán độ tương đồng chuỗi và đưa ra kết quả tối ưu trong ngưỡng sai số ≤ 2
5	Cần phân tích Kanji xuất hiện trong câu văn thực tế	Gặp câu tiếng Nhật thực tế “私は日本語 を勉強します”, người học muốn hệ thống tự động bóc tách và tra cứu thông tin các chữ Kanji: 私, 日, 本, 語, 勉, 強.	Phân tích câu (Sentence Analysis)	Cơ chế chuyển đổi định dạng chuỗi mã hóa UTF-8 sang ký tự rộng (wchar_t), kết hợp hàm kiểm tra vùng mã Kanji (isKanjiWChar) và bảng băm DJB2.
6	Lọc ra danh sách các từ vùng mà trong đó chỉ chứa các Kanji đã học	Sinh viên đã hoàn thành Bài 1 đến Bài 5, muốn lọc ra tất cả từ vựng chỉ được cấu thành bởi các Kanji thuộc phạm vi đã học đó	Lọc từ vùng thông minh	LearnedHashSet (hash set với 256 bucket) lưu tập Kanji đã học. Duyệt từ vựng và kiểm tra từng Kanji với hai điều kiện: tồn tại trong targetKanjiList và thuộc LearnedHashSet, kết hợp cơ chế dừng sớm để tối ưu hiệu năng.
7	Tra cứu nhanh theo cấu trúc	Người học muốn xem toàn bộ 16 Kanji của một bài học theo	Quản lý bài học phân tầng	Cấu trúc dữ liệu phân cấp (KanjiList → Kanji → Vocab → Sample)

	bài học của giáo trình	đúng trình tự giáo trình, sau đó xem chi tiết từng Kanji kèm từ vựng, ý nghĩa, ví dụ...		được nạp và xử lý thông qua thư viện cJSON.
--	------------------------	---	--	---

2.2. Cơ sở lý thuyết

2.2.1. Kiến thức nền tảng

a) Thư viện JSON - Phân tích cú pháp JSON trong C

Định dạng json

JSON (JavaScript Object Notation) là một định dạng trao đổi dữ liệu nhẹ, dễ đọc và dễ viết đối với con người, đồng thời dễ dàng để máy phân tích cú pháp và sinh ra [8] [9]. Trong đề tài, toàn bộ cơ sở dữ liệu 512 Kanji được lưu trữ dưới dạng một mảng JSON, mỗi phần tử là một đối tượng chứa các trường dữ liệu lồng nhau (string, array, object). Để đọc và phân tích JSON trong ngôn ngữ C (vốn không hỗ trợ sẵn kiểu dữ liệu này), cần một thư viện bên thứ ba.

Thư viện cJSON cho ngôn ngữ C

cJSON là thư viện mã nguồn mở, được viết hoàn toàn bằng C thuần, siêu nhẹ (chỉ gồm hai tệp cJSON.c và cJSON.h), không phụ thuộc vào thư viện ngoài, được phát hành theo giấy phép MIT [10]. Nó cung cấp các hàm để phân tích chuỗi JSON thành cây cấu trúc trong bộ nhớ, truy xuất các trường, cũng như sinh ngược lại chuỗi JSON từ cấu trúc.

Cấu trúc nội bộ

Thư viện định nghĩa một struct cJSON như sau (đã được rút gọn):

```
typedef struct cJSON {
    struct cJSON *next;    // con trỏ đến phần tử kế tiếp trong mảng/object
    struct cJSON *prev;    // con trỏ đến phần tử trước đó
    struct cJSON *child;    // con trỏ đến phần tử con đầu tiên (nếu là array/object)
    int type;              // kiểu dữ liệu: cJSON_String, cJSON_Number, cJSON_Array, ...
    char *valuestring;      // giá trị nếu type là cJSON_String
    int valueint;           // giá trị nếu type là cJSON_Number (dạng số nguyên)
    double valuedouble;     // giá trị nếu type là cJSON_Number (dạng số thực)
    char *string;          // tên key nếu phần tử nằm trong một object
} cJSON;
```

Bảng 2.2: Các hàm cJSON sử dụng trong dự án

Hàm cJSON	Chức năng
cJSON_Parse()	Phân tích chuỗi JSON (dạng char*) thành cây đối tượng cJSON*. Trả về NULL nếu parse thất bại.
cJSON_Delete()	Giải phóng toàn bộ cây cJSON và bộ nhớ đã cấp phát cho nó. Tránh rò rỉ bộ nhớ.
cJSON_GetArraySize()	Trả về số lượng phần tử của một mảng JSON.
cJSON_GetArrayItem()	Lấy phần tử thứ index từ một mảng JSON.
cJSON_GetObjectItem()	Lấy một trường (key) từ đối tượng JSON. Trả về cJSON* hoặc NULL nếu không tồn tại.
cJSON_IsArray()	Kiểm tra một đối tượng cJSON có phải kiểu mảng hay không.

Việc sử dụng thư viện cJSON cho phép quản lý dữ liệu phân cấp rõ ràng và chính xác hơn nhiều so với file văn bản phẳng như .txt. Đối với dữ liệu Kanji, thông tin được tổ chức theo mô hình quan hệ phân tầng có tính liên kết chặt chẽ thay vì các trường thuộc tính độc lập. Từ nút gốc là một chữ Kanji với các định danh cốt lõi (chữ Hán, âm Hán Việt), hệ thống triển khai các nhánh dữ liệu con phụ thuộc, bao gồm tập hợp các phương thức biểu diễn âm đọc Onyomi - Kunyomi, danh sách các từ vựng phái sinh được cấu thành, và sâu hơn là các trường thông tin cấu trúc chi tiết của từng từ vựng như giải thích ngữ nghĩa, câu ví dụ minh họa,... Nếu dùng dữ liệu phẳng để biểu diễn, việc phân tích cú pháp (parsing) trở nên phức tạp, dễ sai lệch chỉ mục và khó mở rộng. Ngược lại, cấu trúc lồng nhau của cJSON giúp bảo toàn tính toàn vẹn dữ liệu và ánh xạ trực tiếp sang các struct trong C một cách tường minh.

Ngoài ra, việc sử dụng cJSON giúp tách biệt hoàn toàn dữ liệu khỏi mã nguồn: muốn cập nhật, sửa lỗi hay mở rộng thêm Kanji, chỉ cần chỉnh sửa tệp JSON mà không cần biên dịch lại chương trình.

b) Xử lý Unicode UTF-8, UTF-16 trên console Windows

Unicode là một tiêu chuẩn công nghiệp toàn cầu, gán cho mỗi ký tự (chữ cái, chữ số, tượng hình, dấu câu, emoji) một mã số duy nhất gọi là code point (ví dụ chữ Kanji “日” có mã U+65E5) [11]. Để lưu trữ và truyền tải các code point này trong bộ nhớ máy tính, cần có các phương thức mã hóa. Hai phương thức phổ biến nhất là UTF-8 và UTF-16.

UTF-8 sử dụng độ dài byte biến thiên từ 1 đến 4 byte cho mỗi ký tự. Các ký tự ASCII (0–127) chiếm 1 byte, chữ Việt có dấu và hầu hết chữ cái Latin mở rộng

chiếm 2 byte, còn chữ Kanji thường chiếm 3 byte. UTF-8 được ưa chuộng trong các tệp văn bản, JSON, HTML.

UTF-16 mã hóa hầu hết các ký tự thông dụng (thuộc mặt bảng đa ngữ cơ bản – BMP, U+0000..U+FFFF) bằng 2 byte cố định, và dùng cặp đại diện 4 byte cho các ký tự hiếm. Windows sử dụng UTF-16 LE làm mã hóa nội bộ cho các API hệ thống (console, tên tệp, hộp thoại) và cho luồng nhập từ bàn phím.

Xử lý luồng dữ liệu Unicode đa byte đầu vào bằng hệ thống Win32 API

Trên môi trường console Windows (CMD, PowerShell), khi người dùng gõ một chữ Kanji, bàn phím và hệ điều hành tạo ra luồng dữ liệu UTF-16 LE, được lưu tạm vào bộ đệm của console [12]. Các hàm thư viện C chuẩn như scanf, fgets hoạt động trên luồng byte thô, dựa vào code page hiện tại của console để chuyển đổi giữa byte và ký tự. Code page là bảng ánh xạ giới hạn (tối đa 256 ký tự), không thể biểu diễn đồng thời Kanji, Hiragana và tiếng Việt có dấu. Ngay cả khi đặt code page thành UTF-8 (65001) bằng lệnh chcp, tầng đệm của thư viện C (stdio buffering) vẫn không được thiết kế để giải mã chính xác luồng UTF-16 LE đa byte, dẫn đến hiện tượng cắt byte, sai độ dài chuỗi và sinh ra ký tự rác (mojibake). Hậu quả là không thể nhập được câu tiếng Nhật hoặc tiếng Việt có dấu một cách tin cậy.

Để khắc phục, đề tài từ bỏ các hàm Standard I/O và sử dụng trực tiếp Win32 API, được triển khai trong hàm inputString() (trích tệp src/utils.c). Quy trình gồm ba bước:

- 1) GetStdHandle(STD_INPUT_HANDLE) lấy luồng nhập chuẩn của console.
- 2) ReadConsoleW đọc trực tiếp bộ đệm UTF-16 LE vào mảng wchar_t* mà không qua bất kỳ chuyển đổi code page nào, đảm bảo giữ nguyên mã nguồn gốc của từng ký tự [6].
- 3) Sau khi loại bỏ ký tự xuống dòng, WideCharToMultiByte với tham số CP_UTF8 chuyển đổi chuỗi UTF-16 thành UTF-8 và lưu vào buffer (kiểu char*). Kết quả là một chuỗi UTF-8 hoàn chỉnh, đồng nhất với định dạng dữ liệu trong tệp JSON [5].

```
void inputString(char *buffer, int maxLength) {
#ifdef _WIN32
    wchar_t *wBuf = (wchar_t*)malloc(maxLength * sizeof(wchar_t));
    HANDLE hStdin = GetStdHandle(STD_INPUT_HANDLE);
    DWORD read = 0;
    ReadConsoleW(hStdin, wBuf, maxLength - 1, &read, NULL);
```

```

// Xóa ký tự xuống dòng
if (read >= 2 && wBuf[read-2] == L'\r' && wBuf[read-1] == L'\n')
    wBuf[read-2] = L'\0';
// Chuyển UTF-16 sang UTF-8
WideCharToMultiByte(CP_UTF8, 0, wBuf, -1, buffer, maxLength, NULL,
NULL);
    free(wBuf);
#endif
}

```

Xử lý chuỗi đa byte và giải pháp đồng nhất ký tự ngữ nghĩa bằng wchar_t

Mặc dù char* UTF-8 tối ưu cho lưu trữ và xuất dữ liệu, nó lại gây ra sai số nghiêm trọng trong các thuật toán duyệt từng ký tự như tìm kiếm chuỗi con (KMP, src/substring_search.c), khoảng cách Levenshtein (src/fuzzy_search.c) và phân tích câu (src/sentence_analysis.c).

Nguyên nhân là trong ngôn ngữ C, con trỏ char* trỏ đến mảng byte; toán tử i++ chỉ dịch chuyển 1 byte, không phải 1 ký tự ngữ nghĩa. Do mỗi Kanji trong UTF-8 chiếm 3 byte, ví dụ một chuỗi gồm hai Kanji “漢字” được lưu thành 6 byte [0xE6,0xB1,0x89,0xE5,0xAD,0x97]. Vòng lặp với i++ sẽ đọc lần lượt từng byte đơn lẻ (như 0xE6, 0xB1, ...) thay vì từng chữ, làm cho thuật toán tính sai độ dài và so khớp sai giá trị.

Để khắc phục, đề tài xây dựng hàm convertToWchar() (trích tệp src/utlis.c) chuyển đổi chuỗi UTF-8 sang mảng wchar_t* (UTF-16):

```

wchar_t* convertToWchar(const char *source) {
    if (!source || *source == '\0') return NULL;
    int w_len = MultiByteToWideChar(CP_UTF8, 0, source, -1, NULL, 0);
    wchar_t *w_str = (wchar_t*)malloc(w_len * sizeof(wchar_t));
    MultiByteToWideChar(CP_UTF8, 0, source, -1, w_str, w_len);
    return w_str;
}

```

Hàm này sử dụng MultiByteToWideChar để giải mã mỗi cụm byte UTF-8 thành một code point Unicode, sau đó lưu vào một phần tử wchar_t. Trên Windows, wchar_t có kích thước 2 byte, đủ để chứa toàn bộ các Kanji thông dụng (vốn thuộc khối BMP, dải U+4E00..U+9FFF) [13] [14]. Nhờ đó, chuỗi “漢字” được biểu diễn thành mảng wchar_t gồm hai phần tử: [0x6C49, 0x5B57]. Bây giờ, trên mảng

wchar_t*, toán tử i++ tự động nhảy sizeof(wchar_t) = 2 byte nhờ cơ chế toán hạng con trỏ của C, đảm bảo mỗi bước đúng một ký tự ngữ nghĩa. Các thuật toán KMP, Levenshtein và phân tích câu do đó hoạt động chính xác tuyệt đối.

2.2.2. Công nghệ, ngôn ngữ và công cụ phát triển

- Ngôn ngữ: C (C99/C11) – lựa chọn vì kiểm soát bộ nhớ thủ công tối ưu, hiệu năng cao, phù hợp học phần cấu trúc dữ liệu.
- Trình biên dịch: GCC (MinGW-w64) trên Windows.
- IDE: Visual Studio Code với tiện ích C/C++ của Microsoft.
- Thư viện:
 - + cJSON v1.7.19: phân tích dữ liệu JSON.
 - + <wchar.h>, <windows.h>: xử lý ký tự rộng và Win32 API.
 - + <stdlib.h>, <string.h>: quản lý bộ nhớ, thao tác chuỗi.
 - + <time.h>: khởi tạo bộ sinh số ngẫu nhiên cho trắc nghiệm.
- Hệ điều hành: Windows 10/11
- Script build: build.bat (dùng cho biên dịch).

3. TỔ CHỨC CẤU TRÚC DỮ LIỆU VÀ THUẬT TOÁN

3.1. Phát biểu bài toán

3.1.1. *Nạp và phân giải dữ liệu từ tệp cấu trúc JSON*

- **Mô tả:** Chuyển đổi toàn bộ cơ sở dữ liệu lưu trữ tĩnh từ tệp dữ liệu cấu trúc JSON vào bộ nhớ RAM dưới dạng các cấu trúc dữ liệu (struct) trong C ngay khi khởi động hệ thống.
- **Input:** Tệp dữ liệu tĩnh (data/kanjiData.json) chứa toàn bộ danh sách chữ Kanji, từ vựng, âm đọc và câu ví dụ mẫu dưới dạng văn bản mã hóa UTF-8.
- **Output:** Toàn bộ dữ liệu được tổ chức, ánh xạ và lưu trữ thành công lên bộ nhớ tạm thời (RAM) của hệ thống, sẵn sàng cho các chức năng khác truy xuất ngay lập tức.

3.1.2. *Quản lý và hiển thị dữ liệu phân hoạch theo bài học*

- **Mô tả:** Phân chia danh sách tổng dữ liệu Kanji thành các bài học nhỏ có kích thước cố định để người dùng học theo lộ trình tuyến tính, đồng thời cung cấp giao diện xem chi tiết thông tin của từng chữ.
- **Input:** Số thứ tự bài học do người dùng lựa chọn từ menu điều khiển (ví dụ: bài 1, bài 2, bài 3).
- **Output:** Màn hình hiển thị danh sách thu gọn gồm chính xác 16 chữ Kanji thuộc bài học tương ứng; và giao diện hiển thị toàn bộ thông tin chi tiết (bộ thủ, số nét, âm đọc, nghĩa, từ vựng đi kèm) khi người dùng chọn xem một chữ cụ thể.
- **Ví dụ:** Người dùng chọn Bài số 1. Hệ thống hiển thị danh sách 16 chữ từ chữ thứ 1 đến 16 của Bài 1. Khi người dùng chọn tiếp chữ 「一」 trong danh sách, màn hình sẽ hiển thị chi tiết: Bộ thủ: Nhất, Số nét: 1, Hán Việt: NHẤT, Nghĩa: Số một, Âm On: ICHI, đi kèm từ vựng mẫu 「一つ」.

3.1.3. *Tìm kiếm chính xác dựa trên cơ chế phân băm*

- **Mô tả:** Thực hiện truy vấn nhanh các thực thể (chữ Kanji hoặc từ vựng) có nội dung khớp hoàn toàn 100% với từ khóa do người dùng cung cấp, không chấp nhận sai lệch ký tự hoặc khoảng trắng thừa.
- **Input:** Một từ khóa nhập từ bàn phím, có thể là chữ Kanji gốc, chữ Hiragana hoặc phiên âm Romaji của từ cần tìm.
- **Output:** Khối thông tin chi tiết đầy đủ của đúng thực thể tìm thấy hoặc thông báo cảnh báo nếu từ khóa không tồn tại chính xác trong hệ thống.

- **Ví dụ:** Người dùng nhập từ khóa 「先生」 hoặc 「sensei」. Hệ thống thực hiện tìm kiếm và trả về thông tin chi tiết của từ vựng 「先生」 (Tiên sinh). Nếu nhập sai, hệ thống sẽ đưa ra cảnh báo không tìm thấy kết quả.

3.1.4. *Tìm kiếm tiền tố ứng dụng cấu trúc cây Trie*

- **Mô tả:** Quét dữ liệu và liệt kê tất cả các từ vựng tiếng Nhật có phần phiên âm Latinh (Romaji) bắt đầu bằng chuỗi ký tự tiền tố do người dùng nhập vào.
- **Input:** Một phần chuỗi ký tự chữ cái Latinh (Romaji) đại diện cho phần đầu của từ (tiền tố).
- **Output:** Danh sách toàn bộ các từ vựng trong hệ thống có phần phiên âm Romaji bắt đầu bằng chính xác chuỗi ký tự tiền tố đã nhập.
- **Ví dụ:** Người dùng nhập vào chuỗi tiền tố là 「ga」. Hệ thống quét kho dữ liệu và trả về danh sách gồm các từ vựng như: 「gaikoku」, 「gakusei」, 「gakkou」 ... do các từ này đều bắt đầu bằng hai ký tự "ga".

3.1.5. *Tìm kiếm theo chuỗi con bằng thuật toán KMP*

- **Mô tả:** Tìm kiếm sâu và trích xuất tất cả từ vựng mà trong nội dung các thuộc tính (từ gốc, furigana, romaji, giải nghĩa Tiếng Việt) có chứa chuỗi ký tự tìm kiếm tại bất kỳ vị trí nào.
- **Input:** Một từ khóa tìm kiếm bất kỳ, có thể bằng tiếng Nhật (Kanji, Hiragana, Romaji) hoặc bằng chữ tiếng Việt (nghĩa của từ).
- **Output:** Danh sách toàn bộ từ vựng mà trong thông tin của chúng (ở một trong bốn trường: từ gốc, hiragana, romaji hoặc giải nghĩa tiếng Việt) có chứa cụm ký tự trùng khớp với từ khóa.
- **Ví dụ:** Người dùng nhập từ khóa tiếng Việt là 「học」. Hệ thống phân tích văn bản và trả về danh sách các từ như 「đại học」, 「học sinh」, 「học tập」 vì trong trường giải nghĩa hoặc từ gốc của các từ này đều tồn tại cụm chuỗi con là 「học」.

3.1.6. *Tìm kiếm mờ tính toán khoảng cách Levenshtein*

- **Mô tả:** Nhận diện và trả về các từ vựng gần đúng với từ khóa tìm kiếm khi người dùng nhập sai chính tả hoặc thiếu ký tự.
- **Input:** Từ khóa truy vấn bằng chuỗi ký tự Kanji, Hiragana hoặc Romaji bị gõ lỗi, thiếu nét, nhớ không rõ hoặc sai chính tả do người dùng nhập vào.
- **Output:** Danh sách các từ vựng có độ tương đồng cao nhất trong hệ thống, kèm theo chỉ số hiển thị mức độ sai lệch (khoảng cách sai số ký tự) nằm trong ngưỡng cho phép.

- **Ví dụ:** Người dùng gõ nhầm chuỗi 「akko」 do gõ thiếu và sai từ gốc chính xác. Hệ thống tính toán độ tương đồng và đưa ra gợi ý từ đúng là 「gakkou」 kèm thông tin mức độ lệch ký tự là 2, do khoảng cách chỉnh sửa này nằm trong giới hạn sai sót tối đa.

3.1.7. *Lọc ra danh sách từ vựng chỉ chứa các Kanji đã học*

- **Mô tả:** Cô lập kho từ vựng, chỉ trích xuất các từ phù hợp với đúng tiến độ tích lũy bài học hiện tại để phục vụ ôn tập tập trung, không bị lẫn từ mới chưa học hay bỏ sót các từ có thể học được trong năng lực hiện có.
- **Input:** Danh sách các số hiệu bài học cụ thể mà người dùng chọn để ôn tập (ví dụ: các bài 1 và 3).
- **Output:** Danh sách các từ vựng chứa ít nhất 1 kanji thuộc các bài đã chọn, với điều kiện bắt buộc là tất cả các chữ cấu thành còn lại (các chữ Kanji bên trong từ đó) đều phải thuộc phạm vi các chữ đã được học tính từ Bài 1 đến bài lớn nhất trong danh sách chỉ định.
- **Ví dụ:** Người dùng nhập danh sách bài ôn tập là 「1 3」. Hệ thống quét các từ vựng trong bài 1 và bài 3. Từ 「学生」 sẽ được trả về nếu cả hai chữ 「学」 và 「生」 đều nằm trong tập Kanji tích lũy từ bài 1 đến bài 3 và ít nhất 1 chữ Kanji trong đó thuộc bài 1 hoặc 3. Ngược lại, nếu từ chứa bất kì Kanji thuộc bài 4 trở đi, từ đó sẽ bị loại bỏ để đảm bảo an toàn kiến thức.

3.1.8. *Phân tích các Kanji trong câu tiếng Nhật*

- **Mô tả:** Tiếp nhận một chuỗi câu văn bản tiếng Nhật tự do nhập từ bàn phím, thực hiện bóc tách và nhận diện các ký tự Kanji đơn lẻ hiện diện trong câu, sau đó hiển thị thông tin tra cứu chi tiết của chúng.
- **Input:** Một đoạn văn hoặc một câu tiếng Nhật tự do chứa hỗn hợp nhiều loại chữ viết (Kanji, Hiragana, Katakana, chữ số, dấu câu).
- **Output:** Danh sách cô lập gồm riêng các chữ Kanji xuất hiện bên trong câu đó, kèm theo bảng hiển thị thông tin tra cứu chi tiết (bộ thủ, cách đọc, ý nghĩa) cho từng chữ được tìm thấy.
- **Ví dụ:** Người dùng nhập vào câu tự do: 「私は日本語を勉強します。」. Hệ thống sẽ tự động quét, bỏ qua các ký tự Hiragana (は, を, します) và dấu câu, trích xuất được danh sách các chữ Kanji đơn lẻ gồm: 「私」, 「日」, 「本」, 「勉」, 「強」, đồng thời hiển thị bảng thông tin chi tiết về bộ thủ, ý nghĩa, cách đọc cho từng chữ này.

3.2. Cấu trúc dữ liệu

3.2.1. Cấu trúc tệp tin dữ liệu JSON (*kanjiData.json*)

Tệp kanjiData.json đóng vai trò như kho dữ liệu phi quan hệ, tổ chức dưới dạng cây phân cấp. Nội dung gồm một mảng các đối tượng Kanji, mỗi đối tượng có:

- Thuộc tính cơ bản: stt, kanji, hanViet, radical, stroke, description.
- Mảng lồng cho âm đọc: on, kun.
- Mảng vocabs chứa từ vựng liên quan.

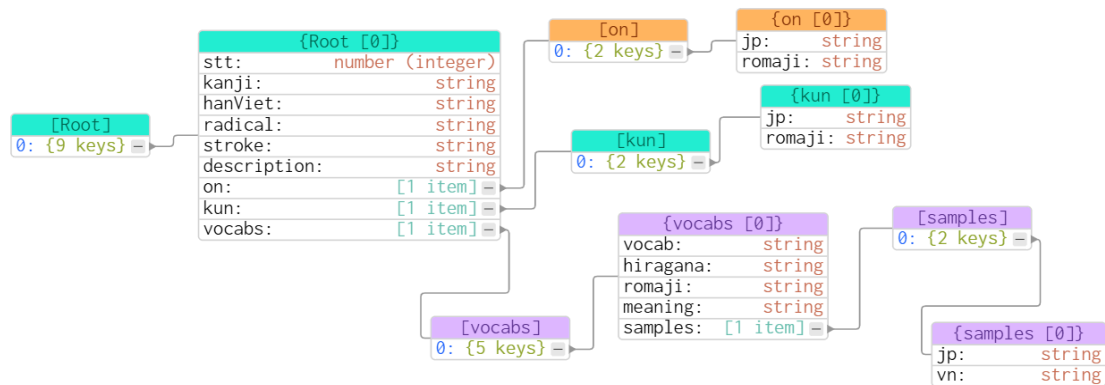
Bên trong mỗi vocab có thêm mảng con cấp 3 cho samples – các câu ví dụ song ngữ.

```
[
  {
    "stt": 1,
    "kanji": "一",
    "hanViet": "nhất",
    "radical": "一",
    "stroke": "1",
    "description": "nhất, one radical",
    "on": [
      { "jp": "イチ", "romaji": "ichi" },
      { "jp": "イツ", "romaji": "itsu" }
    ],
    "kun": [
      { "jp": "ひと-", "romaji": "hito-" },
      { "jp": "ひと.つ", "romaji": "hito.tsu" }
    ],
    "vocabs": [
      {
        "vocab": "一",
        "hiragana": "いち",
        "romaji": "ichi",
        "meaning": "số 1",
```

```

"samples": [
  { "jp": "これは一です。", "vn": "Đây là số 1." },
  { "jp": "一たす二は三です。", "vn": "1 cộng 2 bằng 3." }
]
}
]
}
]

```



Hình 3.1: Tổ chức dữ liệu file kanjiData.json

Cấu trúc phân tích cJSON

Thư viện cJSON (định nghĩa tại lib/cJSON.h) là lớp trung gian giúp chuyển đổi chuỗi JSON thô thành cấu trúc cây trong bộ nhớ C. Dữ liệu được quản lý dưới dạng danh sách liên kết đôi, trong đó mỗi nút đại diện cho một cặp khóa–giá trị, một mảng hoặc một đối tượng, sẵn sàng ánh xạ sang các struct lỗi.

```

typedef struct cJSON {
    struct cJSON *next;    // Nút kế tiếp cùng cấp
    struct cJSON *prev;    // Nút trước cùng cấp
    struct cJSON *child;   // Nút con (mảng hoặc đối tượng lồng)

    int type;              // Kiểu dữ liệu (String, Number, Array, Object...)
    char *valuelstring;     // Giá trị dạng chuỗi
    int valueint;           // Giá trị dạng số nguyên
    double valuedouble;     // Giá trị dạng số thực

```

```
char *string;          // Tên khóa (Key)
} cJSON;
```

3.2.2. Cấu trúc dữ liệu lỗi

Các cấu trúc được khai báo tại data_structures.h và được khởi tạo động thông qua module src/parse_json_to_struct.c (gồm các hàm parseSamples, parseVocabs, parseYomi, và parseJsonToStruct). Đây là nguồn dữ liệu duy nhất lưu trữ toàn bộ cơ sở dữ liệu ngôn ngữ sau khi giải mã từ kanjiData.json.

Dưới đây là định nghĩa chi tiết các cấu trúc dữ liệu lỗi được trích từ tệp data_structures.h:

- SampleInfo: câu ví dụ song ngữ Nhật - Việt.
- VocabInfo: từ vựng, cách đọc Hiragana, phiên âm Romaji, nghĩa tiếng Việt; mỗi trường lưu song song dạng char* và wchar_t*
- YomiInfo: âm đọc Kana và Romaji tương ứng.
- KanjiInfo: thông tin định danh Kanji, mảng âm ON/KUN, mảng từ vựng liên quan.
- KanjiListInfo: cấu trúc bao quát, quản lý mảng động toàn bộ Kanji của hệ thống.

```
typedef struct SampleInfo Sample;
typedef struct VocabInfo Vocab;
typedef struct YomiInfo Yomi;
typedef struct KanjiInfo Kanji;
typedef struct KanjiListInfo KanjiList;

struct SampleInfo {
    char *jp;
    char *vn;
};

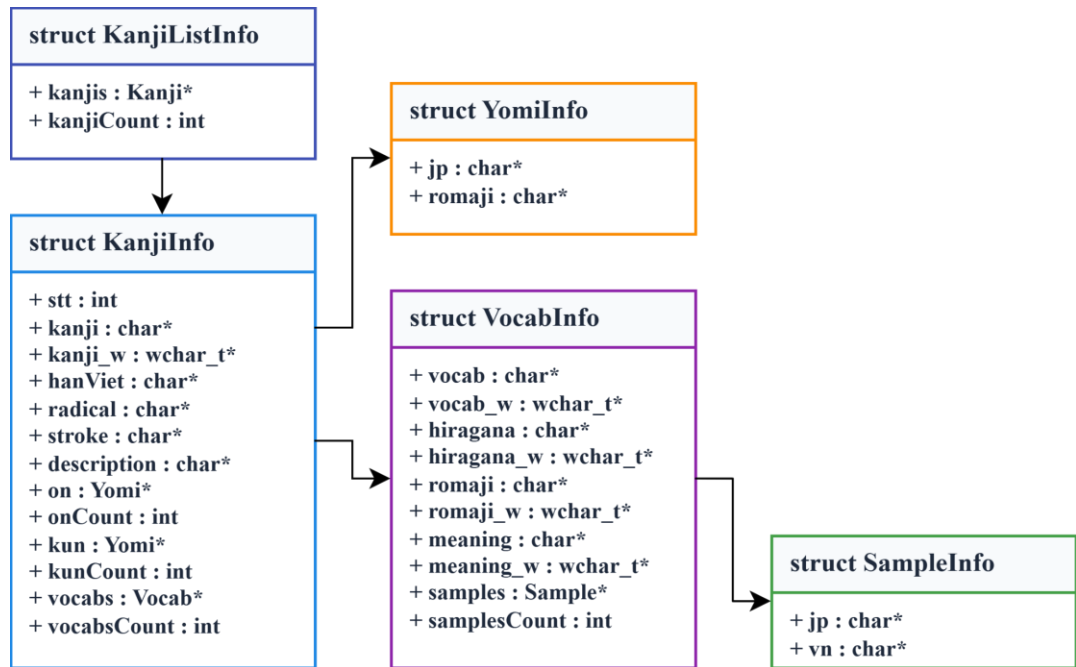
struct VocabInfo {
    char *vocab; wchar_t *vocab_w;
    char *hiragana; wchar_t *hiragana_w;
```

```
char *romaji; wchar_t *romaji_w;
char *meaning; wchar_t *meaning_w;
Sample *samples; int samplesCount;
};

struct YomiInfo {
    char *jp;
    char *romaji;
};

struct KanjiInfo {
    int stt;
    char *kanji; wchar_t *kanji_w;
    char *hanViet; char *radical; char *stroke; char *description;
    Yomi *on; int onCount;
    Yomi *kun; int kunCount;
    Vocab *vocabs; int vocabsCount;
};

struct KanjiListInfo {
    Kanji *kanjis; int kanjiCount;
};
```

Hình 3.2: Sơ đồ quan hệ giữa các cấu trúc dữ liệu lõi

3.2.3. Cấu trúc dữ liệu cho chức năng tra cứu

Chức năng tra cứu được chia thành 4 phương thức tìm kiếm độc lập nhằm tối ưu hóa hiệu năng và thuật toán xử lý dữ liệu:

a) Tìm kiếm chính xác (Exact Search) - Bảng băm (Hash Table)

Được khai báo tại `src/kanji_search_exact.h` và `src/vocab_search_exact.h`. Khởi tạo thông qua các hàm `initHashTableKanji` và `initHashTableVocab`. Bảng băm kích thước cố định (10007), sử dụng phương pháp kết xích với danh sách liên kết đơn để xử lý va chạm.

Đối với Kanji, khóa tra cứu là ký tự Kanji hoặc chuỗi Hán Việt, ánh xạ tới con trỏ Kanji*. (`src/kanji_search_exact.h`)

```

// Trích từ src/kanji_search_exact.h
struct HashNodeForKanji {
    char *key;           // Từ khóa (kanji hoặc hanViet)
    Kanji *kanji;        // Con trỏ tới dữ liệu Kanji đầy đủ
    struct HashNodeForKanji *next; // Con trỏ tới node tiếp theo (xử lý va chạm)
};

struct HashTableForKanji {
    struct HashNodeForKanji **table; // Mảng các bucket
    int size;                        // Kích thước bảng (10007)
}
  
```

```

    int count;                // Tổng số phần tử đã lưu
};

typedef struct HashNodeForKanji HashNodeK;
typedef struct HashTableForKanji HashTableK;

```

Đối với Từ vựng, mỗi vocab tạo ra 4 nút băm độc lập, tương ứng với các khóa: vocab, hiragana, romaji, và meaning.

```

// Trích từ src/vocab_search_exact.h
struct HashNodeForVocab {
    char *key;                // Từ khóa (vocab/hiragana/romaji/meaning)
    Vocab *vocab;             // Con trỏ tới dữ liệu Vocab đầy đủ
    struct HashNodeForVocab *next; // Con trỏ tới node tiếp theo (xử lý va chạm)
};

struct HashTableForVocab {
    struct HashNodeForVocab **table; // Mảng các bucket
    int size;                        // Kích thước bảng (10007)
    int count;                       // Tổng số phần tử đã lưu
};

typedef struct HashNodeForVocab HashNode;
typedef struct HashTableForVocab HashTable;

```

b) Tìm kiếm mờ (Fuzzy Search) - Levenshtein Distance

Phương thức này thực hiện duyệt tuyến tính toàn bộ danh sách từ vựng, áp dụng thuật toán tính khoảng cách Levenshtein giữa từ khóa nhập vào và các trường dữ liệu ký tự rộng (wchar_t*) của từ vựng, bao gồm: vocab_w – Mặt chữ Kanji gốc, hiragana_w – Cách đọc Furigana (Hiragana), romaji_w – Phiên âm Latinh (Romaji)

c) Tìm kiếm chuỗi con (Substring Search) - KMP Algorithm

Phương thức này thực hiện duyệt tuyến tính toàn bộ danh sách từ vựng, áp dụng thuật toán KMP (Knuth-Morris-Pratt) để tìm kiếm mẫu chuỗi con trong các trường dữ liệu ký tự rộng (wchar_t*) của từ vựng, bao gồm: vocab_w – Mặt chữ

Kanji gốc, hiragana_w – Cách đọc Furigana (Hiragana), romaji_w – Phiên âm Latinh (Romaji), meaning_w – Nghĩa Tiếng Việt

d) Tìm kiếm tiền tố (Prefix Search) - Cấu trúc Cây tiền tố (Trie)

Lưu trữ cấu trúc cây tiền tố (Prefix Tree) đa nhánh. Mỗi nút (TrieNode) chứa một mảng cố định gồm 28 con trỏ nhánh đại diện cho dải chữ cái Latinh từ a-z cộng thêm 2 ký tự đặc biệt (- và .) trong chuỗi Romaji. Tại các nút đánh dấu kết thúc một từ hoàn chỉnh (isEndOfWord = 1), nút đó lưu trữ một mảng động các con trỏ (Vocab**) trỏ thẳng tới các cấu trúc từ vựng tương ứng trong dữ liệu lõi [15].

```
// Định nghĩa cấu trúc nút cây Trie trong src/prefix_search.h
#define ALPHABET_SIZE 28
struct TrieNodeInfo {
    struct TrieNodeInfo *children[ALPHABET_SIZE]; // Mảng cố định 28 con trỏ trỏ
    // đến các nút con cấp kế tiếp
    int isEndOfWord; // Biến cờ (0 hoặc 1) đánh dấu nút kết thúc của
    // một từ Romaji
    Vocab **vocabs; // Mảng động chứa các con trỏ trỏ tới cấu trúc
    // Vocab gốc thỏa mãn
    int vocabCount; // Số lượng từ vựng đăng ký lưu trữ tại nút này
};
```

3.2.4. Cấu trúc dữ liệu cho chức năng lọc từ vựng đã học

Được định nghĩa nội bộ và khởi tạo, cấp phát vùng nhớ trực tiếp bên trong module src/filter_learned_lesson.c (hàm insertLearnedKanji).

Lưu trữ tập hợp các chữ Kanji người dùng đã học dưới dạng một Tập hợp băm (Hash Set) thu gọn với kích thước HASH_SIZE = 256 gàu băm. Cấu trúc này tối ưu riêng cho việc không lưu trữ trường giá trị (value), chỉ lưu trữ khóa ký tự rộng (wchar_t* kanji) đóng vai trò kiểm tra nhanh toán học xem một chữ Kanji bất kỳ đã thuộc hay chưa với độ phức tạp thời gian đạt $O(1)$.

```
// Định nghĩa cấu trúc cục bộ phục vụ thuật toán lọc tại src/filter_learned_lesson.c
#define HASH_SIZE 256

typedef struct HashNode {
    wchar_t* kanji; // Ký tự chữ Kanji dạng Wide char đóng vai trò Khóa
    // (Key)
```

```

    struct HashNode* next;          // Địa chỉ phần tử tiếp theo khi xảy ra đụng độ băm
} HashNode;

typedef struct {
    HashNode* buckets[HASH_SIZE]; // Mảng tĩnh chứa các Bucket băm kích thước
    cố định 256
} LearnedHashSet;

```

3.2.5. Cấu trúc dữ liệu cho chức năng phân tích câu

Được định nghĩa cục bộ và khởi tạo thông qua hàm khởi tạo bảng băm phân tích createKanjiHashTable đặt trong module src/sentence_analysis.c. Đây là bảng băm tạm thời (size 512) trong RAM. Nó ánh xạ mã ký tự rộng của Kanji sang địa chỉ vùng nhớ gốc để tăng tốc quá trình bóc tách Kanji trong câu văn và tự giải phóng hoàn toàn bộ nhớ sau khi sử dụng.

```

// Định nghĩa cấu trúc cục bộ phục vụ phân tách câu tại src/sentence_analysis.c
typedef struct KanjiNode {
    Kanji* kanjiData;          // Tham chiếu trực tiếp đến vùng nhớ lưu thông tin gốc
    của Kanji
    struct KanjiNode* next;     // Con trỏ xử lý xung đột chuỗi tại chỉ mục băm
} KanjiNode;

typedef struct {
    KanjiNode* buckets[512];    // Mảng quản lý các Bucket băm kích thước cố
    định 512 phần tử
} KanjiHashTable;

```

3.3. Thuật toán

Hệ thống triển khai 06 nhóm thuật toán chính phục vụ các chức năng tra cứu, tìm kiếm và xử lý dữ liệu trên nền ngôn ngữ Nhật.

3.3.1. Thuật toán băm (Hash) cho tìm kiếm chính xác

a) Giải thuật băm DJB2

Sử dụng hàm băm của Dan Bernstein để ánh xạ chuỗi ký tự (char*) thành chỉ số băm trong bảng kích thước cố định 10007. Thuật toán khởi tạo giá trị hash = 5381 và cập nhật theo công thức $hash = ((hash \ll 5) + hash) + c$ (tương đương $hash * 33$

+ c) cho từng ký tự. Phép dịch bit $\ll 5$ được ưu tiên sử dụng thay cho phép nhân để tối ưu tốc độ xử lý ở cấp độ phần cứng [16].

```
// Trích từ src/kanji_search_exact.c và src/vocab_search_exact.c
static int hashGetIndex(char *str, int tableSize) {
    unsigned long hash = 5381; // Giá trị khởi tạo theo nghiên cứu của Dan Bernstein
    int c;
    while ((c = *str++)) {
        hash = ((hash << 5) + hash) + c; // hash * 33 + c
    }
    return (int)(hash % tableSize);
}
```

b) Xử lý va chạm bằng phương pháp kết xích (Chaining)

Khi xảy ra xung đột (hai khóa khác nhau có cùng chỉ số băm), hệ thống sử dụng danh sách liên kết đơn để lưu trữ các nút tại mỗi gàu băm (bucket). Kỹ thuật chèn vào đầu danh sách được áp dụng để đảm bảo chi phí thời gian cố định $O(1)$.

- **Bảng băm Kanji:** kích thước 10007, mỗi Kanji chèn 2 key (kanji + hanViet)
 - + Xây dựng bảng băm Kanji:

```
// Trích từ src/kanji_search_exact.c
void buildHashTableForKanji(KanjiList *L, HashTableK *ht) {
    if (!L || !ht) return;
    for (int i = 0; i < L->kanjiCount; i++) {
        Kanji *k = &L->kanjis[i];

        if (k->kanji) insertKanjiToHT(ht, k->kanji, k);
        if (k->hanViet) insertKanjiToHT(ht, k->hanViet, k);
    }
}
```

- + Chèn vào đầu danh sách:

```
// Trích từ src/kanji_search_exact.c
void insertKanjiToHT(HashTableK *ht, char *key, Kanji *kanji) {
    int index = hashGetIndex(key, ht->size);
    HashNodeK *newNode = createHashNodeK(key, kanji);
```

```
// Chèn vào đầu danh sách (head insertion) - O(1)
newNode->next = ht->table[index];
ht->table[index] = newNode;
ht->count++;
}
```

+ Thao tác tìm kiếm:

```
// Trích từ src/kanji_search_exact.c
void exactlySearchingKanji(HashTableK *ht, char *key) {
    int index = hashGetIndex(key, ht->size);
    HashNodeK *current = ht->table[index];

    while (current != NULL) {
        if (strcmp(current->key, key) == 0) {
            printOutKanji(current->kanji);
            return;
        }
        current = current->next;
    }
    // Không tìm thấy
}
```

- **Bảng băm Vocab:** kích thước 10007, mỗi Vocab chèn 4 key (vocab, hiragana, romaji, meaning)

+ Xây dựng bảng băm Từ vựng:

```
// Trích từ src/vocab_search_exact.c
void buildHashTableForVocab(KanjiList *L, HashTable *ht) {
    int i, j;
    for (i = 0; i < L->kanjiCount; i++) {
        Kanji *k = &L->kanjis[i];
        for (j = 0; j < k->vocabsCount; j++) {
            Vocab *v = &k->vocabs[j];

            insertVocabToHT(ht, v->vocab, v);
        }
    }
}
```

```

        insertVocabToHT(ht, v->romaji, v);
        insertVocabToHT(ht, v->hiragana, v);
        insertVocabToHT(ht, v->meaning, v);
    }
}
}

```

+ Chèn vào đầu danh sách:

```

// Trích từ src/vocab_search_exact.c
void insertVocabToHT(HashTable *ht, char *key, Vocab *vocab) {
    if (!ht || !key || !vocab) return;

    int indexHT = hashGetIndex(key, ht->size);
    HashNode *newNode = createHashNode(key, vocab);
    newNode->next = ht->table[indexHT];
    ht->table[indexHT] = newNode;
    ht->count++;
}

```

+ Thao tác tìm kiếm:

```

// Trích từ src/vocab_search_exact.c
void exactlySearchingVocab(HashTable *ht, char *key) {
    if (!ht || !key) return;

    int index = hashGetIndex(key, ht->size);
    HashNode *current = ht->table[index];

    while (current != NULL) {
        if (strcmp(current->key, key) == 0) {
            printOutVocab(current->vocab);
            return;
        }
        current = current->next;
    }
}

```

```
// Không tìm thấy
}
```

c) Độ phức tạp thuật toán băm

Về mặt thời gian, hàm băm DJB2 có độ phức tạp $O(L)$ với L là độ dài chuỗi đầu vào, do thuật toán duyệt qua từng ký tự một lần để tính giá trị băm. Thao tác chèn phần tử vào bảng băm có độ phức tạp $O(1)$ vì chiến lược chèn vào đầu danh sách liên kết không phụ thuộc vào kích thước danh sách. Thao tác tìm kiếm có độ phức tạp trung bình $O(1)$ nếu hàm băm phân phối đều các khóa trên toàn bộ bảng, nhưng trong trường hợp xấu nhất khi tất cả khóa đều băm về cùng một bucket, độ phức tạp có thể suy biến thành $O(n)$ với n là tổng số phần tử trong bảng.

3.3.2. Thuật toán tìm kiếm mờ Levenshtein Distance

Thuật toán Levenshtein Distance được sử dụng để tính khoảng cách chỉnh sửa giữa hai chuỗi ký tự rộng (`wchar_t*`), từ đó xác định các từ vựng gần đúng với từ khóa mà người dùng nhập vào, bất chấp các sai sót chính tả như thêm, xóa hoặc thay thế ký tự [17]. Đây là thành phần cốt lõi của chức năng tìm kiếm mờ (Fuzzy Search), cho phép hệ thống vẫn tìm ra kết quả ngay cả khi người dùng gõ sai một vài ký tự trong mặt chữ Kanji, Furigana hoặc phiên âm Romaji.

Thuật toán được triển khai bằng phương pháp quy hoạch động, xây dựng ma trận có kích thước $(len1 + 1) \times (len2 + 1)$ để lưu khoảng cách giữa các tiền tố của hai chuỗi. Hàng và cột đầu tiên được khởi tạo lần lượt với giá trị từ 0 đến $len1$ và từ 0 đến $len2$, tương ứng với số bước cần thiết để biến chuỗi rỗng thành chuỗi đích (chèn) hoặc biến chuỗi nguồn thành chuỗi rỗng (xóa). Sau đó, ma trận được điền theo công thức truy hồi:

$$dp[i][j] = \min(dp[i-1][j] + 1, dp[i][j-1] + 1, dp[i-1][j-1] + cost)$$

Trong đó $cost = 0$ nếu hai ký tự giống nhau và $cost = 1$ nếu khác nhau.

Giá trị tại ô cuối cùng của ma trận ($dp[len1][len2]$) chính là khoảng cách Levenshtein cần tìm.

Thuật toán Levenshtein có độ phức tạp $O(m \times n)$ với m và n lần lượt là độ dài của hai chuỗi cần so sánh.

```
// Trích từ src/fuzzy_search.c
int levenshteinDistance(wchar_t *s1, wchar_t *s2) {
    int len1 = wcslen(s1);
    int len2 = wcslen(s2);
```



```

int i, j;

int **arr = malloc((len1+1) * sizeof(int*));
for (i = 0; i <= len1; i++) {
    arr[i] = malloc((len2 + 1) * sizeof(int));
}
for (i = 0; i <= len1; i++) arr[i][0] = i;
for (j = 0; j <= len2; j++) arr[0][j] = j;

int cost;
for (i = 1; i <= len1; i++) {
    for (j = 1; j <= len2; j++) {
        cost = (s1[i-1] == s2[j-1]) ? 0 : 1;
        arr[i][j] = min3(
            arr[i-1][j] + 1,    // Xóa ký tự từ s1
            arr[i][j-1] + 1,    // Thêm ký tự vào s1
            arr[i-1][j-1] + cost // Thay thế hoặc giữ nguyên
        );
    }
}
int res = arr[len1][len2];
for (i = 0; i <= len1; i++) free(arr[i]);
free(arr);
return res;
}

```

Để tối ưu hóa hiệu năng và nâng cao trải nghiệm tìm kiếm, hệ thống tích hợp hai cơ chế kiểm soát lỗi nghiêm ngặt:

- Lọc thô theo độ dài: Chỉ thực hiện thuật toán Levenshtein nếu chênh lệch độ dài giữa từ khóa nhập vào (inputLen) và chuỗi đích dữ liệu (targetLen) nhỏ hơn 3 ký tự ($\text{abs}(\text{inputLen} - \text{targetLen}) < 3$), tránh việc tính toán ma trận lãng phí trên các cặp chuỗi không tương đồng.
- Ngưỡng lỗi động (Dynamic Max Gap): Hệ thống tự động điều chỉnh giới hạn sai lệch tối đa cho phép (current_max_gap) tùy thuộc vào độ dài của từ khóa đầu vào để tránh hiện tượng loãng kết quả (nhiều thông tin) khi người dùng nhập chuỗi quá ngắn:

- + Từ khóa ngắn (≤ 3 ký tự): Chỉ cho phép sai sót tối đa 1 lỗi ($\text{current_max_gap} = 1$) nhằm đảm bảo kết quả tìm kiếm có độ chính xác cao.
- + Từ khóa dài hơn (> 3 ký tự): Hệ thống khôi phục về giá trị cấu hình mặc định, cho phép sai sót tối đa 2 lỗi ($\text{current_max_gap} = \text{MAX_GAP}$ với $\#define \text{MAX_GAP } 2$).

```
// Trích từ src/fuzzy_search.c - fuzzySearching()
size_t inputLen = wcslen(wInput);
int current_max_gap = MAX_GAP;
if (inputLen <= 3) {
    current_max_gap = 1;
}
// ... Duyệt tập dữ liệu ...
if (target_w != NULL) {
    size_t targetLen = wcslen(target_w);
    if (abs((int)inputLen - (int)targetLen) < 3) {
        gap = levenshteinDistance(wInput, target_w);
    }
}
if (gap <= current_max_gap) {
    foundCount++;
    printFuzzyResult(gap, v, foundCount);
}
```

3.3.3. Thuật toán tìm kiếm chuỗi con KMP (Knuth-Morris-Pratt)

Thuật toán KMP được sử dụng để tìm kiếm một chuỗi con (pattern) bên trong một chuỗi văn bản (text) với độ phức tạp tuyến tính $O(m+n)$, trong đó m là độ dài của pattern và n là độ dài của text [18]. Điểm đặc biệt của KMP là khả năng không cần quay lui (backtracking) trên văn bản, tận dụng thông tin từ những phần đã khớp trước đó để nhảy qua những vị trí không cần thiết. Trong dự án, thuật toán này được sử dụng cho chức năng tìm kiếm chuỗi con (substring search), giúp tìm ra các từ vựng có chứa một chuỗi ký tự bất kỳ ở bất kỳ vị trí nào (đầu từ, giữa từ hoặc cuối từ).

Thuật toán KMP bao gồm hai giai đoạn chính: xây dựng mảng LPS và thực hiện tìm kiếm.

Giai đoạn 1 - Xây dựng mảng LPS (Longest Prefix Suffix):

Mảng LPS là một mảng số nguyên có độ dài bằng độ dài của pattern, trong đó $lps[i]$ lưu độ dài của tiền tố dài nhất của chuỗi $pat[0..i]$ cũng chính là hậu tố của chính chuỗi đó. Ví dụ, với pattern "ABABAC", mảng LPS được tính như sau:

i	0	1	2	3	4	5
pat[i]	A	B	A	B	A	C
lps[i]	0	0	1	2	3	0

Giải thích giá trị của mảng LPS: $lps[2] = 1$ vì "A" vừa là tiền tố vừa là hậu tố của "ABA"; $lps[3] = 2$ vì "AB" vừa là tiền tố vừa là hậu tố của "ABAB"; $lps[4] = 3$ vì "ABA" vừa là tiền tố vừa là hậu tố của "ABABA".

```
// Trích từ src/substring_search.c
void computeLPSArray(wchar_t *pat, int M, int *lps) {
    int len = 0;    // Độ dài của prefix/suffix dài nhất trước đó
    lps[0] = 0;    // LPS[0] luôn bằng 0
    int i = 1;

    while (i < M) {
        if (pat[i] == pat[len]) {
            len++;
            lps[i] = len;
            i++;
        } else {
            if (len != 0) {
                len = lps[len - 1]; // Quay lui về vị trí có thể khớp trước đó
            } else {
                lps[i] = 0;
                i++;
            }
        }
    }
}
```

Giai đoạn 2 - Thuật toán KMP chính:

Sau khi có mảng LPS, thuật toán KMP thực hiện tìm kiếm bằng cách duy trì hai con trỏ: i duyệt trên văn bản và j duyệt trên pattern. Khi hai ký tự khớp nhau, cả hai con trỏ đều tiến lên. Khi $j == M$, nghĩa là đã tìm thấy toàn bộ pattern trong văn bản, hàm trả về 1. Nếu không khớp và $j > 0$, con trỏ j được đặt về $lps[j-1]$ (nhảy đến vị trí an toàn thay vì quay lui hoàn toàn). Nếu không khớp và $j == 0$, con trỏ i tiến lên một bước.

```
// Trích từ src/substring_search.c
int KMPsearch(wchar_t *pat, wchar_t *txt) {
    int M = wcslen(pat);
    int N = wcslen(txt);
    int *lps = malloc(M * sizeof(int));
    computeLPSArray(pat, M, lps);
    int i = 0, j = 0; // i: chỉ số trên txt, j: chỉ số trên pat
    while (i < N) {
        if (pat[j] == txt[i]) {
            i++;
            j++;
        }
        if (j == M) { // Tìm thấy pattern hoàn chỉnh
            free(lps);
            return 1;
        } else if (i < N && pat[j] != txt[i]) {
            if (j != 0)
                j = lps[j - 1]; // Nhảy về vị trí an toàn thay vì quay lui
            else
                i++;
        }
    }
    free(lps);
    return 0;
}
```

3.3.4. Thuật toán ứng dụng cây tiền tố cho tìm kiếm Romaji

Cấu trúc dữ liệu cây tiền tố (Trie) được sử dụng để xây dựng một cấu trúc cây đa nhánh trên trường romaji của từ vựng, cho phép tìm kiếm tất cả các từ vựng có Romaji bắt đầu bằng một tiền tố nhất định với độ phức tạp $O(L)$, trong đó L là độ dài của tiền tố. Đây là thành phần cốt lõi của chức năng tìm kiếm tiền tố (prefix search), cho phép hệ thống gợi ý từ vựng theo thời gian thực khi người dùng nhập từng ký tự Romaji [15].

Bước 1 - Chuyển đổi ký tự sang chỉ số:

Hàm getCharIndex thực hiện việc ánh xạ một ký tự Romaji sang chỉ số tương ứng trong mảng 28 con trỏ. Ký tự được chuyển về dạng chữ thường trước khi xác định chỉ số: các chữ cái từ 'a' đến 'z' được ánh xạ vào khoảng 0-25, dấu gạch ngang '-' vào chỉ số 26, dấu chấm '.' vào chỉ số 27. Nếu ký tự không thuộc các trường hợp trên, hàm trả về -1 để báo lỗi.

```
// Trích từ src/prefix_search.c
#define ALPHABET_SIZE 28
int getCharIndex(char c) {
    c = tolower(c);
    if (c >= 'a' && c <= 'z') return c - 'a'; // 0-25
    if (c == '-') return 26; // index 26 cho dấu gạch ngang
    if (c == '.') return 27; // index 27 cho dấu chấm
    return -1; // Ký tự không hợp lệ
}
```

Bước 2 - Chèn từ vào Trie:

Duyệt dọc theo cây từ gốc, tạo nút mới nếu nhánh tương ứng chưa tồn tại. Tại nút cuối cùng của từ, đánh dấu isEndOfWord = 1 và lưu con trỏ tới đối tượng từ vựng.

```
// Trích từ src/prefix_search.c
void insertTrie(TrieNode *root, const char *key, Vocab *vocab) {
    TrieNode *current = root;
    // Duyệt theo từng ký tự của key (Romaji)
    for (int i = 0; key[i] != '\0'; i++) {
        int index = getCharIndex(key[i]);
        if (index == -1) continue; // Bỏ qua ký tự không hợp lệ

        if (!current->children[index]) {
```

```

        current->children[index] = createTrieNode();
    }
    current = current->children[index];
}
// Đánh dấu node hiện tại là kết thúc một từ
current->isEndOfWord = 1;
// Thêm con trỏ Vocab vào mảng động của node
Vocab **temp = realloc(current->vocabs, (current->vocabsCount + 1) *
sizeof(Vocab*));
if (temp) {
    current->vocabs = temp;
    current->vocabs[current->vocabsCount++] = vocab;
}
}

```

Bước 3 - Tìm kiếm node theo tiền tố:

Di chuyển trên cây theo các ký tự của tiền tố đầu vào. Nếu gặp nhánh NULL, tiền tố không tồn tại.

```

// Trích từ src/prefix_search.c
TrieNode* searchPrefixNode(TrieNode *root, const char *prefix) {
    TrieNode *current = root;
    for (int i = 0; prefix[i] != '\0'; i++) {
        int index = getCharIndex(prefix[i]);
        if (index == -1) return NULL;    // Ký tự không hợp lệ
        if (!current->children[index]) {
            return NULL;                // Tiền tố không tồn tại
        }
        current = current->children[index];
    }
    return current; // Trả về node tại vị trí kết thúc tiền tố
}

```

Bước 4 - Duyệt toàn bộ từ từ một node:

Từ nút tiền tố vừa tìm được, thực hiện duyệt theo chiều sâu để thu thập tất cả các từ vựng thuộc các nút con phía dưới.

```

// Trích từ src/prefix_search.c
void printAllWordsFromNode(TrieNode *node, int *counter) {
    if (!node) return;

    // Nếu node đánh dấu kết thúc từ, in tất cả Vocab tại node này
    if (node->isEndOfWord && node->vocabs) {
        for (int i = 0; i < node->vocabsCount; i++) {
            (*counter)++;
            Vocab *v = node->vocabs[i];
            // In thông tin từ vựng...
        }
    }

    // đệ quy duyệt tất cả các nhánh con
    for (int i = 0; i < ALPHABET_SIZE; i++) {
        if (node->children[i]) {
            printAllWordsFromNode(node->children[i], counter);
        }
    }
}

```

Bảng 3.1: Bảng so sánh nhanh các tiêu chí của 04 thuật toán tìm kiếm

Tiêu chí	Hash Table (Exact Search)	Trie (Prefix Search)	KMP (Substring Search)	Levenshtein (Fuzzy Search)
Cấu trúc dữ liệu nền tảng	Bảng băm liên kết (Chaining)	Cây tiền tố đa nhánh (28 nhánh)	Mảng tiền tố (LPS) + Duyệt tuần tự	Ma trận quy hoạch động (DP)
Độ phức tạp thời gian (Lý thuyết)	$O(L)$ băm + $O(1)$ trung bình tìm kiếm	$O(L)$ theo độ dài từ khóa	$O(N + M)$ với N, M là độ dài văn bản và mẫu	$O(m \times n)$ với m, n là độ dài hai chuỗi

Độ phức tạp bộ nhớ (Space)	$O(K)$ với K là số lượng khóa	$O(\Sigma \text{ tổng độ dài tất cả các từ})$	$O(M)$ cho mảng tiền tố LPS	$O(m \times n)$ cho ma trận trạng thái
Thời gian thực thi (Trường hợp xấu)	$O(n)$ khi xảy ra xung đột cao	$O(L + \text{số lượng kết quả})$	$O(N \times M)$	$O(m \times n)$ cố định theo ma trận
Tìm kiếm gần đúng (Fuzzy)	Không hỗ trợ	Không hỗ trợ	Không hỗ trợ	Hỗ trợ tối ưu (Ngưỡng sai số ≤ 2)
Tìm kiếm theo tiền tố (Prefix)	Không hỗ trợ	Hỗ trợ tối ưu	Không hỗ trợ	Không hỗ trợ
Tìm kiếm chuỗi con (Substring)	Không hỗ trợ	Không hỗ trợ	Hỗ trợ tối ưu	Hiệu suất thấp, không khuyến nghị
Tìm kiếm chính xác (Exact)	Hỗ trợ tối ưu	Khả thi nhưng hiệu suất không cao	Khả thi nhưng tốc độ xử lý chậm	Khả thi nhưng tốc độ xử lý chậm
Độ chính xác kết quả	100% (Khớp hoàn toàn)	100% (Khớp chuỗi tiền tố)	100% (Khớp phân đoạn chuỗi)	70% - 95% (Phụ thuộc vào cấu hình ngưỡng)
Khả năng xử lý Unicode (wchar_t)	Tương thích hoàn toàn	Tương thích thông qua Romaji	Tương thích hoàn toàn	Tương thích hoàn toàn
Kịch bản thực tế	"Tôi nhớ chính xác từ 先生"	"Tôi chỉ nhớ bắt đầu bằng ka-"	"Tôi chỉ nhớ bắt đầu bằng ka-"	"Tôi gõ sai 'gakkou' thành 'gakou'"

3.3.5. Thuật toán lọc từ vựng dựa trên Kanji đã học

Thuật toán lọc từ vựng dựa trên Kanji đã học nhằm trích xuất các từ vựng thỏa mãn đồng thời hai điều kiện: thứ nhất, từ vựng phải chứa ít nhất một Kanji thuộc các

bài học do người dùng chọn; thứ hai, toàn bộ các Kanji có trong từ vựng đó đều phải nằm trong tập hợp các Kanji đã được học từ bài 1 đến bài học tối đa mà người dùng đã tiếp cận. Thuật toán này được triển khai trong module `src/filter_learned_lesson.c`, phục vụ chức năng lọc từ vựng đã học, giúp người dùng ôn tập các từ vựng phù hợp với trình độ hiện tại.

Các bước thực hiện:

Bước 1 - Xây dựng tập hợp Kanji đã học (LearnedHashSet):

Quá trình xây dựng tập hợp Kanji đã học bắt đầu bằng việc duyệt qua tất cả các Kanji thuộc các bài học được chọn từ bài 1 đến bài học tối đa (`maxLesson`) do người dùng xác định. Với mỗi bài học, chỉ số bắt đầu và kết thúc được tính dựa trên hằng số `KANJI_PER_LESSON = 16`. Tại mỗi Kanji, chữ Kanji dạng `wchar_t*` được thêm vào `LearnedHashSet` thông qua hàm `addLearnedKanji`. Đồng thời, nếu bài học hiện tại nằm trong danh sách các bài học được người dùng chọn, Kanji đó cũng được thêm vào `targetKanjiList` để phục vụ việc kiểm tra điều kiện "chứa Kanji từ bài đã chọn".

```
// Trích từ src/filter_learned_lesson.c
// Duyệt qua tất cả Kanji của các bài học được chọn (từ bài 1 đến maxLesson)
for (int lesson = 1; lesson <= maxLesson; lesson++) {
    int start = (lesson - 1) * KANJI_PER_LESSON;
    int end = start + KANJI_PER_LESSON;
    for (int j = start; j < end; j++) {
        wchar_t* kanjiW = allData->kanjis[j].kanji_w;
        addLearnedKanji(learnedHash, kanjiW); // Thêm vào hash set
        // Nếu bài học này được chọn, thêm vào targetKanjiList
        if (isSelectedLesson(lesson)) {
            targetKanjiList[targetCount++] = kanjiW;
        }
    }
}
```

Bước 2 - Hàm băm cho Kanji (trên ký tự rộng):

`LearnedHashSet` sử dụng hàm băm DJB2 biến thể cho kiểu dữ liệu `wchar_t*`. Hàm băm này khởi tạo giá trị `hash = 5381`, duyệt qua từng ký tự trong chuỗi và cập nhật `hash = ((hash << 5) + hash) + *str++`. Sau khi duyệt toàn bộ chuỗi, giá trị băm

được lấy modulo với `HASH_SIZE = 256` để ánh xạ vào một trong 256 bucket của bảng băm.

```
// Trích từ src/filter_learned_lesson.c
unsigned int hashKanji(const wchar_t* str) {
    unsigned int hash = 5381;
    while (*str) {
        hash = ((hash << 5) + hash) + *str++;
    }
    return hash % HASH_SIZE; // HASH_SIZE = 256
}
```

Bước 3 - Kiểm tra từ vựng:

Quá trình kiểm tra từ vựng được thực hiện bằng cách duyệt từng ký tự trong chuỗi `vocab_w`; nếu ký tự là Kanji (xác định bằng `isKanjiWChar`), nó được tách thành chuỗi đơn `singleKanji` và kiểm tra hai điều kiện: thứ nhất, Kanji có nằm trong `targetKanjiList` (các Kanji thuộc bài học đã chọn) để thiết lập cờ `hasTargetKanji`; thứ hai, Kanji có nằm trong `LearnedHashSet` (tập Kanji đã học) thông qua `containsLearnedKanji`; nếu bất kỳ Kanji nào không thỏa mãn điều kiện thứ hai, biến `allKanjiKnown` được đặt bằng 0 và vòng lặp kết thúc sớm để tối ưu; cuối cùng, từ vựng chỉ được xuất ra khi cả hai điều kiện đều đúng (`hasTargetKanji` && `allKanjiKnown`).

```
// Trích từ src/filter_learned_lesson.c
const wchar_t* text = vocab->vocab_w;
int hasTargetKanji = 0; // Có chứa Kanji từ bài đã chọn?
int allKanjiKnown = 1; // Tất cả Kanji đều đã học?
for (int pos = 0; text[pos] != L'\0'; pos++) {
    wchar_t ch = text[pos];
    if (isKanjiWChar(ch)) { // Nếu là ký tự Kanji
        wchar_t singleKanji[2] = {ch, L'\0'};

        // Kiểm tra điều kiện (1): có nằm trong targetKanjiList không?
        if (!hasTargetKanji) {
            for (int t = 0; t < targetCount; t++) {
                if (wcscmp(singleKanji, targetKanjiList[t]) == 0) {
```

```

        hasTargetKanji = 1;
        break;
    }
}
}
// Kiểm tra điều kiện (2): có nằm trong LearnedHashSet không?
if (!containsLearnedKanji(learnedHash, singleKanji)) {
    allKanjiKnown = 0;
    break; // Thoát sớm nếu phát hiện Kanji chưa học
}
}
// Chỉ xuất từ vựng nếu thỏa mãn cả hai điều kiện
if (hasTargetKanji && allKanjiKnown) {
    // In từ vựng
}

```

3.3.6. Thuật toán phân tích các Kanji trong câu

Thuật toán phân tích câu nhận một câu tiếng Nhật từ người dùng, trích xuất tất cả các ký tự Kanji có trong câu, sau đó tra cứu và hiển thị thông tin chi tiết của từng Kanji từ cơ sở dữ liệu. Chức năng này được triển khai trong module `src/sentence_analysis.c`, cho phép người dùng nhập bất kỳ câu tiếng Nhật nào và nhận được giải thích ngay lập tức về các Kanji xuất hiện trong câu đó, hỗ trợ quá trình đọc hiểu và học tập.

Để xác định một ký tự Unicode có phải là Kanji hay không, hệ thống sử dụng hàm `isKanjiWChar` kiểm tra mã Unicode của ký tự đó. Kanji trong bảng mã Unicode thuộc hai khoảng chính: CJK Unified Ideographs (0x4E00 – 0x9FFF) [14] chứa khoảng 20.992 ký tự Kanji thông dụng, và CJK Unified Ideographs Extension A (0x3400 – 0x4DBF) [13] chứa khoảng 6.582 ký tự Kanji mở rộng. Nếu ký tự thuộc các khoảng mã này, nó được xác định là Kanji.

```

// Trích từ src/utils.h
int isKanjiWChar(wchar_t wc) {
    return (wc >= 0x4E00 && wc <= 0x9FFF) || // CJK Unified Ideographs (20992 ký
tự)

```

```
(wc >= 0x3400 && wc <= 0x4DBF);    // CJK Unified Ideographs Extension A (6582 ký tự)
}
```

Xây dựng bảng băm Kanji tạm thời:

```
// Trích từ src/sentence_analysis.c
#define HASH_SIZE_SENTENCE 512
#define HASH_SIZE 512

unsigned int hashKanjiChar(wchar_t wc) {
    unsigned long hash = 5381;
    hash = ((hash << 5) + hash) + (unsigned long)wc;
    return hash % HASH_SIZE;
}

KanjiHashTable* createKanjiHashTable(KanjiList *allData) {
    KanjiHashTable* table = calloc(1, sizeof(KanjiHashTable));
    for (int i = 0; i < allData->kanjiCount; i++) {
        wchar_t* kw = allData->kanjis[i].kanji_w;
        if (!kw || !kw[0]) continue;
        unsigned int idx = hashKanjiChar(kw[0]); // Băm dựa trên ký tự đầu tiên (cũng là duy nhất)

        KanjiNode* node = malloc(sizeof(KanjiNode));
        node->kanjiData = &allData->kanjis[i];
        node->next = table->buckets[idx];
        table->buckets[idx] = node;
    }
    return table;
}
```

Quy trình phân tích câu:

```
// Trích từ src/sentence_analysis.c
void analyzeJapaneseSentence(KanjiList *allData) {
    // 1. Xây dựng bảng băm Kanji tạm thời
```

```

KanjiHashTable* kanjiTable = createKanjiHashTable(allData);
// 2. Nhập câu từ người dùng
char sentence[MAX_SENTENCE];
inputString(sentence, MAX_SENTENCE);
// 3. Chuyển sang wchar_t để xử lý Unicode
wchar_t* wsentence = convertToWchar(sentence);
// 4. Duyệt từng ký tự trong câu
for (int i = 0; wsentence[i] != L'\0'; i++) {
    wchar_t ch = wsentence[i];
    // 5. Nếu là ký tự Kanji, tra cứu trong bảng băm
    if (isKanjiWChar(ch)) {
        Kanji* info = findKanji(kanjiTable, ch);
        if (info) {
            printOutKanjiInfo(info); // In thông tin chi tiết
        }
    }
}
// 6. Giải phóng bảng băm tạm thời
freeKanjiHashTable(kanjiTable);
free(wsentence);
}

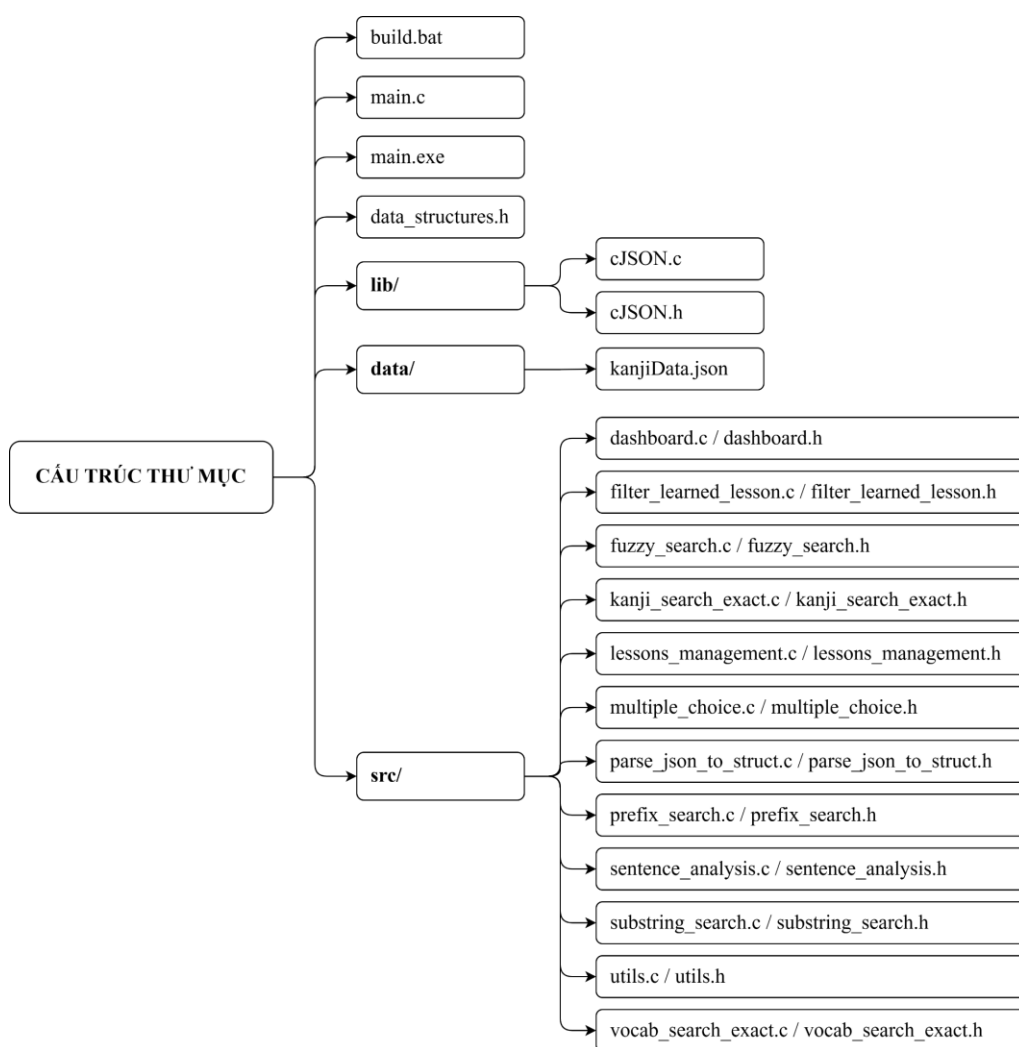
```

4. CHƯƠNG TRÌNH VÀ KẾT QUẢ

4.1. Tổ chức chương trình

4.1.1. Cấu trúc thư mục dự án

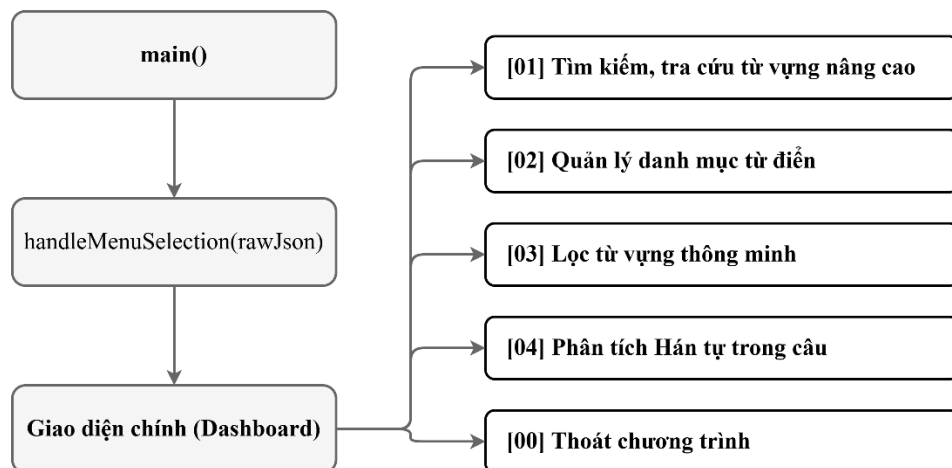
Chương trình được tổ chức thành các thư mục và tệp riêng biệt. Tệp khởi tạo và thực thi gồm main.c, main.exe và script build.bat. Thư viện ngoài được đặt trong thư mục lib/ với các tệp cJSON.c và cJSON.h. Dữ liệu đầu vào nằm trong thư mục data/ với tệp kanjiData.json. Toàn bộ mã nguồn được triển khai trong thư mục src/, bao gồm nhiều mô-đun chức năng như dashboard, filter_learned_lesson, fuzzy_search, kanji_search_exact, prefix_search, sentence_analysis, ...



Hình 4.1: Sơ đồ cấu trúc thư mục dự án

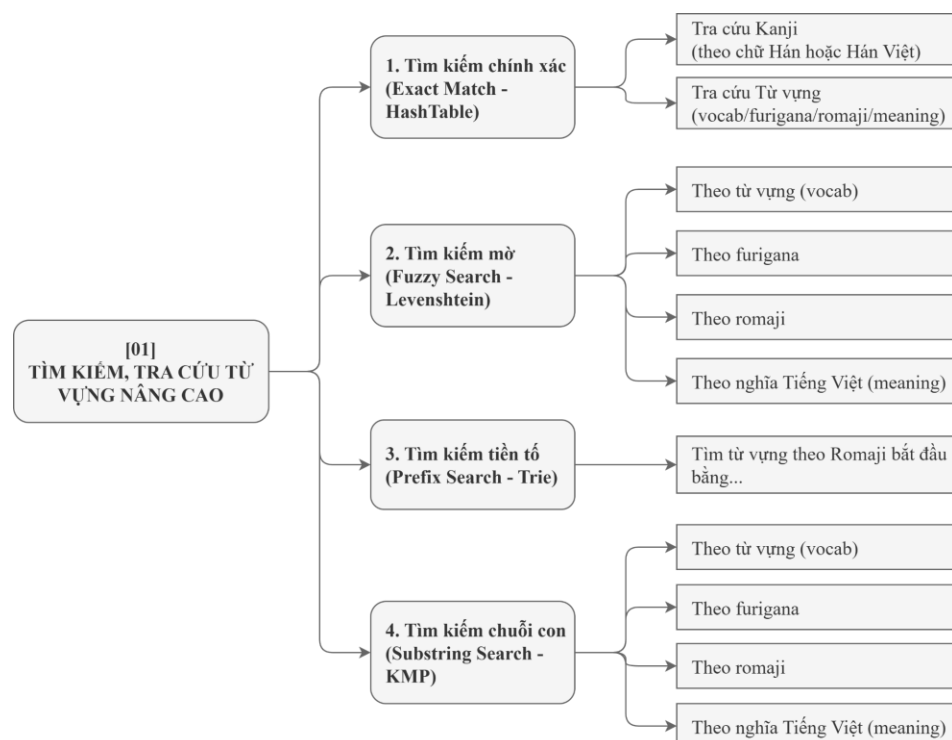
4.1.2. Tổ chức các thành phần chức năng

a) Phần Dashboard



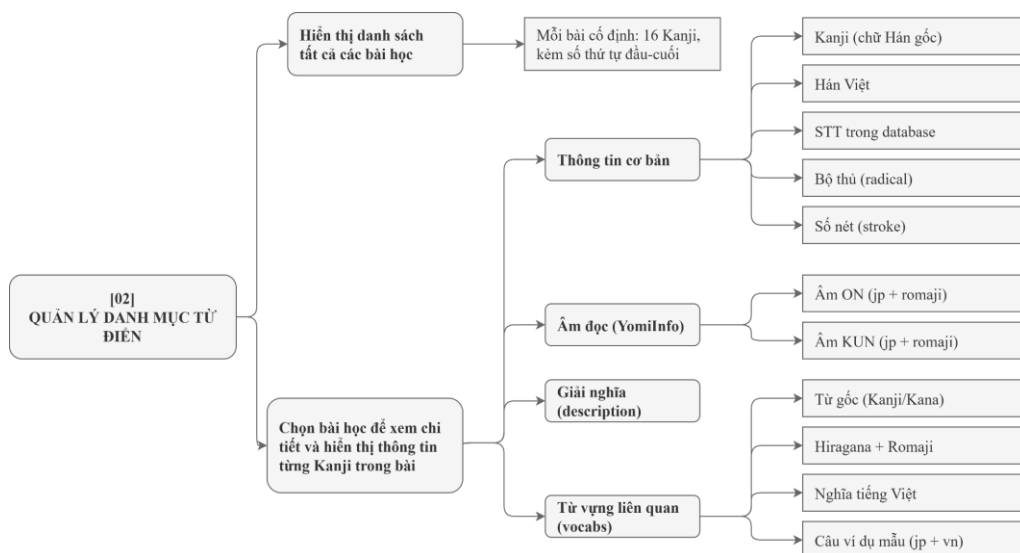
Hình 4.2: Sơ đồ tổ chức các thành phần chức năng (Giao diện chính)

b) Phần [01] Tìm kiếm, tra cứu từ vựng nâng cao



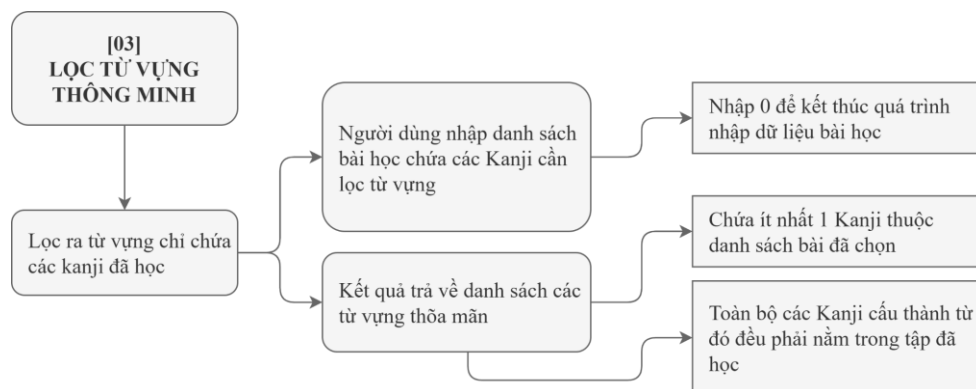
Hình 4.3: Sơ đồ cấu trúc các thuật toán tìm kiếm

c) Phần [02] Quản lý danh mục từ điển



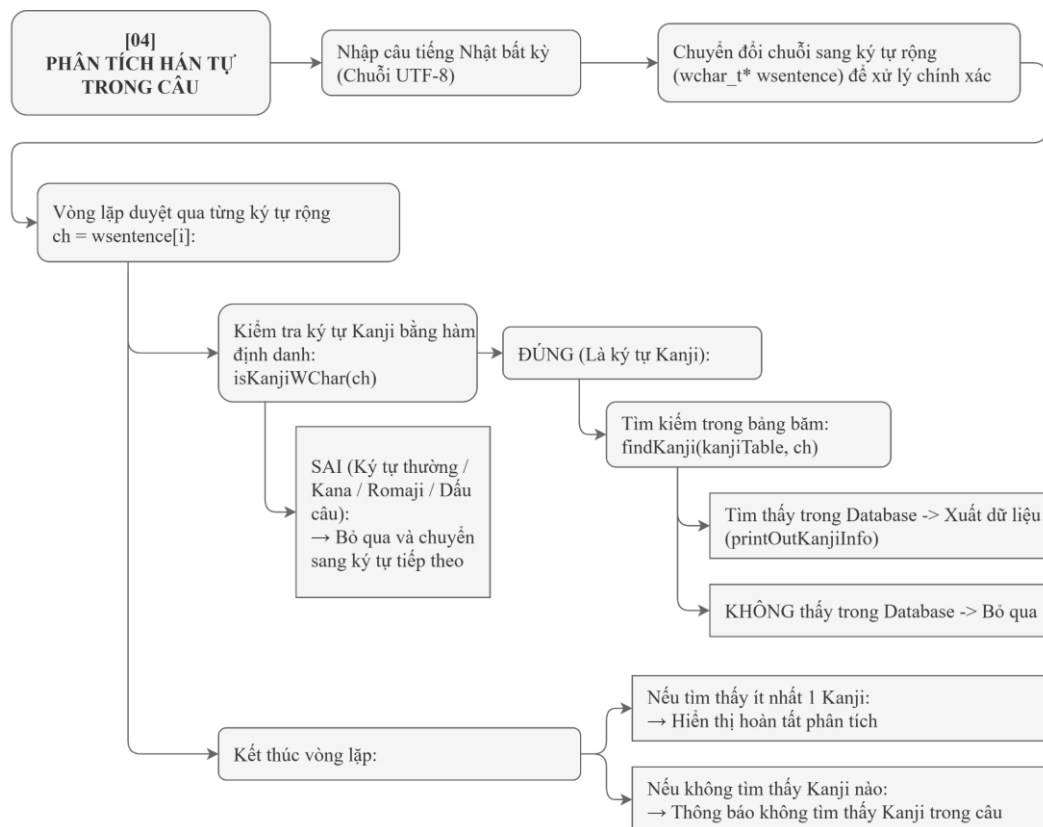
Hình 4.4: Sơ đồ logic chức năng hiển thị và quản lý thông tin từ điển

d) Phần [03] Lọc từ vựng thông minh



Hình 4.5: Sơ đồ cách hoạt của chức năng lọc từ vựng

e) Phần [04] Phân tích Kanji (Hán tự) trong câu



Hình 4.6: Sơ đồ tổng quan các logic xử lý chức năng phân tích câu

4.2. Ngôn ngữ cài đặt

4.2.1. Ngôn ngữ, thư viện và kỹ thuật cài đặt

- Ngôn ngữ lập trình: Sử dụng ngôn ngữ C (tiêu chuẩn C99/C11)
- Trình biên dịch: GCC (MinGW-w64 hoặc MSYS2) và được cấu hình biến môi trường PATH
- Thư viện sử dụng:
- Tiêu chuẩn: stdio.h, stdlib.h, string.h, wchar.h, time.h, ctype.h, windows.h
- Bên thứ ba: cJSON
- Mã hóa: UTF-8 (đầu vào/đầu ra), wchar_t (xử lý nội bộ)
- Giao diện: Giao diện dòng lệnh (CLI - Command Line Interface) kết hợp sử dụng mã màu ANSI 24-bit (True Color) [19] với các macro tự định nghĩa như CL_KANJI, CL_KANA, CL_PRIMARY,... [20]

4.2.2. *Hướng dẫn biên dịch và chạy chương trình*

Để thực thi chương trình trên môi trường Windows, người dùng cần mở Command Prompt (CMD) hoặc PowerShell hoặc Windows Terminal và thực hiện tuần tự các bước sau.

Bước 1 – Kiểm tra trình biên dịch C: Trước tiên, cần đảm bảo hệ thống đã cài đặt trình biên dịch C (MinGW-w64 hoặc GCC). Người dùng nhập lệnh `gcc --version` để kiểm tra phiên bản. Nếu chưa có, cần cài đặt MinGW-w64 và thêm đường dẫn bin vào biến môi trường PATH trước khi tiếp tục.

Bước 2 – Điều hướng đến thư mục dự án: Sử dụng lệnh `cd` để di chuyển vào thư mục chứa mã nguồn, ví dụ: `cd C:\Users\TenNguoiDung\Documents\Code Project`.

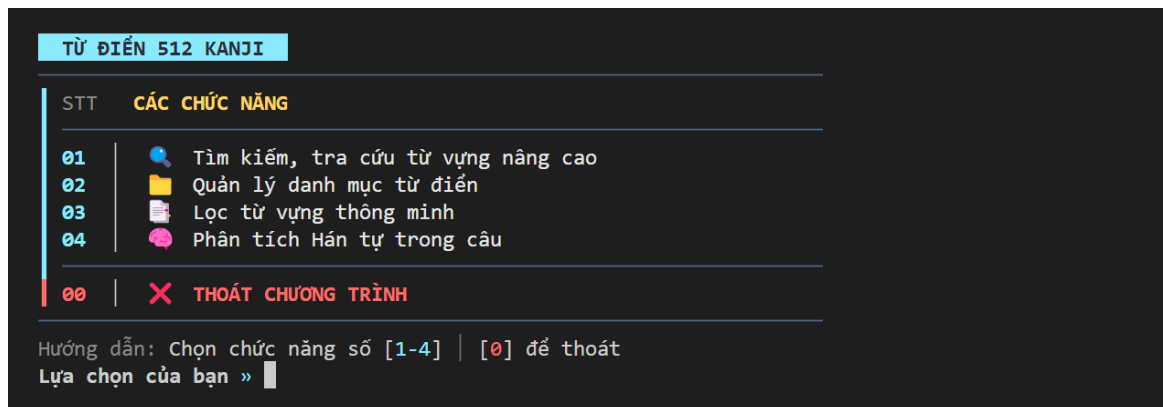
Bước 3 – Biên dịch chương trình: Thực thi tệp batch build.bat bằng lệnh `build.bat` hoặc `.\build.bat`. Tệp batch là một tệp văn bản chứa các dòng lệnh được thực thi tuần tự bởi Command Prompt, tương tự như một kịch bản (script) tự động hóa thao tác trên Windows. Tệp này sẽ tự động gọi gcc với tất cả các tham số cần thiết, liên kết các module của project cùng với các thư mục chứa tệp tiêu đề -Ilib -Isrc. Quá trình biên dịch tạo ra tệp thực thi main.exe. Trong trường hợp biên dịch thất bại, tệp batch hiển thị thông báo lỗi và dừng quá trình, giúp người dùng dễ dàng nhận diện và khắc phục sự cố.

Bước 4 – Chạy chương trình: Sau khi biên dịch thành công, nhập lệnh `main.exe` hoặc `.\main.exe` để khởi chạy ứng dụng. Chương trình sẽ tự động đọc và parse dữ liệu từ data/kanjiData.json, sau đó hiển thị menu chính để người dùng tương tác.

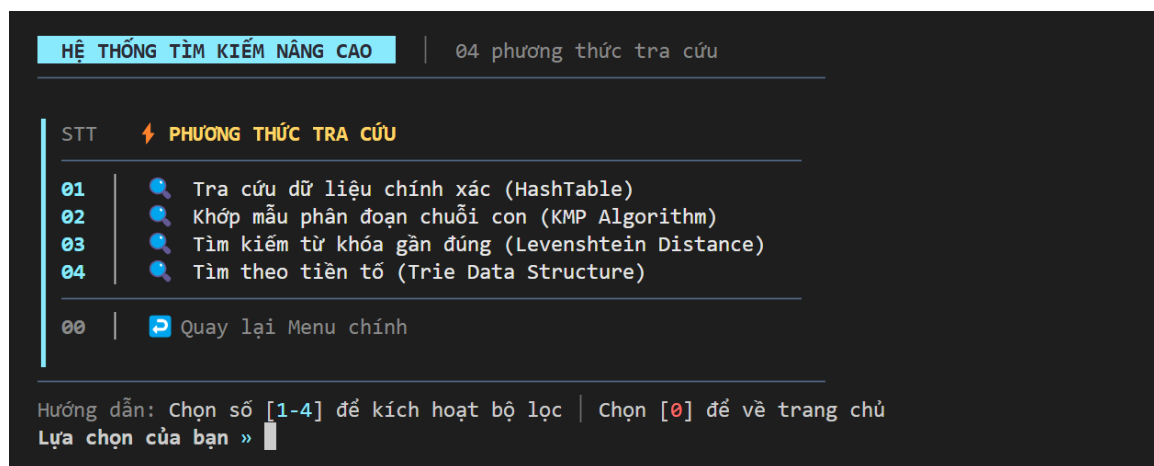
Bước 5 – Thoát chương trình: Người dùng có thể nhấn tổ hợp phím `Ctrl + C` để thoát khỏi chương trình bất kỳ lúc nào, hoặc lựa chọn tùy chọn thoát trong menu chính của chương trình.

4.3. Kết quả

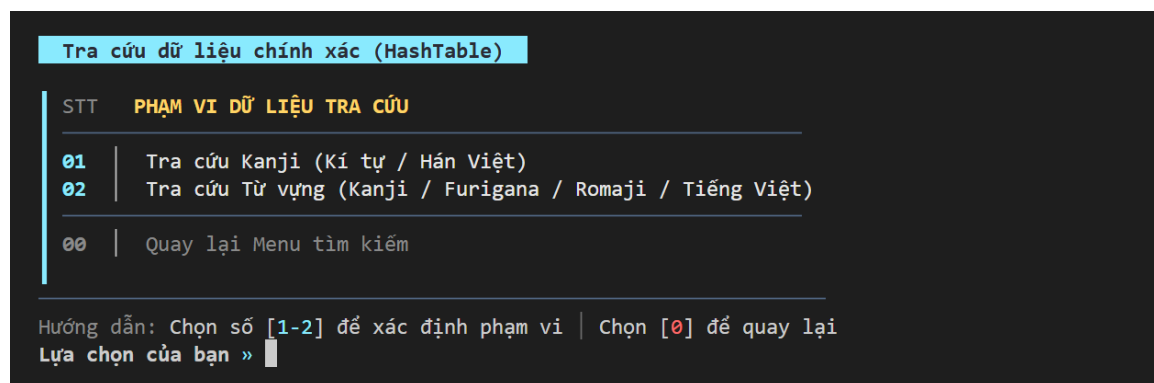
4.3.1. Giao diện chính của chương trình



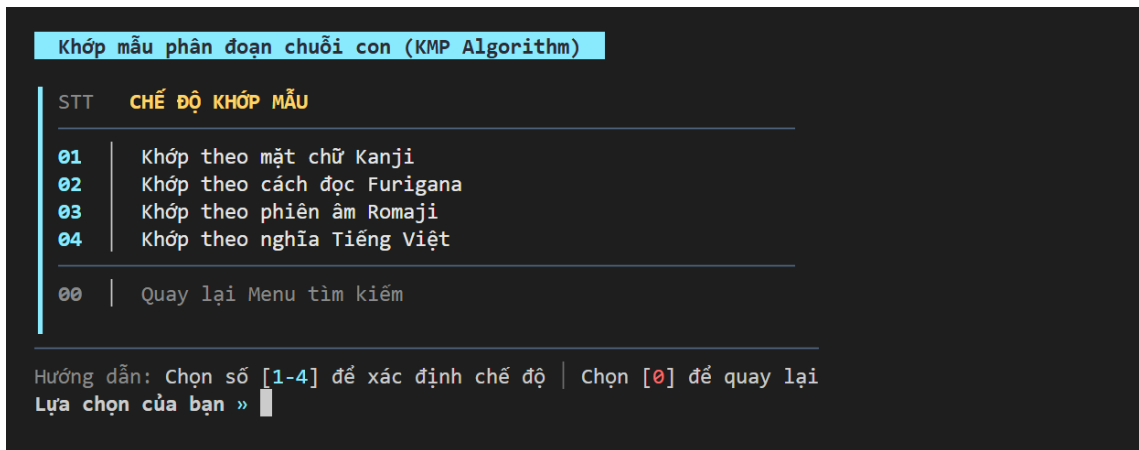
Hình 4.7: Giao diện chính kèm danh sách các chức năng



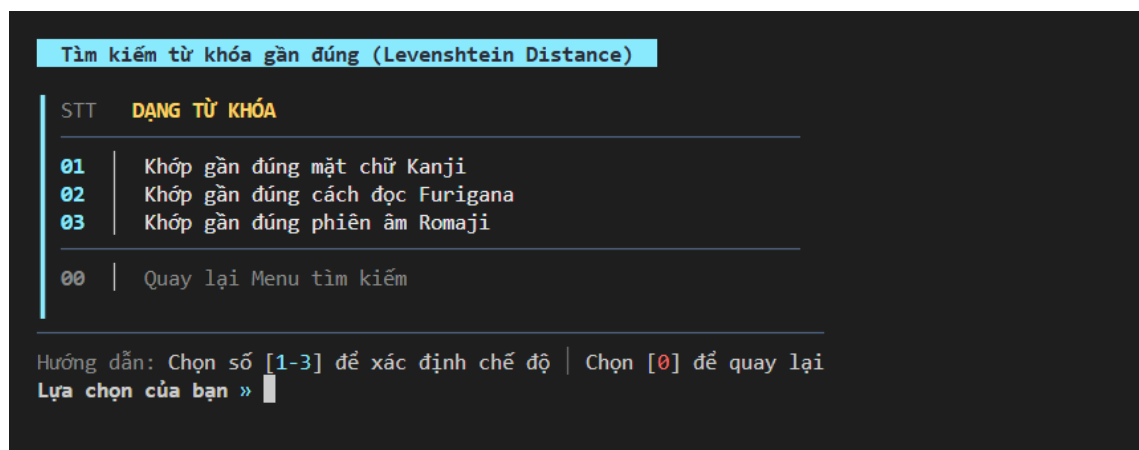
Hình 4.8: Giao diện lựa 04 phương thức tìm kiếm



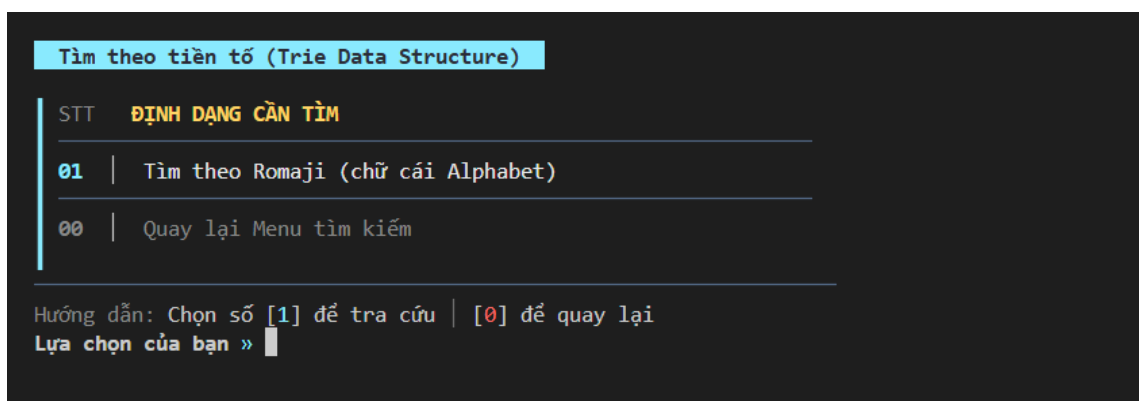
Hình 4.9: Giao diện chức năng tìm kiếm chính xác (Hashtable)



Hình 4.10: Giao diện các lựa chọn chức năng tra cứu theo chuỗi con (KMP Algorithm)



Hình 4.11: Giao diện chức năng tìm kiếm mờ (Levenshtein Distance)



Hình 4.12: Giao diện chức năng tìm kiếm theo tiền tố (Trie)

DANH SÁCH BÀI HỌC TỔNG HỢP		
Bài 01	(1 - 16)	二 月 下 力 行 小 出 走 夏 川 理 菜 矢 室 近 動 始 官 顔 戸 熱 泣 忘 部 和 泳 疲 婚 初 賀 失 機 神 陰
Bài 02	(17 - 32)	一 日 上 田 見 大 入 休 春 山 料 音 家 教 遠 運 医 図 頭 場 丸 笑 覺 全 平 遊 吉 取 勝 飛 申 危
Bài 03	(33 - 48)	三 火 中 男 米 高 市 起 秋 林 反 歌 族 羽 者 止 終 館 声 所 冷 怒 決 必 戰 疲 婚 初 賀 失 機 神 陰
Bài 04	(49 - 64)	四 水 外 女 來 安 町 貝 冬 森 飯 自 親 習 暑 步 石 昔 特 屋 甘 幸 定 要 爭 涼 供 歲 絕 速 調 戾
Bài 05	(65 - 80)	五 木 右 子 良 新 村 買 朝 空 牛 軋 兄 漢 寒 使 研 借 別 堂 汚 悲 比 荷 政 涼 兩 枚 對 遲 查 吸
Bài 06	(81 - 96)	六 金 工 學 食 古 雨 壳 晝 海 豚 棄 姉 字 重 送 究 代 竹 都 果 苦 受 由 治 靜 兩 枚 對 遲 查 吸
Bài 07	(97 - 112)	七 士 左 生 飲 元 電 読 夕 化 鳥 写 弟 式 輕 洗 留 貸 合 巢 卵 痛 授 屈 經 公 若 冊 繞 駐 相 放
Bài 08	(113 - 128)	八 曜 前 先 會 氣 車 書 方 花 肉 真 妹 試 低 急 有 地 答 區 皿 恥 徒 利 濟 國 老 億 辭 泊 談 宴
Bài 09	(129 - 144)	九 本 後 何 耳 多 馬 婦 晚 天 茶 台 私 驗 弱 開 產 世 正 池 酒 配 練 弘 法 込 息 点 投 船 案 齒
Bài 10	(145 - 160)	十 人 午 父 間 少 駅 勉 夜 赤 予 央 夫 宿 惡 題 暗 押 藥 度 計 建 付 辛 表 寝 際 怒 奧 段 約 席 君 繪
Bài 11	(161 - 176)	百 今 門 母 言 広 社 弓 心 青 野 映 妻 題 暗 押 藥 度 計 建 付 辛 表 寝 際 怒 奧 段 約 席 君 繪
Bài 12	(177 - 192)	千 寺 間 年 話 早 校 虫 手 白 菜 画 主 文 太 引 働 回 京 物 片 眠 卒 踊 閨 側 將 号 束 島 達 横
Bài 13	(193 - 208)	万 時 東 去 立 長 店 強 足 黒 切 羊 住 英 豆 思 員 用 集 品 焼 殘 違 活 係 葉 祖 倍 守 陸 星 当
Bài 14	(209 - 224)	円 半 西 每 待 明 銀 持 体 色 作 洋 糸 質 短 知 士 民 不 旅 消 念 役 未 義 景 育 次 過 港 雪 伝
Bài 15	(225 - 240)	口 刀 南 王 周 好 病 名 首 魚 未 服 氏 間 光 考 仕 注 便 通 固 感 皆 宅 議 記 性 々 夢 橋 降 細
Bài 16	(241 - 256)	目 分 北 國 週 友 院 語 道 犬 味 着 紙 說 風 死 事 意 以 進 個 情 彼 祭 党 形 招 他 的 交 直 無

>> Bạn cần xem bài số:

Hình 4.13: Giao diện quản lý toàn bộ kanji và các bài học

4.3.2. Kết quả thực thi của chương trình

a) Tra cứu dữ liệu chính xác (HashTable)

KẾT QUẢ TRA CỨU DỮ LIỆU

Từ khóa: "学"

Kanji

: 学

Hán Việt

: học

STT: 54

Bộ thủ

: ヨ, 冫, 子

Số nét: 8

Giải nghĩa

: học, study, learning, science

[Âm ON]

: ガク (gaku)

[Âm KUN]

: まな.ぶ (mana.bu)

Từ vựng liên quan (8):

01. 学生 (がくせい • gakusei) học sinh, sinh viên

わたしは 学生 です。

Tôi là sinh viên.

あの 学生 は 熱心です。

Sinh viên đó rất nhiệt huyết.

学生 の 本です。

Sách của sinh viên.

02. 大学 (だいがく • daigaku) đại học

大学 に いきます。

Đi đến trường đại học.

大学 で べんきょうします。

Học ở trường đại học.

おおい 大学 です。

Một trường đại học lớn nhĩ.

03. 学校 (がっこう • gakkou) trường học

学校 へ いきましょう。

Chúng ta hãy cùng đến trường nào.

学校 は 八時から です。

Trường học bắt đầu từ 8 giờ.

わたしの 学校 は ことです。

Trường của tôi ở đây.

04. 学部 (がくぶ • gakubu) ngành học

KẾT QUẢ TRA CỨU DỮ LIỆU

Từ khóa: "nhân"

Kanji : 人

Hán Việt : nhân STT: 26

Bộ thủ : 人 Số nét: 2

Giải nghĩa : nhân, person

[Âm ON] : ジン (jin), ニン (nin)

[Âm KUN] : ひと (hito), -り (-ri), -と (-to)

Từ vựng liên quan (8):

01. 人 (ひと • hito) người

あそこ に 人 が います。

Có người ở đằng kia.

あの 人 は 先生です。

Người đó là giáo viên.

どの 人 ですか。

Là người nào vậy?

02. 日本人 (にほんじん • nihonjin) người nhật

わたしは 日本人 です。

Tôi là người Nhật.

あの 人 は 日本人 ですか。

Người đó là người Nhật phải không?

日本人 の 友達 が います。

Tôi có bạn là người Nhật.

03. 一人 (ひとり • hitori) một người

へやに 一人 います。

Có một người ở trong phòng.

きょうだいは 一人 です。

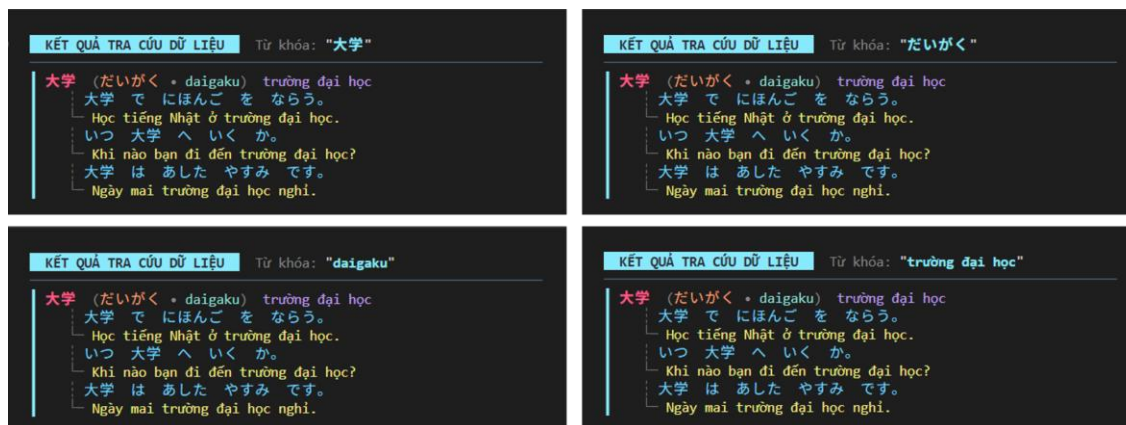
Tôi có một anh chị em.

りんごを 一人 で たべました。

Tôi đã ăn táo một mình.

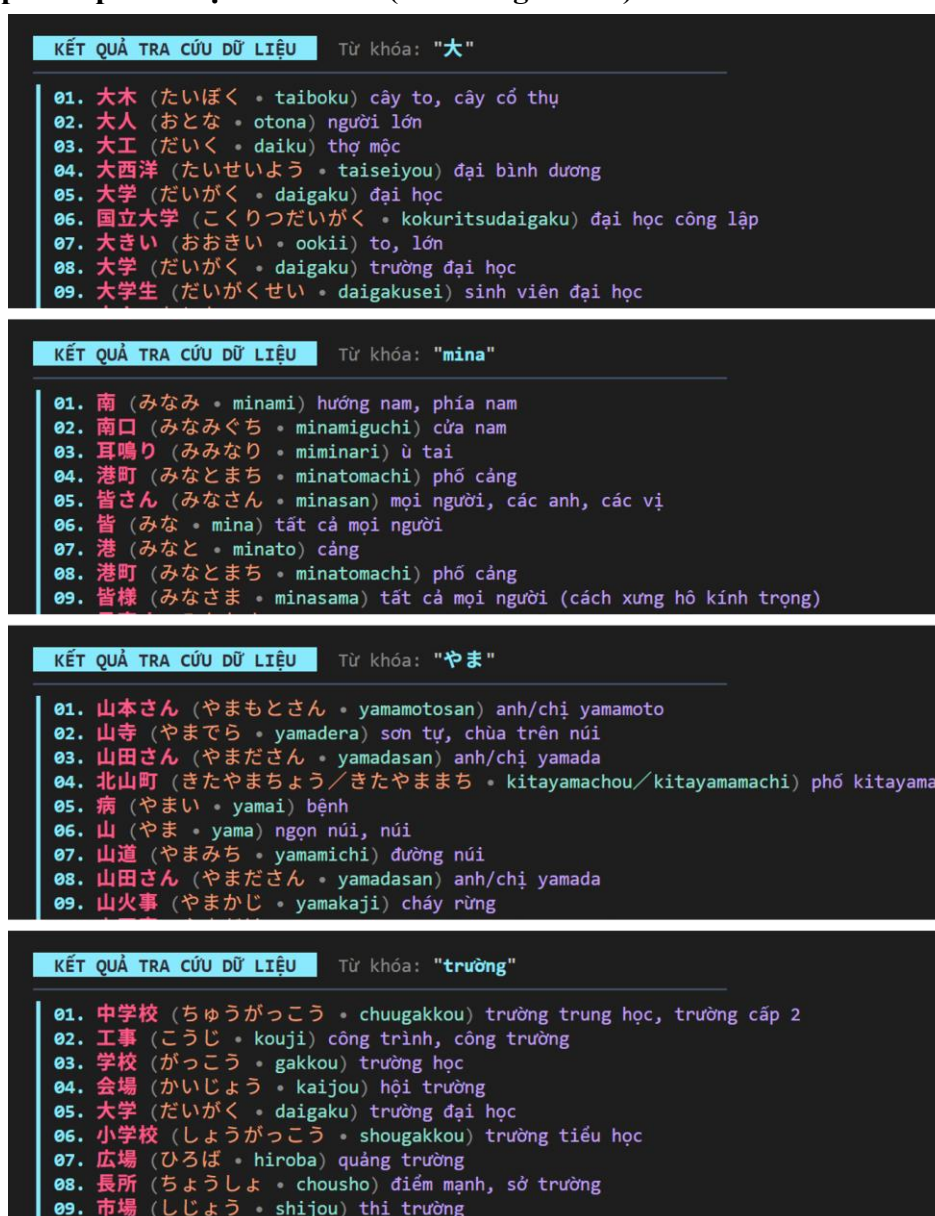
04. 二人 (ふたり • futari) hai người

Hình 4.14: Giao diện kết quả tìm kiếm chính xác với Kanji và Hán Việt



Hình 4.15: Giao diện kết quả tìm kiếm chính xác (Kanji, Furigana, Romaji và Tiếng Việt)

b) Khớp mẫu phân đoạn chuỗi con (KMP Algorithm)



Hình 4.16: Giao diện kết quả tìm kiếm theo chuỗi con (Kanji, Romaji, Furigana, Tiếng Việt)

c) Tìm kiếm từ khóa gần đúng (Levenshtein Distance)

KẾT QUẢ TRA CỨU DỮ LIỆU

Từ khóa: "大学生"

01. 学生 (がくせい • gakusei) học sinh, sinh viên [GAP: 1]
02. 大学 (だいがく • daigaku) đại học [GAP: 1]
03. 学生 (がくせい • gakusei) học sinh, sinh viên [GAP: 1]
04. 大学 (だいがく • daigaku) trường đại học [GAP: 1]
05. 大学生 (だいがくせい • daigakusei) sinh viên đại học [EXACT]
06. 小学生 (しょうがくせい • shougakusei) học sinh tiểu học [GAP: 1]
07. 大学院 (だいがくいん • daigakuin) viện đào tạo sau đại học [GAP: 1]
08. 大学院生 (だいがくいんせい • daigakuinsei) sinh viên cao học [GAP: 1]
09. 留学生 (りゅうがくせい • ryuugakusei) du học sinh [GAP: 1]

✓ Tìm thấy 9 kết quả phù hợp.

KẾT QUẢ TRA CỨU DỮ LIỆU

Từ khóa: "びょういん"

KẾT QUẢ TRA CỨU DỮ LIỆU

Từ khóa: "gakuse"

Hình 4.17: Giao diện kết quả tìm kiếm mờ với từ khóa Kanji, Furigana và Romaji

d) Tìm kiếm theo tiền tố (Trie Data Structure)

KẾT QUẢ TRA CỨU DỮ LIỆU

Từ khóa: "gaku"

01. 学部 (がくぶ • gakubu) ngành học
 - どの 学部 ですか。
 - Thuộc khoa nào?
 - 経済の 学部 です。
 - Khoa kinh tế.
02. 学園祭 (がくえんさい • gakuensai) ngày hội trường, lễ hội tổ chức tại trường
 - がっこう の 学園祭。
 - Lễ hội trường học.
 - 学園祭 は おもしろい。
 - Lễ hội trường rất thú vị.
03. 学問 (がくもん • gakumon) học vấn
 - 学問 は たいせつ です。
 - Học vấn là quan trọng.
 - 学問 を べんきょう します。

Hình 4.18: Giao diện kết quả tìm kiếm theo tiền tố

e) Danh sách Kanji

BÀI 01 : DANH SÁCH KANJI			BÀI 02 : DANH SÁCH KANJI			BÀI 32 : DANH SÁCH KANJI		
01	一	nhất	17	日	nhật	497	危	nguy
02	二	nhị	18	月	nguyệt	498	険	hiểm
03	三	tam	19	火	hỏa	499	拾	thập
04	四	tứ	20	水	thủy	500	捨	xả
05	五	ngũ	21	木	mộc	501	戾	lệ
06	六	lục	22	金	kim	502	吸	hấp
07	七	thất	23	土	thổ	503	放	phóng
08	八	bát	24	曜	diệu	504	変	biến
09	九	cửu	25	本	bản/bốn	505	齒	xi
10	十	thập	26	人	nhân	506	髮	phát
11	百	bách	27	今	kim	507	絵	hội
12	千	thiên	28	寺	tự	508	横	hoành
13	万	vạn	29	時	thời	509	当	đương
14	円	viên	30	半	bán	510	伝	truyền
15	口	khẩu	31	刀	đao	511	細	tế
16	目	mục	32	分	phân	512	無	vô

Hình 4.19: Giao diện hiển thị danh sách kanji các bài học

THÔNG TIN CHI TIẾT KANJI		THÔNG TIN CHI TIẾT KANJI	
Kanji: 一 Hán Việt : nhất STT: 1 Bộ thủ : 一 Số nét: 1 Giải nghĩa : nhất, one, one radical [Âm ON] : イチ (ichi), イツ (itsu) [Âm KUN] : ヒト- (hito-), ヒト.つ (hito.tsu) Từ vựng liên quan (9): 01. 一 (いち • ichi) 1, số 1 - これは 一です。 - Đây là số 1. - 一たす 二は 三です。 1 cộng 2 bằng 3. - 一から はじめます。 - Bắt đầu từ 1. 02. 一つ (ひとつ • hitotsu) một cái - かばんが 一つ あります。 - Có một cái cặp. - パンを 一つ ください。 - Cho tôi một cái bánh mì. - おかしを 一つ たべました。 - Tôi đã ăn một cái kẹo. 03. 一時 (いちじ • ichiji) một giờ - いま 一時 です。 - Bây giờ là 1 giờ. - 一時 に あいましょう。 - Chúng ta hãy gặp nhau lúc 1 giờ. - きのう 一時 に ねました。 - Hôm qua tôi đã ngủ lúc 1 giờ. 04. 一分 (いっぶん • ippun) một phút - 一分 まってください。 - Hãy đợi 1 phút. - あと 一分 です。 - Còn 1 phút nữa. - 一分 で おわります。 - Sẽ xong trong 1 phút.		Kanji: 百 Hán Việt : bách STT: 11 Bộ thủ : 一, 白 Số nét: 6 Giải nghĩa : bách, bá, mạch, hundred [Âm ON] : ヒャク (hyaku), ビャク (byaku) [Âm KUN] : もも (momo) Từ vựng liên quan (8): 01. 百 (ひゃく • hyaku) 100 - 百えん あります。 - Có 100 yên. - 百にん います。 - Có 100 người. - 百まい かいました。 - Đã mua 100 tờ/tấm. 02. 二百 (にひゃく • nihyaku) 200 - 二百えん です。 - Là 200 yên. - 二百にん きます。 200 người sẽ đến. - 二百まい あります。 - Có 200 tờ/tấm. 03. 三百 (さんびゃく • sanbyaku) 300 - 三百えん ください。 - Cho tôi 300 yên. - 三百にん います。 - Có 300 người. - 三百まい かいました。 - Đã mua 300 tờ/tấm. 04. 六百 (ろっびゃく • roppyaku) 600 - 六百えん です。 - Là 600 yên. - 六百にん います。 - Có 600 người. - 六百まい あります。 - Có 600 tờ/tấm.	

Hình 4.20: Giao diện hiển thị chi tiết nội dung từng Kanji

f) Lọc từ vựng

LỌC CÁC TỪ VỰNG CHỈ CHỨA CÁC KANJI ĐÃ HỌC

Nhập các bài học cần trích xuất từ vựng (nhập 0 để kết thúc):
 >> 3 5 0

DANH SÁCH TỪ VỰNG THỎA MÃN ĐIỀU KIỆN LỌC

001. 北口 (きたぐち - kitaguchi) cửa bắc
 002. 来月 (らいげつ - raigetsu) tháng sau
 003. 今週 (こんしゅう - konshuu) tuần này
 004. 時間 (じかん - jikan) thời gian
 005. 一時間 (いちじかん - ichijikan) một giờ, một tiếng (thời lượng)
 006. 前半 (ぜんはん - zenhan) hiệp một
 007. 上 (うえ - ue) ở trên, phía trên
 008. 上げる (あげる - ageru) giơ lên, tặng
 009. 上る (のぼる - noboru) leo lên, lên tới
 010. 下 (した - shita) bên dưới, phía dưới
 011. 下げる (さげる - sageru) giảm, hạ xuống
 012. 下さい (ください - kudasai) cho (tôi)
 013. 上下 (じょうげ - jouge) lên xuống, sự dao động
 014. 下ろす (おろす - orosu) bốc xuống, dỡ xuống, rút (tiền)
 015. 中 (なか - naka) ở giữa, bên trong
 016. 中国 (ちゅうごく - chuugoku) trung quốc
 017. 一年中 (いちねんじゅう - ichinenjuu) suốt năm
 018. 外 (そと - soto) ở ngoài, bên ngoài
 019. 外国 (がいこく - gaikoku) nước ngoài
 020. 外国人 (がいこくじん - gaikokujin) người nước ngoài
 021. 外の (ほかの - hokano) ngoài ra, cái khác
 022. 外す (はずす - hazusu) cới ra, tháo rời ra
 023. 右 (みぎ - migi) bên phải
 024. 左右 (さゆう - sayuu) trái phải
 025. 工学 (こうがく - kougaku) công học, kỹ thuật
 026. 左 (ひだり - hidari) bên trái
 027. 左右 (さゆう - sayuu) trái phải
 028. 前 (まえ - mae) trước, phía trước
 029. 午前 (ごぜん - gozen) buổi sáng (a.m)
 030. 午前中 (ごぜんちゅう - gozenchuu) suốt buổi sáng, trong buổi sáng
 031. 三年前 (さんねんまえ - sannenmae) 3 năm trước, cách đây 3 năm
 032. 前半 (ぜんはん - zenhan) hiệp một
 033. 後ろ (うしろ - ushiro) ở đằng sau, phía sau
 034. クラスの後 (クラスのあと - kurasunoato) sau lớp học
 035. 後で (あとで - atode) sau đó
 036. 午後 (ごご - gogo) buổi chiều (p.m)
 037. 後半 (こうはん - kouhan) hiệp sau
 038. 後ほど (のちほど - nochihodo) sau
 039. 後れる (おくれる - okureru) chậm lại đằng sau, tụt hậu
 040. 午前 (ごぜん - gozen) buổi sáng (a.m)
 041. 午後 (ごご - gogo) buổi chiều (p.m)
 042. 午前中 (ごぜんちゅう - gozenchuu) suốt buổi sáng, trong buổi sáng
 043. 門 (もん - mon) cánh cổng
 044. 間 (あいだ - aida) ở giữa
 045. 時間 (じかん - jikan) thời gian
 046. 二時間 (にじかん - nijikan) 2 giờ, 2 tiếng đồng hồ
 047. 一週間 (いっしゅうかん - issshuukan) một tuần
 048. 人間 (にんげん - ningen) con người
 049. 東 (ひがし - higashi) hướng đông, phía đông
 050. 東口 (ひがしぐち - higashiguchi) cửa đông
 051. 中東 (ちゅうとう - chuutou) trung đông
 052. 西 (にし - nishi) hướng tây, phía tây
 053. 西口 (にしぐち - nishiguchi) cửa tây

Hình 4.21: Giao diện kết quả lọc từ vựng các bài thỏa mãn điều kiện

g) Phân tích các Kanji có trong câu

TRA CỨU KANJI TRONG CÂU

Hãy nhập câu cần phân tích các Kanji trong đó:
新聞を読みます。

>> 新聞を読みます。

Kanji : 新 - tân (STT: 85)
Bộ thủ : 親, 斤 Số nét: 13
Giải nghĩa : tân, new

[Âm ON] : シン (shin)
[Âm KUN] : あたら.しい (atara.shii), あら.た (ara.ta), あら- (ara-), にい- (nii-)

Từ vựng liên quan (7):
01. 新しい (あたらしい • atarashii) mới
02. 新聞 (しんぶん • shinbun) báo chí
03. 新幹線 (しんかんせん • shinkansen) tàu cao tốc
04. 新年 (しんねん • shinnen) năm mới
05. 新鮮な (しんせんな • shinsenna) tươi mới
06. 新たな (あらたな • aratana) mới
07. 新潟 (にいがた • niigata) niigata

Kanji : 聞 - văn (STT: 74)
Bộ thủ : 門, 耳 Số nét: 14
Giải nghĩa : văn, văn, văn, hear, ask, listen

[Âm ON] : ブン (bun), モン (mon)
[Âm KUN] : き.く (ki.ku), き.こえる (ki.koeru)

Từ vựng liên quan (5):
01. 聞く (きく • kiku) nghe, hỏi
02. 聞こえる (きこえる • kikoeru) nghe thấy
03. 新聞 (しんぶん • shinbun) báo chí
04. 聞き取る (ききとる • kikitoru) nghe hiểu
05. 前代未聞 (ぜんだいみもん • zendaimimon) việc chưa từng nghe thấy, chưa có tiền lệ

Kanji : 読 - đọc (STT: 119)
Bộ thủ : 言, 売 Số nét: 14
Giải nghĩa : đọc, đọc, read

[Âm ON] : ドク (doku), トク (toku), トウ (tou)
[Âm KUN] : よ.む (yo.mu), -よ.み (-yo.mi)

Từ vựng liên quan (6):
01. 読む (よむ • yomu) đọc
02. 読み物 (よみもの • yomimono) sách (tài liệu) đọc
03. 読書 (どくしょ • dokusho) việc đọc sách
04. 読者 (どくしゃ • dokusha) độc giả, đọc giả
05. 句読点 (くどうてん • kutouten) dấu chấm câu
06. 愛読書 (あいどくしょ • aidokusho) cuốn sách yêu thích

Hình 4.22: Giao diện kết quả tìm thông tin của các Kanji trong câu

4.3.3. Nhận xét đánh giá

Về tổng thể, chương trình được phát triển bám sát ý tưởng ban đầu của đề tài, hướng đến mục tiêu xây dựng một công cụ tra cứu và quản lý Kanji hiệu quả dành riêng cho sinh viên trong chương trình Xseeds, đồng thời đã được mở rộng với nhiều tính năng vượt bậc hỗ trợ quá trình học tập toàn diện. Cụ thể, chương trình đã đáp ứng được các yêu cầu cốt lõi sau:

- Hệ thống tra cứu đa dạng cho phép người học tìm kiếm Kanji và từ vựng một cách nhanh chóng, linh hoạt và hiệu quả. Người dùng có thể lựa chọn bốn phương thức tra cứu phù hợp với nhu cầu: tìm kiếm chính xác, tra cứu khi chỉ nhớ một phần, tra cứu dựa trên tiền tố, hoặc tìm kiếm khi chỉ nhớ mơ hồ. Nhờ đó, việc học và tra cứu trở nên thuận tiện hơn, đáp ứng nhiều tình huống khác nhau trong quá trình tiếp cận ngôn ngữ.
- Quản lý bài học khoa học: Toàn bộ 32 bài học với 512 chữ Kanji và hơn 3.500 từ vựng được tổ chức theo cấu trúc rõ ràng, cho phép người dùng duyệt chi tiết, nhanh chóng và đầy đủ
- Phân tích các Kanji có trong câu tiếng Nhật: Chức năng chiết xuất và tra cứu tức thì các ký tự Kanji xuất hiện trong câu văn giúp người học tra cứu hàng loạt nhanh chóng, ứng dụng kiến thức vào ngữ cảnh thực tế, tăng khả năng đọc hiểu và tiếp cận tài liệu chuyên ngành.
- Lọc từ vựng thông minh theo tiến độ: Tính năng tự động loại bỏ từ vựng chứa Kanji chưa học dựa trên tiến độ hoàn thành bài học, đảm bảo người học luôn tiếp cận đúng trọng tâm, không lãng phí thời gian với những từ vựng vượt quá trình độ hiện tại.

Nhìn chung, chương trình đã đáp ứng và hoàn thiện vượt bậc so với mục tiêu ban đầu. Các chức năng được xây dựng bám sát nhu cầu thực tế của sinh viên tham gia chương trình Xseeds, có tính hệ thống cao, dễ sử dụng. Bên cạnh đó, Hệ thống thể hiện rõ tính ứng dụng của lý thuyết "Cấu trúc dữ liệu và Giải thuật" vào giải quyết bài toán thực tiễn trong giáo dục ngôn ngữ, có tiềm năng phát triển thành một nền tảng học tập Kanji chuyên nghiệp và toàn diện, góp phần đào tạo nguồn nhân lực Công nghệ thông tin chất lượng cao.

5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận

Qua quá trình nghiên cứu, thiết kế và hiện thực hóa đề tài "Xây dựng công cụ hỗ trợ tra cứu Kanji (Hán tự) tích hợp số hóa giáo trình 512 Kanji Look And Learn", nhóm chúng em đã đạt được những kết quả quan trọng, vừa khẳng định giá trị khoa học vừa đáp ứng nhu cầu thực tiễn của sinh viên Công nghệ thông tin - Ngoại ngữ Nhật thuộc chương trình đào tạo Xseeds.

Đề tài đã giúp nhóm củng cố và làm chủ kiến thức nền tảng cũng như tiếp cận nhiều kiến thức nâng cao trong ngôn ngữ lập trình C. Cụ thể, nhóm đã thành công trong việc xây dựng một chương trình dòng lệnh hoàn chỉnh, tích hợp nhiều cấu trúc dữ liệu quan trọng ,đi kèm với hệ thống thuật toán tìm kiếm đa dạng . Đặc biệt, việc xử lý Unicode (UTF-8/UTF-16) thông qua Win32 API (ReadConsoleW, WideCharToMultiByte) [7] và phân tích cú pháp dữ liệu JSON sử dụng thư viện cJSON là một thử thách lớn nhưng đã được nhóm vượt qua thành công, mở ra hướng tiếp cận bài bản cho các ứng dụng đa ngôn ngữ sau này.

Mặc dù đã được đầu tư về mặt thời gian và công sức, do phạm vi và quy mô của đề tài cũng như những hạn chế khách quan về kinh nghiệm thực tiễn, sản phẩm vẫn còn những khiếm khuyết cần được khắc phục trong các phiên bản sau. Chúng em tin tưởng rằng những kiến thức và kỹ năng tích lũy được qua quá trình thực hiện đồ án này sẽ là hành trang quý báu cho sự phát triển chuyên môn trong tương lai. Đồng thời, chúng em hy vọng sản phẩm sẽ góp một phần nhỏ vào nỗ lực ứng dụng công nghệ thông tin vào lĩnh vực giáo dục, giúp việc học tập trở nên hiệu quả và tiếp cận hơn với đông đảo người học.

5.2. Hướng phát triển

Trên cơ sở các kết quả nghiên cứu và thực nghiệm đã đạt được, nhóm thực hiện đề tài xác định các định hướng phát triển tiếp theo nhằm hoàn thiện giải pháp kỹ thuật, tối ưu hóa hiệu năng vận hành và nâng cao trải nghiệm người dùng, cụ thể bao gồm:

- *Nâng cấp kiến trúc nền tảng và tối ưu hóa giao diện (UI/UX)*: Chuyển đổi mô hình ứng dụng dòng lệnh hiện tại sang cấu trúc ứng dụng Web động (Web Application) và phát triển ứng dụng di động (Mobile Application) đa nền tảng (iOS và Android). Hướng đi này giúp người học tiếp cận hệ thống tra cứu và ôn tập mọi lúc, mọi nơi thông qua giao diện trực quan, thân thiện và linh hoạt trên đa dạng kích thước màn hình.

- *Mở rộng quy mô và chuẩn hóa cơ sở dữ liệu:* Phát triển kho tri thức Kanji vượt khỏi phạm vi 512 chữ của giáo trình Kanji Look and Learn hiện tại. Hệ thống sẽ tích hợp toàn diện kho dữ liệu Hán tự theo các cấp độ năng lực Nhật ngữ (từ N5 đến N1) và đồng bộ hóa với hệ sinh thái học liệu từ các giáo trình uy tín khác như Minna no Nihongo, Shin Kanzen Master và Soumatome.
- *Ứng dụng Trí tuệ nhân tạo (AI) và cá nhân hóa lộ trình học tập:* Nghiên cứu tích hợp các mô hình học máy (Machine Learning) nhằm thu thập, phân tích hành vi và tiến độ học tập thực tế của từng người dùng. Từ đó, hệ thống có thể tự động nhận diện các mảng kiến thức còn khiếm khuyết để kiến nghị các thuật toán sinh đề trắc nghiệm và lộ trình ôn tập cá nhân hóa tối ưu.
- *Tích hợp cơ chế tương tác và trò chơi hóa (Gamification):* Xây dựng thêm các phân hệ tương tác cộng đồng, hỗ trợ phản hồi hai chiều và áp dụng các mô hình trò chơi hóa học tập (như bảng xếp hạng, thử thách hàng ngày, hệ thống huy hiệu) nhằm gia tăng động lực thúc đẩy và sự hứng thú bền vững cho người học.
- *Chuyển giao công nghệ và phát triển cộng đồng:* Tiến hành mở mã nguồn (Open-source) của dự án để cộng đồng lập trình viên cùng khai thác, đóng góp và hoàn thiện hệ sinh thái. Đồng thời, nhóm hướng tới việc liên kết và đề xuất tích hợp sản phẩm như một công cụ hỗ trợ chính thức trong chương trình giảng dạy chiến lược Xseeds của công ty Sun Asterisk, nhằm tối đa hóa giá trị ứng dụng thực tiễn trong môi trường giáo dục.

TÀI LIỆU THAM KHẢO

- [1] E. Banno, Y. Ikeda, Y. Ohno, C. Shinagawa and K. Tokashiki, Genki I: An integrated course in elementary Japanese, Tokyo: The Japan Times, 2011.
- [2] S. N. (. S. A. Inc.), "Sun* hợp tác với nhiều trường Đại học lớn tại Châu Á và Nam Mỹ, mở rộng dự án giáo dục toàn cầu mang tên Xseeds," 2022. [Online]. Available: <https://news.sun-asterisk.com/vi/p/sun-hop-tac-voi-nhieu-truong-dai-hoc-lon-tai-chau-a-va-nam-my-mo-rong-du-an-giao-duc-toan-cau-mang-ten-xseeds-XDl8kzedADg>.
- [3] S. A. Inc., "Xseeds Hub - Nền tảng kết nối nhân tài CNTT toàn cầu," 2023. [Online]. Available: <https://sun-asterisk.com/service/xseeds-hub/>.
- [4] T. J. Times, 512 Kanji Look and Learn, Tokyo: The Japan Times Publishing, 2009.
- [5] M. Corporation, "WideCharToMultiByte function (stringapiset.h)," [Online]. Available: <https://learn.microsoft.com/en-us/windows/win32/api/stringapiset/nf-stringapiset-widechartomultibyte>.
- [6] M. Corporation, "ReadConsoleW function (consoleapi2.h)," [Online]. Available: <https://learn.microsoft.com/en-us/windows/console/readconsolew>.
- [7] M. Corporation, "Win32 API programming reference," [Online]. Available: <https://learn.microsoft.com/th-th/windows/win32/api/>.
- [8] E. International, "The JSON data interchange syntax (ECMA-404, 2nd ed.)," 2017. [Online]. Available: <https://www.ecma-international.org/publications-and-standards/standards/ecma-404/>.

- [9] JSON.org, "Introducing JSON," [Online]. Available: <https://www.json.org/>.
- [10] D. Gamble, "cJSON: Ultralightweight JSON parser in ANSI C," [Online]. Available: <https://github.com/DaveGamble/cJSON>.
- [11] T. U. Consortium, "The Unicode Standard, Version 15.1.0," 2023. [Online]. Available: <https://home.unicode.org/>.
- [12] R. Gillam, Unicode demystified: A practical programmer's guide to the encoding standard, Boston: Addison-Wesley, 2003.
- [13] T. U. Consortium, "CJK Unified Ideographs Extension A (U+3400–U+4DBF)," [Online]. Available: <https://unicode.org/charts/PDF/U3400.pdf>.
- [14] T. U. Consortium, "CJK Unified Ideographs (U+4E00–U+9FFF)," [Online]. Available: <https://unicode.org/charts/PDF/U4E00.pdf>.
- [15] E. Fredkin, "Trie Memory," *Communications of the ACM*, vol. 3, no. 9, pp. 490-499, 1962.
- [16] D. J. (J. B. Bernstein, "djb2 hash function," 1991. [Online]. Available: <http://www.cse.yorku.ca/~oz/hash.html>.
- [17] V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions, and reversals," *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707-710, 1966.
- [18] D. E. Knuth, J. H. Morris and V. R. Pratt, "Fast pattern matching in strings," *SIAM Journal on Computing*, vol. 6, no. 323-350, p. 2, 1977.
- [19] E. International, "Control functions for coded character sets (ECMA-48, 5th ed.)," 1991. [Online]. Available: <https://www.ecma-international.org/publications-and-standards/standards/ecma-48/>.

- [20] W. contributors, "ANSI escape code," 2024. [Online].
Available: https://en.wikipedia.org/wiki/ANSI_escape_code.

PHỤ LỤC

Directory structure:

```
└─ PBL_1/
   ├── build.bat
   ├── data_structures.h
   ├── main.c
   ├── data/kanjiData.json
   ├── lib/
   │   └─ cJSON.h/.c
   └─ src/
       ├── dashboard.c
       ├── dashboard.h
       ├── filter_learned_lesson.c
       ├── filter_learned_lesson.h
       ├── fuzzy_search.c
       ├── fuzzy_search.h
       ├── kanji_search_exact.c
       ├── kanji_search_exact.h
       ├── lessons_management.c
       ├── lessons_management.h
       ├── parse_json_to_struct.c
       ├── parse_json_to_struct.h
       ├── prefix_search.c
       ├── prefix_search.h
       ├── sentence_analysis.c
       ├── sentence_analysis.h
       ├── substring_search.c
       ├── substring_search.h
       ├── utils.c
       ├── utils.h
       ├── vocab_search_exact.c
       └── vocab_search_exact.h
```

FILE: build.bat

```
@echo off
echo Compiling project...
gcc main.c ^
    src/parse_json_to_struct.c ^
    src/vocab_search_exact.c ^
    src/kanji_search_exact.c ^
    src/fuzzy_search.c ^
    src/substring_search.c ^
    src/lessons_management.c ^
    src/filter_learned_lesson.c ^
    src/sentence_analysis.c ^
```

```

src/prefix_search.c ^
src/utils.c ^
src/dashboard.c ^
lib/cJSON.c ^
-Ilib -Isrc -o main.exe
if %ERRORLEVEL% neq 0 (
    echo Compilation failed!
    exit /b %ERRORLEVEL%
)
echo Build successful. Run with: .\main.exe

```

```

=====
FILE: data_structures.h
=====

```

```

#ifndef DATA_STRUCTURES_H
#define DATA_STRUCTURES_H
#include <wchar.h>
typedef struct SampleInfo Sample;
typedef struct VocabInfo Vocab;
typedef struct YomiInfo Yomi;
typedef struct KanjiInfo Kanji;
typedef struct KanjiListInfo KanjiList;
struct SampleInfo {
    char *jp;
    char *vn;
};
struct VocabInfo {
    char *vocab;
    wchar_t *vocab_w;
    char *hiragana;
    wchar_t *hiragana_w;
    char *romaji;
    wchar_t *romaji_w;
    char *meaning;
    wchar_t *meaning_w;
    Sample *samples;
    int samplesCount;
};
struct YomiInfo {
    char *jp;
    char *romaji;
};
struct KanjiInfo {
    int stt;
    char *kanji;
    wchar_t *kanji_w;
    char *hanViet;
    char *radical;
    char *stroke;
    char *description;
    Yomi *on;

```

```

    int onCount;
    Yomi *kun;
    int kunCount;
    Vocab *vocabs;
    int vocabsCount;
};
struct KanjiListInfo {
    Kanji *kanjis;
    int kanjiCount;
};
#endif

```

FILE: main.c

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#ifdef _WIN32
    #include <windows.h>
#endif
#include "parse_json_to_struct.h"
#include "data_structures.h"
#include "dashboard.h"
int main() {
    configUTF8();
    const char *fileAddress = "data/kanjiData.json";
    char *rawJson = readFileToString(fileAddress);
    if (!rawJson) {
        printf("Không tìm thấy file: %s\n", fileAddress);
        return 1;
    }
    handleMenuSelection(rawJson);
    free(rawJson);
    return 0;
}

```

FILE: src/dashboard.c

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#ifdef _WIN32
    #include <windows.h>
#endif
#include "kanji_search_exact.h"
#include "vocab_search_exact.h"
#include "fuzzy_search.h"
#include "substring_search.h"
#include "prefix_search.h"
#include "lessons_management.h"

```

```

#include "filter_learned_lesson.h"
#include "sentence_analysis.h"
#include "parse_json_to_struct.h"
#include "utils.h"
#include "../data_structures.h"
#include "dashboard.h"
void displayMenu(void) {
    printf("\n");
    printf(" " BG_HIGHLIGHT BOLD " TỪ ĐIỂN 512 KANJI \n" RESET);
    printf(" " CL_BORDER
"
" RESET "\n");
    printf(" " CL_PRIMARY " | " CL_DIM "%-6s" CL_HEADER BOLD "%-35s" RESET
"\n", "STT", "CÁC CHỨC NĂNG");
    printf(" " CL_PRIMARY " | " CL_BORDER
"
" RESET "\n");
    printf(" " CL_PRIMARY " | " CL_KEY BOLD "01" RESET " | 🔍 " CL_TEXT
"%-35s" RESET "\n",
        "Tìm kiếm, tra cứu từ vựng nâng cao");
    printf(" " CL_PRIMARY " | " CL_KEY BOLD "02" RESET " | 📁 " CL_TEXT
"%-35s" RESET "\n",
        "Quản lý danh mục từ điển");
    printf(" " CL_PRIMARY " | " CL_KEY BOLD "03" RESET " | 📄 " CL_TEXT
"%-35s" RESET "\n",
        "Lọc từ vựng thông minh");
    printf(" " CL_PRIMARY " | " CL_KEY BOLD "04" RESET " | 🧠 " CL_TEXT
"%-35s" RESET "\n",
        "Phân tích Hán tự trong câu");
    printf(" " CL_PRIMARY " | " CL_BORDER
"
" RESET "\n");
    printf(" " CL_ERROR " | " CL_ERROR BOLD "00" RESET " | ✖ " CL_ERROR
BOLD "%-35s" RESET "\n",
        "THOÁT CHƯƠNG TRÌNH");
    printf(" " CL_BORDER
"
" RESET "\n");
    printf(" " CL_DIM "Hướng dẫn:" RESET " Chọn chức năng số [" CL_PRIMARY "1-4"
RESET "]" CL_DIM " | " RESET " [" CL_ERROR "0" RESET "]" để thoát\n");
    printf(" " BOLD "Lựa chọn của bạn" RESET " " CL_PRIMARY ">>" RESET);
}
void menuExactSearch(HashTable *vocabHT, HashTableK *kanjiHT) {
    int choice;
    char keyword[256];
    while (1) {
        clearScreen();
        printf("\n");

```

```

printf(" " BG_HIGHLIGHT BOLD " Tra cứu dữ liệu chính xác (HashTable) "
RESET);
printf("\n\n");
printf(" " CL_PRIMARY " | " CL_DIM "%-6s" CL_HEADER BOLD "%-35s"
RESET "\n", "STT", "PHẠM VI DỮ LIỆU TRA CỨU");
printf(" " CL_PRIMARY " | " CL_BORDER
"_____") RESET "\n");
printf(" " CL_PRIMARY " | " CL_KEY BOLD "01" RESET " | " CL_TEXT "%-
35s" RESET "\n",
    "Tra cứu Kanji (Kí tự / Hán Việt)");
printf(" " CL_PRIMARY " | " CL_KEY BOLD "02" RESET " | " CL_TEXT "%-
35s" RESET "\n",
    "Tra cứu Từ vựng (Kanji / Furigana / Romaji / Tiếng Việt)");
printf(" " CL_PRIMARY " | " CL_BORDER
"_____") RESET "\n");
printf(" " CL_PRIMARY " | " CL_DIM BOLD "00" RESET " | " CL_DIM "%-
35s" RESET "\n",
    "Quay lại Menu tìm kiếm");
printf(" " CL_PRIMARY " | " RESET "\n");
printf(" " CL_BORDER
"_____") RESET "\n");
printf(" " CL_DIM "Hướng dẫn:" RESET " Chọn số [" CL_PRIMARY "1-2" RESET
"] để xác định phạm vi " CL_DIM " | " RESET " Chọn [" CL_ERROR "0" RESET "] để
quay lại\n");
printf(" " BOLD "Lựa chọn của bạn" RESET " " CL_PRIMARY ">>" RESET
CL_KEY);
if (scanf("%d", &choice) != 1) {
    while(getchar() != '\n');
    continue;
}
getchar();
printf(RESET);
if (choice == 0) break;
if (choice != 1 && choice != 2) {
    printf("\n " CL_ERROR " 🚫 [LỖI]: Phạm vi chọn không hợp lệ. Vui lòng thử lại!"
RESET "\n");
printf("\n " CL_BORDER
"_____") RESET "\n");
printf(" " CL_DIM "➡️ Nhấn " CL_PRIMARY "Enter" CL_DIM " để tiếp tục..."
RESET);
getchar();
continue;
}
printf("\n");

```

```

printf(" " CL_BORDER ">>> Nhập từ khóa: " RESET CL_LOGO BOLD);
inputString(keyword, 256);
printf(RESET);
clearScreen();
printf("\n");
printf(" " BG_HIGHLIGHT BOLD " KẾT QUẢ TRA CỨU DỮ LIỆU " RESET);
printf(" " CL_DIM "Từ khóa:" RESET " \'" CL_LOGO BOLD "%s" RESET "\n",
keyword);
printf(" " CL_BORDER
"
" RESET "\n");
if (choice == 1) {
    exactlySearchingKanji(kanjiHT, keyword);
} else {
    exactlySearchingVocab(vocabHT, keyword);
}
printf("\n " CL_BORDER
"
" RESET "\n");
printf(" " CL_DIM "→ Nhân " CL_PRIMARY "Enter" CL_DIM " để tiếp tục ..."
RESET);
getchar();
}
}
void menuKMPSearch(KanjiList *L) {
    int option;
    char keyword[256];
    while (1) {
        clearScreen();
        printf("\n");
        printf(" " BG_HIGHLIGHT BOLD " Khớp mẫu phân đoạn chuỗi con (KMP
Algorithm) " RESET);
        printf("\n\n");
        printf(" " CL_PRIMARY " | " CL_DIM "%-6s" CL_HEADER BOLD "%-35s"
RESET "\n", "STT", "CHẾ ĐỘ KHỚP MẪU");
        printf(" " CL_PRIMARY " | " CL_BORDER
"
" RESET "\n");
        printf(" " CL_PRIMARY " | " CL_KEY BOLD "01" RESET " | " CL_TEXT "%-
35s" RESET "\n",
            "Khớp theo mặt chữ Kanji");
        printf(" " CL_PRIMARY " | " CL_KEY BOLD "02" RESET " | " CL_TEXT "%-
35s" RESET "\n",
            "Khớp theo cách đọc Furigana");
        printf(" " CL_PRIMARY " | " CL_KEY BOLD "03" RESET " | " CL_TEXT "%-
35s" RESET "\n",
            "Khớp theo phiên âm Romaji");
        printf(" " CL_PRIMARY " | " CL_KEY BOLD "04" RESET " | " CL_TEXT "%-
35s" RESET "\n",

```

```

        "Khớp theo nghĩa Tiếng Việt");
    printf("          " CL_PRIMARY " | " CL_BORDER
    "
    " RESET "\n");
    printf(" " CL_PRIMARY " | " CL_DIM BOLD "00" RESET " | " CL_DIM "%-
35s" RESET "\n",
        "Quay lại Menu tìm kiếm");
    printf(" " CL_PRIMARY " | " RESET "\n");
    printf("          " CL_BORDER
    "
    " RESET "\n");
    printf(" " CL_DIM "Hướng dẫn:" RESET " Chọn số [" CL_PRIMARY "1-4" RESET
"] để xác định chế độ " CL_DIM " | " RESET " Chọn [" CL_ERROR "0" RESET "] để quay
lại\n");
    printf(" " BOLD "Lựa chọn của bạn" RESET " " CL_PRIMARY ">> " RESET
CL_KEY);
    if (scanf("%d", &option) != 1) {
        while(getchar() != '\n');
        continue;
    }
    getchar();
    printf(RESET);
    if (option == 0) break;
    if (option < 1 || option > 4) {
        printf("\n " CL_ERROR " 🚫 [LỖI]: Chế độ chọn không hợp lệ. Vui lòng thử lại!"
RESET "\n");
        printf("\n          " CL_BORDER
    "
    " RESET "\n");
    printf(" " CL_DIM "➡️ Nhân " CL_PRIMARY "Enter" CL_DIM " để tiếp tục..."
RESET);
    getchar();
    continue;
    }
    printf("\n");
    printf(" " CL_BORDER ">>> Nhập từ khóa: " RESET CL_LOGO BOLD);
    inputString(keyword, 256);
    printf(RESET);
    clearScreen();
    printf("\n");
    printf(" " BG_HIGHLIGHT BOLD " KẾT QUẢ TRA CỨU DỮ LIỆU " RESET);
    printf(" " CL_DIM "Từ khóa:" RESET " \'" CL_LOGO BOLD "%s" RESET "\"\n",
keyword);
    printf("          " CL_BORDER
    "
    " RESET "\n");
    substringSearching(L, keyword, option);

```

```

    printf("\n\n");
}

void menuFuzzySearch(KanjiList *L) {
    int option;
    char keyword[256];
    while (1) {
        clearScreen();
        printf("\n");
        printf(" " BG_HIGHLIGHT BOLD " Tìm kiếm từ khóa gần đúng (Levenshtein Distance) " RESET);
        printf("\n\n");
        printf(" " CL_PRIMARY " | " CL_DIM "%-6s" CL_HEADER BOLD "%-35s" RESET "\n", "STT", "DẠNG TỪ KHÓA");
        printf(" " CL_PRIMARY " | " CL_BORDER "\n\n");
        printf(" " CL_PRIMARY " | " CL_KEY BOLD "01" RESET " | " CL_TEXT "%-35s" RESET "\n", "Khớp gần đúng mặt chữ Kanji");
        printf(" " CL_PRIMARY " | " CL_KEY BOLD "02" RESET " | " CL_TEXT "%-35s" RESET "\n", "Khớp gần đúng cách đọc Furigana");
        printf(" " CL_PRIMARY " | " CL_KEY BOLD "03" RESET " | " CL_TEXT "%-35s" RESET "\n", "Khớp gần đúng phiên âm Romaji");
        printf(" " CL_PRIMARY " | " CL_BORDER "\n\n");
        printf(" " CL_PRIMARY " | " CL_DIM BOLD "00" RESET " | " CL_DIM "%-35s" RESET "\n", "Quay lại Menu tìm kiếm");
        printf(" " CL_PRIMARY " | " RESET "\n");
        printf(" " CL_BORDER "\n\n");
        printf(" " CL_DIM "Hướng dẫn:" RESET " Chọn số [" CL_PRIMARY "1-3" RESET "]" để xác định chế độ " CL_DIM " | " RESET " Chọn [" CL_ERROR "0" RESET "]" để quay lại\n");
        printf(" " BOLD "Lựa chọn của bạn" RESET " " CL_PRIMARY "» " RESET CL_KEY);
        if (scanf("%d", &option) != 1) {
            while(getchar() != '\n');

```



```

        continue;
    }
    getchar();
    printf(RESET);
    if (option == 0) break;
    if (option < 1 || option > 3) {
        printf("\n " CL_ERROR " 🚫 [LỖI]: Chế độ chọn không hợp lệ. Vui lòng thử lại!"
RESET "\n");
        printf("\n
"
CL_BORDER
"_____ " RESET "\n");
        printf(" " CL_DIM "➡️ Nhấn " CL_PRIMARY "Enter" CL_DIM " để tiếp tục..."
RESET);
        getchar();
        continue;
    }
    printf("\n");
    printf(" " CL_BORDER ">>> Nhập từ khóa: " RESET CL_LOGO BOLD);
    inputString(keyword, 256);
    printf(RESET);
    clearScreen();
    printf("\n");
    printf(" " BG_HIGHLIGHT BOLD " KẾT QUẢ TRA CỨU DỮ LIỆU " RESET);
    printf(" " CL_DIM "Từ khóa:" RESET " \'" CL_LOGO BOLD "%s" RESET "\n",
keyword);
    printf("
"
CL_BORDER
"_____ " RESET "\n\n");
    fuzzySearching(L, keyword, option);
    printf("\n
"
CL_BORDER
"_____ " RESET "\n");
    printf(" " CL_DIM "➡️ Nhấn " CL_PRIMARY "Enter" CL_DIM " để thực hiện lượt
tìm kiếm mới..." RESET);
    getchar();
    }
}
void menuPrefixSearch(TrieNode *trieRoot) {
    char keyword[256];
    int choice;
    while (1) {
        clearScreen();
        printf("\n");
        printf(" " BG_HIGHLIGHT BOLD " Tìm theo tiền tố (Trie Data Structure) " RESET);
        printf("\n\n");
        printf(" " CL_PRIMARY " | " CL_DIM "%-6s" CL_HEADER BOLD "%-35s"
RESET "\n", "STT", "ĐỊNH DẠNG CẦN TÌM");

```

```

printf("          CL_PRIMARY          |          CL_BORDER
"
-----" RESET "\n");
printf("  CL_PRIMARY " | " CL_KEY BOLD "01" RESET " | " CL_TEXT "%-
35s" RESET "\n",
      "Tìm theo Romaji (chữ cái Alphabet)");
printf("          CL_PRIMARY          |          CL_BORDER
"
-----" RESET "\n");
printf("  CL_PRIMARY " | " CL_DIM BOLD "00" RESET " | " CL_DIM "%-
35s" RESET "\n",
      "Quay lại Menu tìm kiếm");
printf("  CL_PRIMARY " | " RESET "\n");
printf("          CL_BORDER
"
-----" RESET "\n");
printf("  CL_DIM "Hướng dẫn:" RESET " Chọn số [" CL_PRIMARY "1" RESET "]
để tra cứu " CL_DIM " | " RESET " [" CL_ERROR "0" RESET "] để quay lại\n");
printf("  BOLD "Lựa chọn của bạn" RESET "  CL_PRIMARY ">> " RESET
CL_KEY);
if (scanf("%d", &choice) != 1) {
    while(getchar() != '\n');
    continue;
}
getchar();
printf(RESET);
if (choice == 0) break;
if (choice != 1) {
    printf("\n  CL_ERROR " 🚫 [LỖI]: Lựa chọn không hợp lệ. Vui lòng thử lại!"
RESET "\n");
    printf("\n          CL_BORDER
"
-----" RESET "\n");
printf("  CL_DIM "➡️ Nhấn " CL_PRIMARY "Enter" CL_DIM " để tiếp tục..."
RESET);
getchar();
continue;
}
printf("\n");
printf("  CL_BORDER ">>> Nhập từ khóa: " RESET CL_LOGO BOLD);
inputString(keyword, 256);
printf(RESET);
clearScreen();
printf("\n");
printf("  BG_HIGHLIGHT BOLD " KẾT QUẢ TRA CỨU DỮ LIỆU " RESET);
printf("  CL_DIM "Từ khóa:" RESET " \'" CL_LOGO BOLD "%s" RESET "\n",
keyword);

```

```

printf("                                " CL_BORDER
"_____ " RESET "\n");
TrieNode *matchNode = searchPrefixNode(trieRoot, keyword);
if (!matchNode) {
    printf(" " CL_ERROR " 🚫 [THÔNG BÁO]: " RESET " Không tồn tại từ vựng nào
bắt đầu bằng từ khóa: \"" CL_LOGO_BOLD "%s" RESET "\"\n", keyword);
} else {
    int resultCounter = 0;
    printAllWordsFromNode(matchNode, &resultCounter);
    if (resultCounter == 0) {
        printf(" " CL_ERROR " 🚫 [THÔNG BÁO]: " RESET " Không có kết quả nào
phù hợp hoàn toàn với chuỗi cung cấp.\n");
    } else {
        printf("\n                                " CL_BORDER
"_____ " RESET "\n");
        printf(" " BG_HIGHLIGHT_BOLD " TRUY XUẤT HOÀN TẤT " RESET "
Tìm thấy tổng cộng " CL_KEY_BOLD "%d" RESET " kết quả phù hợp.\n", resultCounter);
    }
}
printf("\n                                " CL_BORDER
"_____ " RESET "\n");

printf(" " CL_DIM " ➡️ Nhấn " CL_PRIMARY "Enter" CL_DIM " để thực hiện lượt
tìm kiếm mới..." RESET);
getchar();
}
}
void caseNo1(KanjiList *L, HashTable *vHT, HashTableK *kHT, TrieNode *trieRoot) {
    int choice;
    while(1) {
        clearScreen();
        printf("\n");
        printf(" " BG_HIGHLIGHT_BOLD " HỆ THỐNG TÌM KIẾM NÂNG CAO "
RESET);
        printf(" " CL_DIM " | 04 phương thức tra cứu" RESET "\n");
        printf("                                " CL_BORDER
"_____ " RESET "\n\n");
        printf(" " CL_PRIMARY " | " CL_DIM "%-6s" CL_HEADER_BOLD "%-35s"
RESET "\n", "STT", " ⚡ PHƯƠNG THỨC TRA CỨU");
        printf("                                " CL_BORDER
"_____ " RESET "\n");
        printf(" " CL_PRIMARY " | " CL_KEY_BOLD "01" RESET " | 🔍 " CL_TEXT
"%-35s" RESET "\n",
        "Tra cứu dữ liệu chính xác (HashTable)");

```

```

printf(" " CL_PRIMARY " | " CL_KEY BOLD "02" RESET " | 🔍 " CL_TEXT
"%-35s" RESET "\n",
    "Khớp mẫu phân đoạn chuỗi con (KMP Algorithm)");
printf(" " CL_PRIMARY " | " CL_KEY BOLD "03" RESET " | 🔍 " CL_TEXT
"%-35s" RESET "\n",
    "Tìm kiếm từ khóa gần đúng (Levenshtein Distance)");
printf(" " CL_PRIMARY " | " CL_KEY BOLD "04" RESET " | 🔍 " CL_TEXT
"%-35s" RESET "\n",
    "Tìm theo tiền tố (Trie Data Structure)");
printf("          CL_PRIMARY          |          CL_BORDER
"
-----" RESET "\n");
printf(" " CL_PRIMARY " | " CL_DIM BOLD "00" RESET " | 🔄 " CL_DIM
"%-35s" RESET "\n",
    "Quay lại Menu chính");
printf(" " CL_PRIMARY " | " RESET "\n");
printf("          CL_BORDER
"
-----" RESET "\n");
printf(" " CL_DIM "Hướng dẫn:" RESET " Chọn số [" CL_PRIMARY "1-4" RESET
"] để kích hoạt bộ lọc " CL_DIM " | " RESET " Chọn [" CL_ERROR "0" RESET "]" để về
trang chủ\n");
printf(" " BOLD "Lựa chọn của bạn" RESET " " CL_PRIMARY ">> " RESET
CL_KEY);
if (scanf("%d", &choice) != 1) {
    while(getchar() != '\n');
    continue;
}
getchar();
printf(RESET);
if (choice == 0) break;
switch (choice) {
    case 1: menuExactSearch(vHT, kHT); break;
    case 2: menuKMPSearch(L); break;
    case 3: menuFuzzySearch(L); break;
    case 4: menuPrefixSearch(trieRoot); break;
    default:
        printf("\n " CL_ERROR " 🚫 [LỖI]: Lựa chọn không hợp lệ. Vui lòng thử lại!"
RESET "\n");
        break;
}
printf("\n          CL_BORDER
"
-----" RESET "\n");
printf(" " CL_DIM "➡️ Nhấn " CL_PRIMARY "Enter" CL_DIM " để tiếp tục hệ thống
tra cứu..." RESET);
getchar();
}

```

```

}
void caseNo2(KanjiList *L) {
    clearScreen();
    displayAllLessons(L);
    selectAndDisplayLesson(L);
}
void caseNo3(KanjiList *L) {
    runFilterLearnedVocab(L);
}
void caseNo4(KanjiList *L) {
    analyzeJapaneseSentence(L);
}
void handleMenuSelection(char *rawJson) {
    int choice;
    KanjiList myData = parseJsonToStruct(rawJson);
    HashTable *vocabHT = createHashTable(HASH_TABLE_SIZE);
    HashTableK *kanjiHT = createHashTableK(HASH_TABLE_SIZE);
    TrieNode *trieRoot = buildTrieFromKanjiList(&myData);
    buildHashTableForVocab(&myData, vocabHT);
    buildHashTableForKanji(&myData, kanjiHT);
    while (1) {
        clearScreen();
        displayMenu();
        if (scanf("%d", &choice) != 1) {
            printf("Vui long nhap so hop le.\n");
            while (getchar() != '\n');
            continue;
        }
        getchar();
        switch (choice) {
            case 1: caseNo1(&myData, vocabHT, kanjiHT, trieRoot); break;
            case 2: caseNo2(&myData); break;
            case 3: caseNo3(&myData); break;
            case 4: caseNo4(&myData); break;
            case 0:
                printf("Dang thoat chuong trinh...\n");
                freeTrie(trieRoot);
                return;
            default:
                printf("Lua chon khong hop le. Vui long chon lai.\n");
                getchar();
        }
    }
}

```

FILE: src/dashboard.h

```

#ifndef DASHBOARD_H
#define DASHBOARD_H
void handleMenuSelection(char *rawJson);

```

#endif

```
=====
FILE: src/filter_learned_lesson.c
=====
```

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <wchar.h>
#ifdef _WIN32
    #include <windows.h>
#endif
#include "../data_structures.h"
#include "filter_learned_lesson.h"
#include "utils.h"
#define KANJI_PER_LESSON 16
#define HASH_SIZE 256
typedef struct HashNode {
    wchar_t* kanji;
    struct HashNode* next;
} HashNode;
typedef struct {
    HashNode* buckets[HASH_SIZE];
} LearnedHashSet;
unsigned int hashKanji(const wchar_t* str) {
    unsigned int hash = 5381;
    while (*str) {
        hash = ((hash << 5) + hash) + *str++;
    }
    return hash % HASH_SIZE;
}
LearnedHashSet* createLearnedHashSet() {
    return (LearnedHashSet*)calloc(1, sizeof(LearnedHashSet));
}
void addLearnedKanji(LearnedHashSet* set, wchar_t* kanji) {
    if (!set || !kanji) return;
    unsigned int idx = hashKanji(kanji);
    HashNode* node = (HashNode*)malloc(sizeof(HashNode));
    node->kanji = kanji;
    node->next = set->buckets[idx];
    set->buckets[idx] = node;
}
int containsLearnedKanji(LearnedHashSet* set, const wchar_t* kanji) {
    if (!set || !kanji) return 0;
    unsigned int idx = hashKanji(kanji);
    HashNode* curr = set->buckets[idx];
    while (curr) {
        if (wcscmp(curr->kanji, kanji) == 0)
            return 1;
        curr = curr->next;
    }
}
```

```

    return 0;
}
void freeLearnedHashSet(LearnedHashSet* set) {
    if (!set) return;
    for (int i = 0; i < HASH_SIZE; i++) {
        HashNode* curr = set->buckets[i];
        while (curr) {
            HashNode* temp = curr;
            curr = curr->next;
            free(temp);
        }
    }
    free(set);
}
void runFilterLearnedVocab(KanjiList *allData) {
    if (!allData || allData->kanjiCount == 0) {
        printf(" " CL_ERROR " 🚫 [LỖI] Không tìm thấy dữ liệu cấu trúc trong phân vùng.\n"
RESET);
        return;
    }
    clearScreen();
    printf("\n " BG_HIGHLIGHT BOLD " LỌC CÁC TỪ VỰNG CHỈ CHỨA CÁC KANJI
ĐÃ HỌC " RESET"\n");
    printf(CL_BORDER
"
-----\n" RESET);
    printf(" " CL_TEXT "Nhập các bài học cần trích xuất từ vựng (nhập 0 để kết thúc):\n"
RESET);
    printf(" >> " CL_LOGO BOLD);
    int selected[50] = {0};
    int numSelected = 0, input, maxLesson = 0;
    while (scanf("%d", &input) == 1 && input != 0 && numSelected < 50) {
        selected[numSelected++] = input;
        if (input > maxLesson) maxLesson = input;
    }
    printf(RESET);
    if (numSelected == 0) {
        printf("\n " CL_WARN " ⚠ Không có phân đoạn bài học nào được ghi nhận.\n"
RESET);
        printf(" " CL_DIM "Nhấn Enter để tiếp tục..." RESET);
        getchar(); getchar();
        return;
    }
    wchar_t** targetKanjiList = malloc((maxLesson * KANJI_PER_LESSON + 10) *
sizeof(wchar_t*));
    LearnedHashSet* learnedHash = createLearnedHashSet();
    if (!targetKanjiList || !learnedHash) {
        printf(" " CL_ERROR " 🚫 [LỖI] Phân bổ bộ nhớ thất bại (Heap Allocation Error).\n"
RESET);

```

```

    free(targetKanjiList);
    freeLearnedHashSet(learnedHash);
    return;
}
int targetCount = 0;
for (int lesson = 1; lesson <= maxLesson; lesson++) {
    int start = (lesson - 1) * KANJI_PER_LESSON;
    int end = start + KANJI_PER_LESSON;
    if (end > allData->kanjiCount) end = allData->kanjiCount;
    for (int j = start; j < end; j++) {
        wchar_t* kanjiW = allData->kanjis[j].kanji_w;
        if (!kanjiW) continue;
        addLearnedKanji(learnedHash, kanjiW);
        for (int k = 0; k < numSelected; k++) {
            if (selected[k] == lesson) {
                targetKanjiList[targetCount++] = kanjiW;
                break;
            }
        }
    }
}
int foundCount = 0;
int maxToCheck = (maxLesson * KANJI_PER_LESSON < allData->kanjiCount)
    ? maxLesson * KANJI_PER_LESSON : allData->kanjiCount;
printf("\n  " BOLD "DANH SÁCH TỪ VỰNG THỎA MÃN ĐIỀU KIỆN LỌC\n"
RESET);
printf(CL_BORDER
"_____
_____\\n" RESET);
for (int i = 0; i < maxToCheck; i++) {
    Kanji *kanjiEntry = &allData->kanjis[i];
    for (int v = 0; v < kanjiEntry->vocabsCount; v++) {
        Vocab *vocab = &kanjiEntry->vocabs[v];
        if (!vocab->vocab || !vocab->vocab_w) continue;
        const wchar_t* text = vocab->vocab_w;
        int hasTargetKanji = 0;
        int allKanjiKnown = 1;
        for (int pos = 0; text[pos] != L'\0'; pos++) {
            wchar_t ch = text[pos];
            if (isKanjiWChar(ch)) {
                wchar_t singleKanji[2] = {ch, L'\0'};
                if (!hasTargetKanji) {
                    for (int t = 0; t < targetCount; t++) {
                        if (wcscmp(singleKanji, targetKanjiList[t]) == 0) {
                            hasTargetKanji = 1;
                            break;
                        }
                    }
                }
            }
        }
        if (!containsLearnedKanji(learnedHash, singleKanji)) {

```



```

        allKanjiKnown = 0;
        break;
    }
}
}
if (hasTargetKanji && allKanjiKnown) {
    foundCount++;
    printf(" " CL_PRIMARY " | " CL_DIM "%0.3d." RESET " " CL_KANJI_BOLD
"%s" RESET " (" CL_KANA "%s" RESET " - " CL_ROMAJI "%s" RESET ") "
CL_MEANING "%s\n",
        foundCount,
        vocab->vocab,
        vocab->hiragana ? vocab->hiragana : "-",
        vocab->romaji ? vocab->romaji : "-",
        vocab->meaning ? vocab->meaning : "-");
}
}
}
printf(CL_BORDER
"
=====
\n" RESET);
if (foundCount == 0) {
    printf(" " CL_WARN "⚠ Hệ thống không tìm thấy từ vựng nào giao thoa giữa các bài
đã chọn.\n" RESET);
} else {
    printf(" " CL_SUCCESS "✓ Tìm thấy tổng cộng %d từ vựng hợp lệ.\n" RESET,
foundCount);
}
freeLearnedHashSet(learnedHash);
free(targetKanjiList);
getchar();
printf(" " CL_DIM "➡ Nhấn Enter để quay lại..." RESET);
getchar();
}

```

FILE: src/filter_learned_lesson.h

```

#ifndef FILTER_LEARNED_LESSON_H
#define FILTER_LEARNED_LESSON_H
#include "../data_structures.h"
typedef struct LearnedVocab {
    char *vocab;
    char *furigana;
    char *romaji;
    char *meaning;
    struct LearnedVocab *next;
} LearnedVocab;
typedef struct LearnedVocabList {
    struct LearnedVocab *head;

```

```

    int count;
} LearnedVocabList;
void runFilterLearnedVocab(KanjiList *allData);
#endif

=====
FILE: src/fuzzy_search.c
=====

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <wchar.h>
#ifdef _WIN32
    #include <windows.h>
#endif
#include "utils.h"
#include "../data_structures.h"
#define MAX_GAP 2
int min3(int a, int b, int c) {
    int m = a;
    if (b < m) m = b;
    if (c < m) m = c;
    return m;
}
int levenshteinDistance(wchar_t *s1, wchar_t *s2) {
    int len1 = wcslen(s1);
    int len2 = wcslen(s2);
    int i, j;
    int **arr = malloc((len1+1) * sizeof(int*));
    for (i = 0; i <= len1; i++) {
        arr[i] = malloc((len2 + 1) * sizeof(int));
    }
    for (i = 0; i <= len1; i++) arr[i][0] = i;
    for (j = 0; j <= len2; j++) arr[0][j] = j;
    int cost;
    for (i = 1; i <= len1; i++) {
        for (j = 1; j <= len2; j++) {
            cost = (s1[i-1] == s2[j-1]) ? 0 : 1;
            arr[i][j] = min3(
                arr[i-1][j] + 1,
                arr[i][j-1] + 1,
                arr[i-1][j-1] + cost
            );
        }
    }
    int res = arr[len1][len2];
    for (i = 0; i <= len1; i++) free(arr[i]);
    free(arr);
    return res;
}
void printFuzzyResult(int gap, Vocab *v, int counter) {

```

```

    if (!v) return;
    char tagStr[32];
    if (gap == 0) {
        sprintf(tagStr, CL_SUCCESS "[EXACT]" RESET);
    } else {
        sprintf(tagStr, CL_DIM "[GAP: %d]" RESET, gap);
    }
    printf(" " CL_PRIMARY " | " RESET CL_LOGO BOLD "%02d. " RESET CL_KANJI
    BOLD "%s " RESET
        CL_DIM "(" RESET CL_KANA "%s" RESET CL_DIM " • " RESET CL_ROMAJI
"%s" RESET CL_DIM ") " RESET CL_MEANING "%s" RESET CL_DIM " %s\n",
        counter,
        v->vocab,
        v->hiragana,
        v->romaji,
        v->meaning,
        tagStr
    );
}

void fuzzySearching(KanjiList *L, char *inputUTF8, int option) {
    wchar_t *wInput = convertToWchar(inputUTF8);
    int foundCount = 0;
    int i, j;
    int gap;
    size_t inputLen = wcslen(wInput);
    int current_max_gap = MAX_GAP;
    if (inputLen <= 3) {
        current_max_gap = 1;
    }
    for (i = 0; i < L->kanjiCount; i++) {
        for (j = 0; j < L->kanjis[i].vocabsCount; j++) {
            Vocab *v = &L->kanjis[i].vocabs[j];
            gap = 256;
            wchar_t *target_w = NULL;
            switch (option) {
                case 1: target_w = v->vocab_w; break;
                case 2: target_w = v->hiragana_w; break;
                case 3: target_w = v->romaji_w; break;
            }
            if (target_w != NULL) {
                size_t targetLen = wcslen(target_w);
                if (abs((int)inputLen - (int)targetLen) < 3) {
                    gap = levenshteinDistance(wInput, target_w);
                }
            }
            if (gap <= current_max_gap) {
                foundCount++;
                printFuzzyResult(gap, v, foundCount);
            }
        }
    }
}

```

```

    }
}
if (foundCount == 0) {
    printf(" " CL_WARN "⚠ Không tìm thấy kết quả nào phù hợp.\n" RESET);
} else {
    printf(" " CL_SUCCESS "✓ Tìm thấy %d kết quả phù hợp.\n" RESET, foundCount);
}
free(wInput);
}

```

```

=====
FILE: src/fuzzy_search.h
=====

```

```

#ifndef FUZZY_SEARCH_H
#define FUZZY_SEARCH_H
#include "../data_structures.h"
#include <wchar.h>
void fuzzySearching(KanjiList *L, const char *inputUTF8, int option);
#endif

```

```

=====
FILE: src/kanji_search_exact.c
=====

```

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#ifdef _WIN32
    #include <windows.h>
#endif
#include "kanji_search_exact.h"
#include "../data_structures.h"
#include "utils.h"
HashTableK* createHashTableK(int size) {
    HashTableK *ht = malloc(sizeof(HashTableK));
    if (ht == NULL) return NULL;
    ht->size = size;
    ht->count = 0;
    ht->table = malloc(sizeof(HashNodeK*) * size);
    for (int i = 0; i < ht->size; i++) {
        ht->table[i] = NULL;
    }
    return ht;
}
HashNodeK* createHashNodeK(char *key, Kanji* kanji) {
    HashNodeK *newNode = malloc(sizeof(HashNodeK));
    if (newNode == NULL) return NULL;
    newNode->key = strdup(key);
    newNode->kanji = kanji;
    newNode->next = NULL;
    return newNode;
}

```

```

static int hashGetIndex(char *str, int tableSize) {
    unsigned long hash = 5381;
    int c;
    while ((c = *str++)) {
        hash = ((hash << 5) + hash) + c;
    }
    return (int)(hash % tableSize);
}

void insertKanjiToHT(HashTableK *ht, char *key, Kanji *kanji) {
    if (!ht || !key || !kanji) return;
    int index = hashGetIndex(key, ht->size);
    HashNodeK *newNode = createHashNodeK(key, kanji);
    newNode->next = ht->table[index];
    ht->table[index] = newNode;
    ht->count++;
}

void buildHashTableForKanji(KanjiList *L, HashTableK *ht) {
    if (!L || !ht) return;
    for (int i = 0; i < L->kanjiCount; i++) {
        Kanji *k = &L->kanjis[i];
        if (k->kanji) insertKanjiToHT(ht, k->kanji, k);
        if (k->hanViet) insertKanjiToHT(ht, k->hanViet, k);
    }
}

void printOutKanji(Kanji *k) {
    if (k == NULL) return;
    printf(" " CL_PRIMARY " | " CL_DIM "Kanji      : " RESET " " CL_KANJI BOLD
"%s" RESET "\n", k->kanji ? k->kanji : "N/A");
    printf(" " CL_PRIMARY " | " CL_DIM "Hán Việt   : " RESET " " CL_MEANING
BOLD "%-15s" RESET " " CL_DIM "STT: %d\n" RESET, k->hanViet ? k->hanViet :
"N/A", k->stt);
    printf(" " CL_PRIMARY " | " CL_DIM "Bộ thủ     : " RESET " %-15s " CL_DIM "Số
nét: %s\n" RESET, k->radical ? k->radical : "N/A", k->stroke ? k->stroke : "N/A");
    printf(" " CL_PRIMARY " | " CL_DIM "Giải nghĩa : " RESET " %s\n", k->description ?
k->description : "N/A");
    printf(" " CL_PRIMARY " | \n");
    printf(" " CL_PRIMARY " | " CL_DIM "[Âm ON] : " RESET " ");
    if (k->onCount > 0) {
        for (int i = 0; i < k->onCount; i++) {
            printf(CL_KANJI BOLD "%s" RESET " " CL_KANA "(%s)" RESET "%s", k-
>on[i].jp, k->on[i].romaji, (i == k->onCount - 1) ? "" : ", ");
        }
    } else printf("N/A");
    printf("\n " CL_PRIMARY " | " CL_DIM "[Âm KUN] : " RESET " ");
    if (k->kunCount > 0) {
        for (int i = 0; i < k->kunCount; i++) {
            printf(CL_KANJI BOLD "%s" RESET " " CL_KANA "(%s)" RESET "%s", k-
>kun[i].jp, k->kun[i].romaji, (i == k->kunCount - 1) ? "" : ", ");
        }
    }
}

```

```

    } else printf("N/A");
    printf("\n " CL_PRIMARY " | \n");
    printf(" " CL_PRIMARY " | " CL_DIM "Từ vựng liên quan (%d):\n" RESET, k-
>vocabsCount);
    if (k->vocabsCount > 0 && k->vocabs != NULL) {
        for (int i = 0; i < k->vocabsCount; i++) {
            Vocab *v = &k->vocabs[i];
            printf(
                " " CL_PRIMARY " | " RESET
                CL_DIM "%02d." RESET " "
                CL_KANJI_BOLD "%s" RESET
                " " CL_DIM "(" RESET
                CL_KANA "%s" RESET
                CL_DIM " • " RESET
                CL_ROMAJI "%s" RESET
                CL_DIM ")" RESET
                " " CL_MEANING "%s\n",
                i + 1,
                v->vocab,
                v->hiragana ? v->hiragana : "-",
                v->romaji ? v->romaji : "-",
                v->meaning ? v->meaning : "-"
            );
            if (v->samplesCount > 0) {
                for (int j = 0; j < v->samplesCount; j++) {
                    printf(
                        " " CL_PRIMARY " | " RESET
                        CL_DIM " | " RESET
                        CL_JP_SENT "%s\n" RESET,
                        v->samples[j].jp
                    );
                    printf(
                        " " CL_PRIMARY " | " RESET
                        CL_DIM " └ " RESET
                        CL_VN_SENT "%s\n" RESET,
                        v->samples[j].vn
                    );
                }
            }
        }
    } else {
        printf(" " CL_PRIMARY " | " RESET CL_DIM "(Hiện không có từ vựng đi kèm)\n"
RESET);
    }
    printf("\n");
}

void exactlySearchingKanji(HashTableK *ht, char *key) {
    if (!ht || !key) return;
    int index = hashGetIndex(key, ht->size);

```

```

    HashNodeK *current = ht->table[index];
    int found = 0;
    while (current != NULL) {
        if (strcmp(current->key, key) == 0) {
            printOutKanji(current->kanji);
            found = 1;
        }
        current = current->next;
    }
    if (!found) {
        printf(" " CL_WARN "\u2609 Không tìm thấy kết quả Kanji cho: " CL_LOGO "%s\n"
RESET, key);
    }
}

```

```

=====
FILE: src/kanji_search_exact.h
=====

```

```

#ifndef kanji_search_exact_H
#define kanji_search_exact_H
#include "../data_structures.h"
#define HASH_TABLE_SIZE 10007
struct HashNodeForKanji {
    char *key;
    Kanji *kanji;
    struct HashNodeForKanji *next;
};
struct HashTableForKanji {
    struct HashNodeForKanji **table;
    int size;
    int count;
};
typedef struct HashNodeForKanji HashNodeK;
typedef struct HashTableForKanji HashTableK;
HashTableK* createHashTableK(int size);
void buildHashTableForKanji(KanjiList *L, HashTableK *ht);
void exactlySearchingKanji(HashTableK *ht, char *key);
#endif

```

```

=====
FILE: src/lessons_management.c
=====

```

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#ifdef _WIN32
    #include <windows.h>
#endif
#include "lessons_management.h"
#include "../data_structures.h"
#include "utils.h"

```

```

#define KANJI_PER_LESSON 16
void displayAllLessons(KanjiList *L) {
    int totalLessons = L->kanjiCount / KANJI_PER_LESSON;
    int i, j;
    printf("\n  " BG_HIGHLIGHT BOLD " DANH SÁCH BÀI HỌC TỔNG HỢP "
RESET"\n");
    for (i = 0; i < totalLessons; i++) {
        int startIndex = i * KANJI_PER_LESSON;
        int endIndex = startIndex + KANJI_PER_LESSON - 1;
        if (endIndex >= L->kanjiCount) endIndex = L->kanjiCount - 1;
        printf("  " CL_PRIMARY " | " CL_DIM "Bài" CL_PRIMARY " %02d " CL_DIM
("(%3d - %3d) : " RESET, i + 1, startIndex + 1, endIndex + 1);
        for (j = 0; j < KANJI_PER_LESSON; j++) {
            printf(CL_KANJI BOLD "%s" RESET " ", L->kanjis[j + startIndex].kanji);
        }
        printf("\n");
    }
}

void displayDetailedKanji(Kanji *k) {
    if (k == NULL) return;
    clearScreen();
    printf("\n " BG_HIGHLIGHT BOLD " THÔNG TIN CHI TIẾT KANJI " RESET"\n\n");
    printf("  " CL_PRIMARY " | " CL_DIM "Kanji:" RESET "  " CL_KANJI BOLD "%s"
RESET "\n", k->kanji ? k->kanji : "N/A");
    printf("  " CL_PRIMARY " | " CL_DIM "Hán Việt  :" RESET "  " CL_MEANING
BOLD "%-15s" RESET "  " CL_DIM "STT: %d\n" RESET, k->hanViet ? k->hanViet :
"N/A", k->stt);
    printf("  " CL_PRIMARY " | " CL_DIM "Bộ thủ  :" RESET " %-15s " CL_DIM "Số
nét: %s\n" RESET, k->radical ? k->radical : "N/A", k->stroke ? k->stroke : "N/A");
    printf("  " CL_PRIMARY " | " CL_DIM "Giải nghĩa : " RESET " %s\n", k->description ?
k->description : "N/A");
    printf("  " CL_PRIMARY " | \n");
    printf("  " CL_PRIMARY " | " CL_DIM "[Âm ON] : " RESET " ");
    if (k->onCount > 0) {
        for (int i = 0; i < k->onCount; i++) {
            printf(CL_KANJI BOLD "%s" RESET "  " CL_KANA "(%s)" RESET "%s", k-
>on[i].jp, k->on[i].romaji, (i == k->onCount - 1) ? "" : ", ");
        }
    } else printf("N/A");
    printf("\n  " CL_PRIMARY " | " CL_DIM "[Âm KUN] : " RESET " ");
    if (k->kunCount > 0) {
        for (int i = 0; i < k->kunCount; i++) {
            printf(CL_KANJI BOLD "%s" RESET "  " CL_KANA "(%s)" RESET "%s", k-
>kun[i].jp, k->kun[i].romaji, (i == k->kunCount - 1) ? "" : ", ");
        }
    } else printf("N/A");
    printf("\n  " CL_PRIMARY " | \n");
}

```



```

    printf(" " CL_PRIMARY " | " CL_DIM "Từ vựng liên quan (%d):\n" RESET, k-
>vocabsCount);
    if (k->vocabsCount > 0 && k->vocabs != NULL) {
        for (int i = 0; i < k->vocabsCount; i++) {
            Vocab *v = &k->vocabs[i];
            printf(
                " " CL_PRIMARY " | " RESET
                CL_DIM "%02d." RESET " "
                CL_KANJI_BOLD "%s" RESET
                " " CL_DIM "(" RESET
                CL_KANA "%s" RESET
                CL_DIM " • " RESET
                CL_ROMAJI "%s" RESET
                CL_DIM ")" RESET
                " " CL_MEANING "%s\n",
                i + 1,
                v->vocab,
                v->hiragana ? v->hiragana : "-",
                v->romaji ? v->romaji : "-",
                v->meaning ? v->meaning : "-"
            );
            if (v->samplesCount > 0) {
                for (int j = 0; j < v->samplesCount; j++) {
                    printf(
                        " " CL_PRIMARY " | " RESET
                        CL_DIM " | " RESET
                        CL_JP_SENT "%s\n" RESET,
                        v->samples[j].jp
                    );
                    printf(
                        " " CL_PRIMARY " | " RESET
                        CL_DIM " └ " RESET
                        CL_VN_SENT "%s\n" RESET,
                        v->samples[j].vn
                    );
                }
            }
            printf(" " CL_PRIMARY " | \n" RESET);
        }
    } else {
        printf(
            " " CL_PRIMARY " | " RESET
            CL_DIM "Không có dữ liệu từ vựng liên quan.\n" RESET
        );
    }
    printf(" " CL_DIM ">> Nhấn Enter để quay lại..." RESET);
    getchar();
}
void selectAndDisplayLesson(KanjiList *L) {

```

```

int lessonChoice;
int totalLessons = (L->kanjiCount + KANJI_PER_LESSON - 1) /
KANJI_PER_LESSON;
printf("\n " CL_DIM ">> Bạn cần xem bài số:" RESET " " CL_LOGO BOLD);
scanf("%d", &lessonChoice);
printf(RESET);
getchar();
if (lessonChoice < 1 || lessonChoice > totalLessons) {
    printf(" " CL_ERROR " 🚫 [LỖI] Chỉ số bài học không tồn tại.\n" RESET);
    return;
}
int startIndex = (lessonChoice - 1) * KANJI_PER_LESSON;
int endIndex = startIndex + KANJI_PER_LESSON;
if (endIndex > L->kanjiCount) endIndex = L->kanjiCount;
while (1) {
    clearScreen();
    printf("\n " BG_HIGHLIGHT BOLD " BÀI %02d : DANH SÁCH KANJI " RESET
"\n", lessonChoice);
    for (int i = startIndex; i < endIndex; i++) {
        printf(" " CL_PRIMARY " | " CL_DIM "%02d | " RESET CL_KANJI BOLD
"%0-6s" RESET CL_DIM " | " RESET " " CL_MEANING "%0-30s" RESET "\n",
            i + 1, L->kanjis[i].kanji, L->kanjis[i].hanViet);
    }
    int kanjiIdx;
    printf(CL_DIM ">> Nhập số thứ tự để xem chi tiết Kanji (0 để thoát):" RESET " "
CL_LOGO BOLD);
    scanf("%d", &kanjiIdx);
    printf(RESET);
    if (kanjiIdx == 0) break;
    if (kanjiIdx < startIndex + 1 || kanjiIdx > endIndex) {
        printf(" " CL_ERROR " 🚫 [LỖI] Lựa chọn không hợp lệ. Vui lòng nhập lại.\n"
RESET);
        getchar();
        continue;
    }
    displayDetailedKanji(&L->kanjis[kanjiIdx - 1]);
    getchar();
}
}

```

FILE: src/lessons_management.h

```

#ifndef LESSONS_MANAGEMENT_H
#define LESSONS_MANAGEMENT_H
#include "../data_structures.h"
void displayAllLessons(KanjiList *L);
void displayDetailedKanji(Kanji *k);
void selectAndDisplayLesson(KanjiList *L);
#endif

```

FILE: src/parse_json_to_struct.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <wchar.h>
#ifdef _WIN32
    #include <windows.h>
#endif
#include "parse_json_to_struct.h"
#include "utils.h"
#include "../lib/cJSON.h"
#include "../data_structures.h"
char* readFileToString(const char *fileName) {
    FILE *file = fopen(fileName, "rb");
    if (!file) return NULL;
    fseek(file, 0, SEEK_END);
    long length = ftell(file);
    fseek(file, 0, SEEK_SET);
    char *data = malloc(length + 1);
    if (data) {
        fread(data, 1, length, file);
        data[length] = '\0';
    }
    fclose(file);
    return data;
}
void configUTF8() {
#ifdef _WIN32
    system("chcp 65001 > nul");
#endif
}
char* safeStrdup(cJSON *item) {
    if (item && item->valuelstring) {
        return strdup(item->valuelstring);
    }
    return strdup("");
}
Yomi* parseYomi(cJSON *yomiArr, int *count) {
    if (!yomiArr || !cJSON_IsArray(yomiArr)) {
        *count = 0;
        return NULL;
    }
    *count = cJSON_GetArraySize(yomiArr);
    Yomi *yomi = malloc(sizeof(Yomi) * (*count));
    for (int i = 0; i < *count; i++) {
        cJSON *item = cJSON_GetArrayItem(yomiArr, i);
        yomi[i].jp = safeStrdup(cJSON_GetObjectItem(item, "jp"));
        yomi[i].romaji = safeStrdup(cJSON_GetObjectItem(item, "romaji"));
    }
}
```

```

    }
    return yomi;
}
Sample* parseSamples(cJSON *samplesArr, int *count) {
    if (!samplesArr || !cJSON_IsArray(samplesArr)) {
        *count = 0;
        return NULL;
    }
    *count = cJSON_GetArraySize(samplesArr);
    Sample *samples = malloc(sizeof(Sample) * (*count));
    for (int i = 0; i < *count; i++) {
        cJSON *item = cJSON_GetArrayItem(samplesArr, i);
        samples[i].jp = safeStrdup(cJSON_GetObjectItem(item, "jp"));
        samples[i].vn = safeStrdup(cJSON_GetObjectItem(item, "vn"));
    }
    return samples;
}
Vocab* parseVocabs(cJSON *vocabsArr, int *count) {
    if (!vocabsArr || !cJSON_IsArray(vocabsArr)) {
        *count = 0;
        return NULL;
    }
    *count = cJSON_GetArraySize(vocabsArr);
    Vocab *vocabs = malloc(sizeof(Vocab) * (*count));
    for (int i = 0; i < *count; i++) {
        cJSON *item = cJSON_GetArrayItem(vocabsArr, i);
        Vocab *v = &vocabs[i];
        v->vocab = safeStrdup(cJSON_GetObjectItem(item, "vocab"));
        v->hiragana = safeStrdup(cJSON_GetObjectItem(item, "hiragana"));
        v->romaji = safeStrdup(cJSON_GetObjectItem(item, "romaji"));
        v->meaning = safeStrdup(cJSON_GetObjectItem(item, "meaning"));
        v->vocab_w = convertToWchar(v->vocab);
        v->hiragana_w = convertToWchar(v->hiragana);
        v->romaji_w = convertToWchar(v->romaji);
        v->meaning_w = convertToWchar(v->meaning);
        v->samples = parseSamples(cJSON_GetObjectItem(item, "samples"), &v->samplesCount);
    }
    return vocabs;
}
KanjiList toJsonToStruct(const char *jsonString) {
    KanjiList data = {NULL, 0};
    cJSON *root = cJSON_Parse(jsonString);
    if (!root) {
        printf(CL_ERROR " ⚠ Lỗi: Parse JSON thất bại!\n" RESET);
        return data;
    }
    data.kanjiCount = cJSON_GetArraySize(root);
    data.kanjis = malloc(sizeof(Kanji) * data.kanjiCount);

```

```

    for (int i = 0; i < data.kanjiCount; i++) {
        cJSON *item = cJSON_GetArrayItem(root, i);
        Kanji *k = &data.kanjis[i];
        k->stt      = cJSON_GetObjectItem(item, "stt") ? cJSON_GetObjectItem(item, "stt")-
>valueint : 0;
        k->kanji    = safeStrdup(cJSON_GetObjectItem(item, "kanji"));
        k->hanViet  = safeStrdup(cJSON_GetObjectItem(item, "hanViet"));
        k->radical  = safeStrdup(cJSON_GetObjectItem(item, "radical"));
        k->stroke   = safeStrdup(cJSON_GetObjectItem(item, "stroke"));
        k->description = safeStrdup(cJSON_GetObjectItem(item, "description"));
        k->on       = parseYomi(cJSON_GetObjectItem(item, "on"), &k->onCount);
        k->kun      = parseYomi(cJSON_GetObjectItem(item, "kun"), &k->kunCount);
        k->vocabs   =      parseVocabs(cJSON_GetObjectItem(item, "vocabs"), &k-
>vocabsCount);
        k->kanji_w = convertToWchar(k->kanji);
    }
    cJSON_Delete(root);
    return data;
}

```

FILE: src/parse_json_to_struct.h

```

#ifndef PARSE_JSON_TO_STRUCT_H
#define PARSE_JSON_TO_STRUCT_H
#include "../data_structures.h"
#include "../lib/cJSON.h"
wchar_t* convertToWchar(const char *source);
char* readFileToString(const char *fileName);
void configUTF8();
KanjiList parseJsonToStruct(const char *jsonString);
#endif

```

FILE: src/prefix_search.c

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include "prefix_search.h"
#include "../data_structures.h"
#include "utils.h"
#define ALPHABET_SIZE 28
TrieNode* createTrieNode() {
    TrieNode *node = (TrieNode*)malloc(sizeof(TrieNode));
    if (node) {
        node->isEndOfWord = 0;
        node->vocabsCount = 0;
        node->vocabs = NULL;
        for (int i = 0; i < ALPHABET_SIZE; i++) {
            node->children[i] = NULL;

```

```

    }
}
return node;
}
int getCharIndex(char c) {
    c = tolower(c);
    if (c >= 'a' && c <= 'z') return c - 'a';
    if (c == '-') return 26;
    if (c == '.') return 27;
    return -1;
}
void insertTrie(TrieNode *root, const char *key, Vocab *vocab) {
    if (!root || !key || strlen(key) == 0) return;
    TrieNode *current = root;
    for (int i = 0; key[i] != '\0'; i++) {
        int index = getCharIndex(key[i]);
        if (index == -1) continue;
        if (!current->children[index]) {
            current->children[index] = createTrieNode();
        }
        current = current->children[index];
    }
    current->isEndOfWord = 1;
    Vocab **temp = (Vocab**)realloc(current->vocabs, (current->vocabsCount + 1) *
sizeof(Vocab*));
    if (temp != NULL) {
        current->vocabs = temp;
        current->vocabs[current->vocabsCount++] = vocab;
    }
}
TrieNode* buildTrieFromKanjiList(KanjiList *L) {
    if (!L) return NULL;
    TrieNode *root = createTrieNode();
    if (!root) return NULL;
    for (int i = 0; i < L->kanjiCount; i++) {
        Kanji *k = &L->kanjis[i];
        for (int j = 0; j < k->vocabsCount; j++) {
            Vocab *v = &k->vocabs[j];
            if (v->romaji && strlen(v->romaji) > 0) {
                insertTrie(root, v->romaji, v);
            }
        }
    }
    return root;
}
TrieNode* searchPrefixNode(TrieNode *root, const char *prefix) {
    if (!root || !prefix) return NULL;
    TrieNode *current = root;
    for (int i = 0; prefix[i] != '\0'; i++) {
        int index = getCharIndex(prefix[i]);

```

```

        if (index == -1) return NULL;
        if (!current->children[index]) {
            return NULL;
        }
        current = current->children[index];
    }
    return current;
}

void freeTrie(TrieNode *root) {
    if (!root) return;
    for (int i = 0; i < ALPHABET_SIZE; i++) {
        if (root->children[i]) {
            freeTrie(root->children[i]);
        }
    }
    if (root->vocabs) {
        free(root->vocabs);
    }
    free(root);
}

void printVocabDetails(Vocab *v) {
    if (!v) return;
    if (v->samplesCount > 0 && v->samples) {
        for (int i = 0; i < v->samplesCount && i < 2; i++) {
            printf(" " CL_PRIMARY " | " RESET " " CL_DIM " | " RESET CL_JP_SENT
"%s\n" RESET,
                v->samples[i].jp ? v->samples[i].jp : "");
            printf(" " CL_PRIMARY " | " RESET " " CL_DIM " | " RESET CL_VN_SENT
"%s\n" RESET,
                v->samples[i].vn ? v->samples[i].vn : "");
        }
    }
    printf(" " CL_PRIMARY " | \n" RESET);
}

void printAllWordsFromNode(TrieNode *node, int *counter) {
    if (!node) return;
    if (node->isEndOfWord && node->vocabs) {
        for (int i = 0; i < node->vocabsCount; i++) {
            (*counter)++;
            Vocab *v = node->vocabs[i];
            printf(" " CL_PRIMARY " | " RESET CL_LOGO BOLD "%02d. " RESET
CL_KANJI BOLD "%s " RESET
                CL_DIM "(" RESET CL_KANA "%s" RESET CL_DIM " • " RESET
CL_ROMAJI "%s" RESET CL_DIM ")" RESET CL_MEANING " %s\n" RESET ,
                *counter,
                v->vocab,
                v->hiragana,
                v->romaji,
                v->meaning);
        }
    }
}

```

```

        printVocabDetails(v);
    }
}
for (int i = 0; i < ALPHABET_SIZE; i++) {
    if (node->children[i]) {
        printAllWordsFromNode(node->children[i], counter);
    }
}
}
}

```

FILE: src/prefix_search.h

```

#ifndef PREFIX_SEARCH_H
#define PREFIX_SEARCH_H
#include "../data_structures.h"
#include "dashboard.h"
#define ALPHABET_SIZE 28
typedef struct TrieNodeInfo TrieNode;
struct TrieNodeInfo {
    TrieNode *children[ALPHABET_SIZE];
    int isEndOfWord;
    Vocab **vocabs;
    int vocabsCount;
};
TrieNode* buildTrieFromKanjiList(KanjiList *L);
TrieNode* searchPrefixNode(TrieNode *root, const char *prefix);
void printAllWordsFromNode(TrieNode *node, int *counter);
void freeTrie(TrieNode *root);
#endif

```

FILE: src/sentence_analysis.c

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <wchar.h>
#ifdef _WIN32
    #include <windows.h>
#endif
#include "../data_structures.h"
#include "filter_learned_lesson.h"
#include "utils.h"
#define MAX_SENTENCE 1024
#define HASH_SIZE 512
typedef struct KanjiNode {
    Kanji* kanjiData;
    struct KanjiNode* next;
} KanjiNode;
typedef struct {
    KanjiNode* buckets[512];

```



```

} KanjiHashTable;
unsigned int hashKanjiChar(wchar_t wc) {
    unsigned long hash = 5381;
    hash = ((hash << 5) + hash) + (unsigned long)wc;
    return hash % HASH_SIZE;
}
KanjiHashTable* createKanjiHashTable(KanjiList *allData) {
    KanjiHashTable* table = (KanjiHashTable*)calloc(1, sizeof(KanjiHashTable));
    if (!table) return NULL;
    for (int i = 0; i < allData->kanjiCount; i++) {
        wchar_t* kw = allData->kanjis[i].kanji_w;
        if (!kw || !kw[0]) continue;
        unsigned int idx = hashKanjiChar(kw[0]);
        KanjiNode* node = (KanjiNode*)malloc(sizeof(KanjiNode));
        if (node) {
            node->kanjiData = &allData->kanjis[i];
            node->next = table->buckets[idx];
            table->buckets[idx] = node;
        }
    }
    return table;
}
Kanji* findKanji(KanjiHashTable* table, wchar_t wc) {
    if (!table) return NULL;
    unsigned int idx = hashKanjiChar(wc);
    KanjiNode* curr = table->buckets[idx];
    while (curr) {
        if (curr->kanjiData->kanji_w[0] == wc)
            return curr->kanjiData;
        curr = curr->next;
    }
    return NULL;
}
void freeKanjiHashTable(KanjiHashTable* table) {
    if (!table) return;
    for (int i = 0; i < 512; i++) {
        KanjiNode* curr = table->buckets[i];
        while (curr) {
            KanjiNode* temp = curr;
            curr = curr->next;
            free(temp);
        }
    }
    free(table);
}
void printOutKanjiInfo(Kanji* info) {
    if (!info) {
        printf("\n " CL_WARN "⚠ Không có thông tin Kanji.\n" RESET);
        return;
    }
}

```

```

    }
    printf(" " CL_PRIMARY " | " CL_DIM "Kanji      :" RESET " " CL_KANJI_BOLD
"%s" RESET " " CL_DIM "—" RESET " " CL_MEANING_BOLD "%s" RESET " "
CL_DIM "(STT: %d)" RESET "\n",
    info->kanji ? info->kanji : "N/A",
    info->hanViet ? info->hanViet : "N/A",
    info->stt);
    printf(" " CL_PRIMARY " | " CL_DIM "Bộ thủ      :" RESET " %-15s " CL_DIM "Số
nét: %s\n" RESET,
    info->radical ? info->radical : "N/A",
    info->stroke ? info->stroke : "N/A");
    printf(" " CL_PRIMARY " | " CL_DIM "Giải nghĩa :" RESET " " CL_TEXT "%s\n",
    info->description ? info->description : "Chưa có mô tả.");
    printf(" " CL_PRIMARY " | \n");
    printf(" " CL_PRIMARY " | " CL_DIM "[Âm ON]      :" RESET " ");
    if (info->onCount > 0) {
        for (int i = 0; i < info->onCount; i++) {
            printf(CL_KANJI_BOLD "%s" RESET " " CL_KANA "(%s)" RESET "%s", info-
>on[i].jp, info->on[i].romaji, (i == info->onCount - 1) ? "" : ", ");
        }
    } else printf("N/A");
    printf("\n " CL_PRIMARY " | " CL_DIM "[Âm KUN]    :" RESET " ");
    if (info->kunCount > 0) {
        for (int i = 0; i < info->kunCount; i++) {
            printf(CL_KANJI_BOLD "%s" RESET " " CL_KANA "(%s)" RESET "%s", info-
>kun[i].jp, info->kun[i].romaji, (i == info->kunCount - 1) ? "" : ", ");
        }
    } else printf("N/A");
    printf("\n " CL_PRIMARY " | \n");
    printf(" " CL_PRIMARY " | " CL_DIM "Từ vựng liên quan (%d):\n" RESET, info-
>vocabsCount);
    if (info->vocabsCount > 0 && info->vocabs != NULL) {
        for (int j = 0; j < info->vocabsCount; j++) {
            Vocab* v = &info->vocabs[j];
            printf(
                " " CL_PRIMARY " | " RESET
                CL_DIM "%02d." RESET " "
                CL_KANJI_BOLD "%s" RESET
                " " CL_DIM "(" RESET
                CL_KANA "%s" RESET
                CL_DIM " • " RESET
                CL_ROMAJI "%s" RESET
                CL_DIM ")" RESET
                " " CL_MEANING "%s\n",
                j + 1,
                v->vocab ? v->vocab : "?",
                v->hiragana ? v->hiragana : "?",
                v->romaji ? v->romaji : "?",
            );
        }
    }
}

```

```

        v->meaning ? v->meaning : "?"
    );
}
} else {
    printf(" " CL_PRIMARY " | " RESET CL_DIM " (Hệ thống chưa nạp từ vựng liên
đối)\n" RESET);
    printf(" " CL_PRIMARY " | \n" RESET);
}
printf("\n");
}
void analyzeJapaneseSentence(KanjiList *allData) {
    clearScreen();
    if (!allData || allData->kanjiCount == 0) {
        printf("\n " CL_ERROR " 🚫 [HỆ THỐNG] Phân vùng bộ nhớ trống. Chưa nạp cơ sở
dữ liệu Kanji.\n" RESET);
        waitForEnter();
        return;
    }
    KanjiHashTable* kanjiTable = createKanjiHashTable(allData);
    if (!kanjiTable) {
        printf("\n " CL_ERROR " 🚫 [HỆ THỐNG] Lỗi nghiêm trọng: Phân bổ thực thể bảng
băm (Heap Allocation) thất bại.\n" RESET);
        return;
    }
    char sentence[MAX_SENTENCE];
    printf("\n " BG_HIGHLIGHT BOLD " TRA CỨU KANJI TRONG CÂU " RESET "\n");
    printf(CL_BORDER
"
-----\n" RESET);
    printf(" " CL_TEXT "Hãy nhập câu cần phân tích các Kanji trong đó:\n" RESET);
    printf(CL_LOGO BOLD " " RESET CL_LOGO BOLD);
    inputString(sentence, MAX_SENTENCE);
    printf(RESET);
    if (strlen(sentence) == 0) {
        printf("\n " CL_WARN " ⚠️ [CẢNH BÁO] Chuỗi rỗng. Tiến trình phân rã bị hủy bỏ
bởi người dùng.\n" RESET);
        freeKanjiHashTable(kanjiTable);
        waitForEnter();
        return;
    }
    wchar_t* wsentence = convertToWchar(sentence);
    if (!wsentence) {
        printf("\n " CL_ERROR " 🚫 [LỖI MÃ HÓA] Không thể chuyển đổi UTF-8 sang bộ
ký tự rộng wchar_t.\n" RESET);
        freeKanjiHashTable(kanjiTable);
        return;
    }
    clearScreen();
    printf(" " CL_PRIMARY ">> " RESET BOLD "%s\n" RESET, sentence);

```

```

printf(CL_BORDER
"
-----\n" RESET);
int found = 0;
for (int i = 0; wsentence[i] != L'\0'; i++) {
    wchar_t ch = wsentence[i];
    if (isKanjiWChar(ch)) {
        Kanji* info = findKanji(kanjiTable, ch);
        if (info) {
            found = 1;
            printOutKanjiInfo(info);
        }
    }
}
if (!found) {
    printf(" " CL_WARN "⚠ Không tìm thấy phần tử Kanji nào thuộc phạm vi dữ liệu đã
học trong câu trên.\n" RESET);
}
printf(CL_BORDER
"
-----\n" RESET);

free(wsentence);
freeKanjiHashTable(kanjiTable);
waitForEnter();
}

=====
FILE: src/sentence_analysis.h
=====
#ifndef SENTENCE_ANALYSIS_H
#define SENTENCE_ANALYSIS_H
#include "../data_structures.h"
void analyzeJapaneseSentence(KanjiList *allData);
#endif

=====
FILE: src/substring_search.c
=====
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <wchar.h>
#ifdef _WIN32
    #include <windows.h>
#endif
#include "utils.h"
#include "../data_structures.h"
void computeLPSArray(wchar_t *pat, int M, int *lps) {
    int len = 0;
    lps[0] = 0;
    int i = 1;

```

```

while (i < M) {
    if (pat[i] == pat[len]) {
        len++;
        lps[i] = len;
        i++;
    } else {
        if (len != 0) {
            len = lps[len - 1];
        } else {
            lps[i] = 0;
            i++;
        }
    }
}
}

int KMPsearch(wchar_t *pat, wchar_t *txt) {
    int M = wcslen(pat);
    int N = wcslen(txt);
    int *lps = malloc(M * sizeof(int));
    computeLPSArray(pat, M, lps);
    int i = 0, j = 0;
    while (i < N) {
        if (pat[j] == txt[i]) {
            i++;
            j++;
        }
        if (j == M) {
            free(lps);
            return 1;
        } else if (i < N && pat[j] != txt[i]) {
            if (j != 0)
                j = lps[j - 1];
            else
                i++;
        }
    }
    free(lps);
    return 0;
}

void printSubstringResult(Vocab *v, int counter) {
    if (!v) return;
    printf(" CL_PRIMARY " | " RESET CL_LOGO BOLD "%02d. " RESET CL_KANJI
    BOLD "%s " RESET
        CL_DIM "(" RESET CL_KANA "%s" RESET CL_DIM " • " RESET CL_ROMAJI
"%s" RESET CL_DIM ") " RESET CL_MEANING "%s\n",
        counter,
        v->vocab,
        v->hiragana,
        v->romaji,

```

```

        v->meaning
    );
}
void substringSearching(KanjiList *L, char *inputUTF8, int option) {
    wchar_t *wInput = convertToWchar(inputUTF8);
    int foundCount = 0;
    size_t inputLen = wcslen(wInput);
    for (int i = 0; i < L->kanjiCount; i++) {
        for (int j = 0; j < L->kanjis[i].vocabsCount; j++) {
            Vocab *v = &L->kanjis[i].vocabs[j];
            wchar_t *target_w = NULL;
            switch (option) {
                case 1: target_w = v->vocab_w; break;
                case 2: target_w = v->hiragana_w; break;
                case 3: target_w = v->romaji_w; break;
                case 4: target_w = v->meaning_w; break;
            }
            if (target_w && KMPsearch(wInput, target_w)) {
                foundCount++;
                printSubstringResult(v, foundCount);
            }
        }
    }
    if (foundCount == 0) {
        printf(" " CL_WARN "\u2609 Không tìm thấy kết quả nào phù hợp với mẫu: " CL_LOGO
"%s\n" RESET, inputUTF8);
    } else {
        printf(" " CL_SUCCESS "\u2713 Tìm thấy %d kết quả phù hợp.\n" RESET, foundCount);
    }
    free(wInput);
}

```

FILE: src/substring_search.h

```

#ifndef SUBSTRING_SEARCH_H
#define SUBSTRING_SEARCH_H
#include "../data_structures.h"
#include <wchar.h>
void substringSearching(KanjiList *L, char *inputUTF8, int option);
#endif

```

FILE: src/utlis.c

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#ifdef _WIN32
    #include <windows.h>
#endif

```

```

#include <wchar.h>
#include <utils.h>
wchar_t* convertToWchar(const char *source)
{
    if (!source || *source == '\0')
        return NULL;
    int w_len = MultiByteToWideChar(CP_UTF8, 0, source, -1, NULL, 0);
    if (w_len <= 0)
        return NULL;
    wchar_t *w_str = (wchar_t*)malloc(w_len * sizeof(wchar_t));
    if (!w_str)
        return NULL;
    MultiByteToWideChar(CP_UTF8, 0, source, -1, w_str, w_len);
    return w_str;
}
int isKanjiWChar(wchar_t wc) {
    return (wc >= 0x4E00 && wc <= 0x9FFF) ||
        (wc >= 0x3400 && wc <= 0x4DBF);
}
void inputString(char *buffer, int maxLength) {
    if (!buffer || maxLength <= 0) return;
    fflush(stdin);
#ifdef _WIN32
    wchar_t *wBuf = (wchar_t*)malloc(maxLength * sizeof(wchar_t));
    if (!wBuf) {
        buffer[0] = '\0';
        return;
    }
    DWORD read = 0;
    HANDLE hStdin = GetStdHandle(STD_INPUT_HANDLE);
    if (ReadConsoleW(hStdin, wBuf, maxLength - 1, &read, NULL) && read > 0) {
        if (read >= 2 && wBuf[read-2] == L'r' && wBuf[read-1] == L'n')
            wBuf[read-2] = L'\0';
        else if (read >= 1 && (wBuf[read-1] == L'n' || wBuf[read-1] == L'r'))
            wBuf[read-1] = L'\0';
        else
            wBuf[read] = L'\0';
        WideCharToMultiByte(CP_UTF8, 0, wBuf, -1, buffer, maxLength, NULL, NULL);
    } else {
        buffer[0] = '\0';
    }
    free(wBuf);
#else
    if (fgets(buffer, maxLength, stdin)) {
        buffer[strcspn(buffer, "\n\r")] = '\0';
    } else {
        buffer[0] = '\0';
    }
#endif
}

```

```

void clearScreen(void) {
    system("cls");
}
void waitEnter(void) {
    printf("\n " CL_DIM ">> Nhấn Enter để tiếp tục..." RESET);
    getchar();
}

```

FILE: src/utils.h

```

#ifndef UTILS_H
#define UTILS_H
#include <wchar.h>
#define CL_PRIMARY "\x1b[38;2;137;234;254m"
#define CL_KEY "\x1b[38;2;137;234;254m"
#define CL_LOGO "\x1b[38;2;137;234;254m"
#define CL_TEXT "\x1b[38;2;230;230;230m"
#define CL_DIM "\x1b[38;2;110;110;110m"
#define CL_WARN "\x1b[38;2;255;212;102m"
#define CL_HEADER "\x1b[38;2;255;212;102m"
#define CL_ERROR "\x1b[38;2;255;107;107m"
#define CL_SUCCESS "\x1b[38;2;114;242;151m"
#define CL_KANJI "\x1b[38;2;255;92;141m"
#define CL_KANA "\x1b[38;2;255;160;122m"
#define CL_ROMAJI "\x1b[38;2;147;226;214m"
#define CL_MEANING "\x1b[38;2;194;161;255m"
#define CL_JP_SENT "\x1b[38;2;117;213;253m"
#define CL_VN_SENT "\x1b[38;2;244;241;134m"
#define CL_BORDER "\x1b[38;2;82;104;133m"
#define BG_PANEL "\x1b[48;2;42;44;54m"
#define BG_MENU "\x1b[48;2;42;44;54m"
#define BG_HIGHLIGHT "\x1b[48;2;137;234;254m\x1b[38;2;42;44;54m"
#define BOLD "\x1b[1m"
#define CL_RESET "\x1b[0m"
#define RESET "\x1b[0m"
wchar_t* convertToWchar(const char *source);
int isKanjiWChar(wchar_t wc);
void inputString(char *buffer, int maxLength);
void clearScreen(void);
void waitEnter(void);
#endif

```

FILE: src/vocab_search_exact.c

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#ifdef _WIN32
    #include <windows.h>
#endif

```



```

#include "vocab_search_exact.h"
#include "../data_structures.h"
#include "utils.h"
HashTable* createHashTable(int size) {
    HashTable *ht = malloc(sizeof(HashTable));
    if (ht == NULL) return NULL;
    ht->size = size;
    ht->count = 0;
    ht->table = malloc(sizeof(HashNode*) * size);
    int i;
    for (i = 0; i < ht->size; i++) {
        ht->table[i] = NULL;
    }
    return ht;
}
HashNode* createHashNode(char *key, Vocab* vocab) {
    HashNode *newNode = malloc(sizeof(HashNode));
    newNode->key = strdup(key);
    newNode->vocab = vocab;
    newNode->next = NULL;
    return newNode;
}
static int hashGetIndex(char *str, int tableSize) {
    unsigned long hash = 5381;
    int c;
    while ((c = *str++)) {
        hash = ((hash << 5) + hash) + c;
    }
    return (int)(hash % tableSize);
}
void insertVocabToHT(HashTable *ht, char *key, Vocab *vocab) {
    if (!ht || !key || !vocab) return;
    int indexHT = hashGetIndex(key, ht->size);
    HashNode *newNode = createHashNode(key, vocab);
    newNode->next = ht->table[indexHT];
    ht->table[indexHT] = newNode;
    ht->count++;
}
void buildHashTableForVocab(KanjiList *L, HashTable *ht) {
    int i, j;
    for (i = 0; i < L->kanjiCount; i++) {
        Kanji *k = &L->kanjis[i];
        for (j = 0; j < k->vocabsCount; j++) {
            Vocab *v = &k->vocabs[j];
            insertVocabToHT(ht, v->vocab, v);
            insertVocabToHT(ht, v->romaji, v);
            insertVocabToHT(ht, v->hiragana, v);
            insertVocabToHT(ht, v->meaning, v);
        }
    }
}

```

```

}
void printOutVocab(Vocab *v) {
    if (v == NULL) return;
    printf(
        " " CL_PRIMARY " | " RESET
        CL_KANJI_BOLD "%s" RESET
        " " CL_DIM "(" RESET
        CL_KANA "%s" RESET
        CL_DIM " • " RESET
        CL_ROMAJI "%s" RESET
        CL_DIM ")" RESET
        " " CL_MEANING "%s\n",
        v->vocab ? v->vocab : "-",
        v->hiragana ? v->hiragana : "-",
        v->romaji ? v->romaji : "-",
        v->meaning ? v->meaning : "-"
    );
    if (v->samplesCount > 0 && v->samples != NULL) {
        for (int i = 0; i < v->samplesCount; i++) {
            printf(
                " " CL_PRIMARY " | " RESET
                CL_DIM " | " RESET
                CL_JP_SENT "%s\n" RESET,
                v->samples[i].jp ? v->samples[i].jp : ""
            );
            printf(
                " " CL_PRIMARY " | " RESET
                CL_DIM " └ " RESET
                CL_VN_SENT "%s\n" RESET,
                v->samples[i].vn ? v->samples[i].vn : ""
            );
        }
    }
    printf(RESET "\n");
}

void exactlySearchingVocab(HashTable *ht, char *key) {
    if (!ht || !key) return;
    int index = hashGetIndex(key, ht->size);
    HashNode *current = ht->table[index];
    int found = 0;
    while (current != NULL) {
        if (strcmp(current->key, key) == 0) {
            printOutVocab(current->vocab);
            found = 1;
        }
        current = current->next;
    }
    if (!found) {

```

```

        printf(" " CL_WARN "⚠ Không tìm thấy kết quả cho từ khóa: " CL_LOGO "%s\n"
RESET, key);
    }
}

```

FILE: src/vocab_search_exact.h

```

#ifndef vocab_search_exact_H
#define vocab_search_exact_H
#include "../data_structures.h"
#define HASH_TABLE_SIZE 10007
struct HashNodeForVocab {
    char *key;
    Vocab *vocab;
    struct HashNodeForVocab *next;
};
struct HashTableForVocab {
    struct HashNodeForVocab **table;
    int size;
    int count;
};
typedef struct HashNodeForVocab HashNode;
typedef struct HashTableForVocab HashTable;
HashTable* createHashTable(int size);
void buildHashTableForVocab(KanjiList *L, HashTable *ht);
void exactlySearchingVocab(HashTable *ht, char *key);
#endif

```